



US012411946B2

(12) **United States Patent**
Kim

(10) **Patent No.:** **US 12,411,946 B2**
(45) **Date of Patent:** **Sep. 9, 2025**

(54) **CYBER THREAT INFORMATION PROCESSING APPARATUS, CYBER THREAT INFORMATION PROCESSING METHOD, AND STORAGE MEDIUM STORING CYBER THREAT INFORMATION PROCESSING PROGRAM**

(71) Applicant: **SANDS LAB INC.**, Seoul (KR)

(72) Inventor: **Ki Hong Kim**, Seoul (KR)

(73) Assignee: **SANDS LAB INC.**, Seoul (KR)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 228 days.

(21) Appl. No.: **18/138,447**

(22) Filed: **Apr. 24, 2023**

(65) **Prior Publication Data**

US 2024/0346135 A1 Oct. 17, 2024

(30) **Foreign Application Priority Data**

Apr. 12, 2023 (KR) 10-2023-0047990

(51) **Int. Cl.**
G06F 21/55 (2013.01)
G06F 21/57 (2013.01)

(52) **U.S. Cl.**
CPC **G06F 21/554** (2013.01); **G06F 21/552** (2013.01); **G06F 21/577** (2013.01); **G06F 2221/034** (2013.01)

(58) **Field of Classification Search**
CPC G06F 21/554; G06F 21/552; G06F 21/577; G06F 2221/034
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

11,443,045 B2 *	9/2022	Rahnama-Moghaddam		G06F 21/577
2020/0358829 A1 *	11/2020	Vo		G06F 40/30
2021/0097176 A1 *	4/2021	Mathews		G06N 5/01
2022/0207142 A1 *	6/2022	Gupta		G06F 21/566
2022/0309166 A1 *	9/2022	Shenoy		G06F 21/577
2023/0009127 A1 *	1/2023	Boyer		G06F 21/563
2023/0038196 A1 *	2/2023	Labreche		G06F 21/577
2023/0179622 A1 *	6/2023	Underwood		H04L 41/14
				726/25
2024/0056458 A1 *	2/2024	Lee		G06F 40/58
2024/0070261 A1 *	2/2024	Bin Huraib		G06F 21/566
2024/0070273 A1 *	2/2024	Almogbil		G06N 3/09

(Continued)

FOREIGN PATENT DOCUMENTS

KR 10-2447279 B1 9/2022

OTHER PUBLICATIONS

Edwin K., "Track Emerging Threats With Feedly AI", Threat Intelligence, Ransomware Attacks, Mar. 2022, pp. 1-13.

(Continued)

Primary Examiner — Linglan Edwards

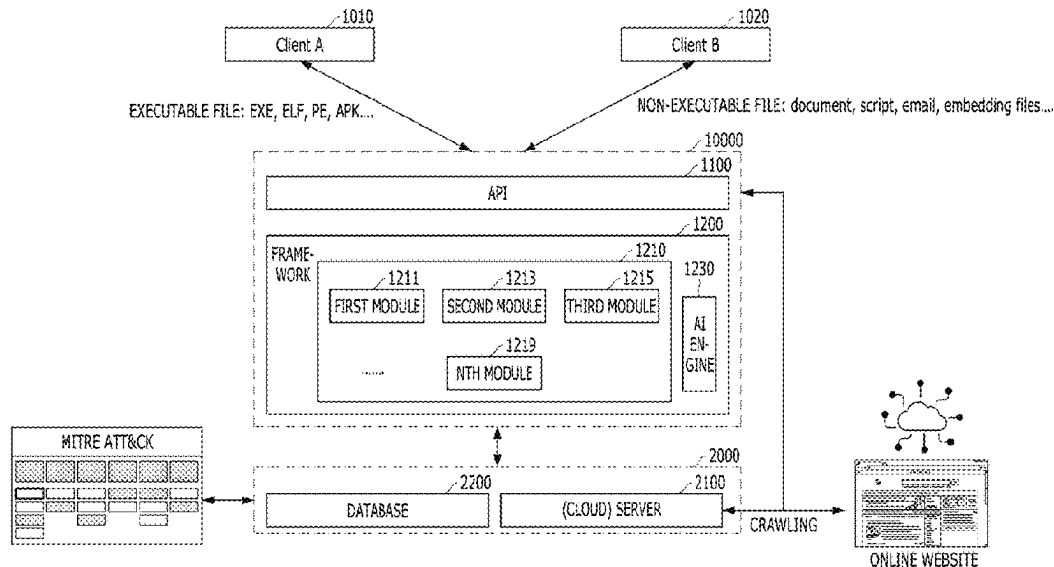
Assistant Examiner — Kamryn J Gillespie

(74) *Attorney, Agent, or Firm* — Morgan, Lewis & Bockius LLP

(57) **ABSTRACT**

Provided is a cyber threat information processing method including receiving input of a file or information on the file from a user through at least one interface; processing cyber threat information related to the received or input file or the information on the file; providing the processed cyber threat information to the user through a user interface; and performing natural language processing on the processed cyber threat information.

6 Claims, 92 Drawing Sheets



(56)

References Cited

U.S. PATENT DOCUMENTS

2024/0107344 A1 * 3/2024 Papanikitas G06F 11/3409
2024/0411896 A1 * 12/2024 Myers G06F 21/552

OTHER PUBLICATIONS

Office Action issued in corresponding Korean Patent Application
No. 10-2023-0047990, dated Feb. 5, 2025.

* cited by examiner

FIG. 1

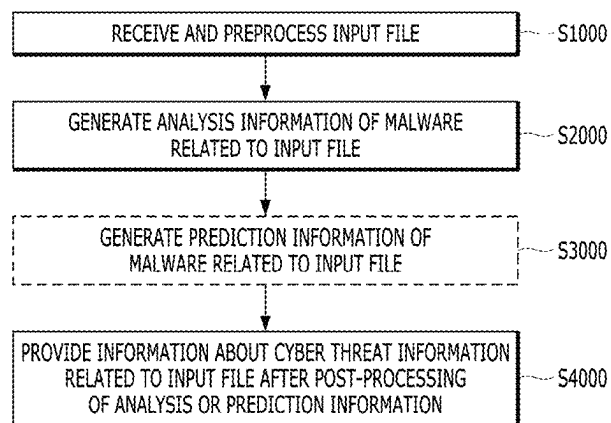


FIG. 2

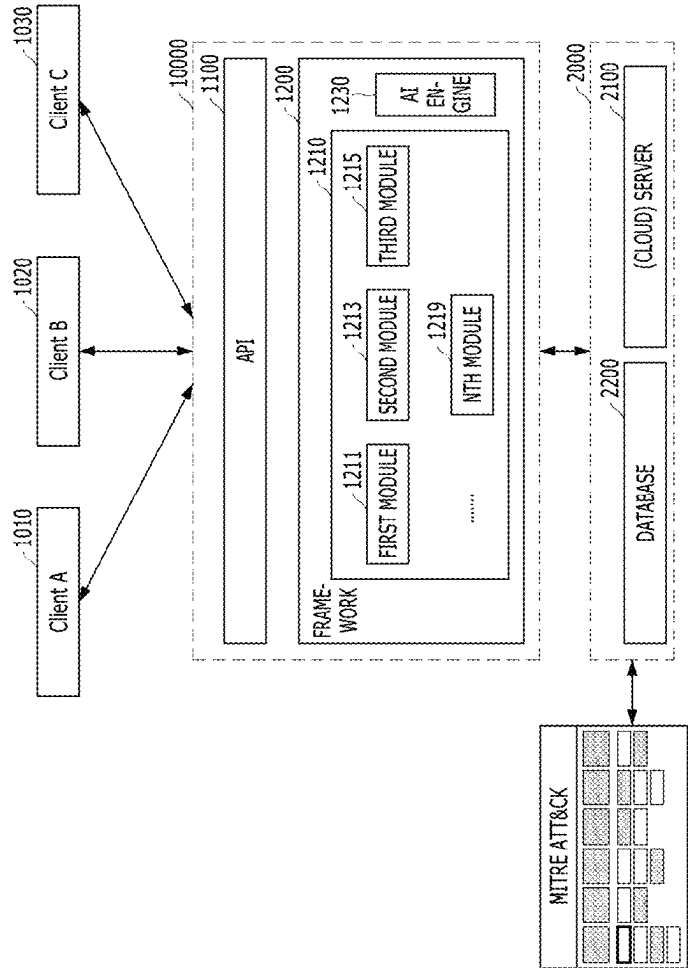


FIG. 3

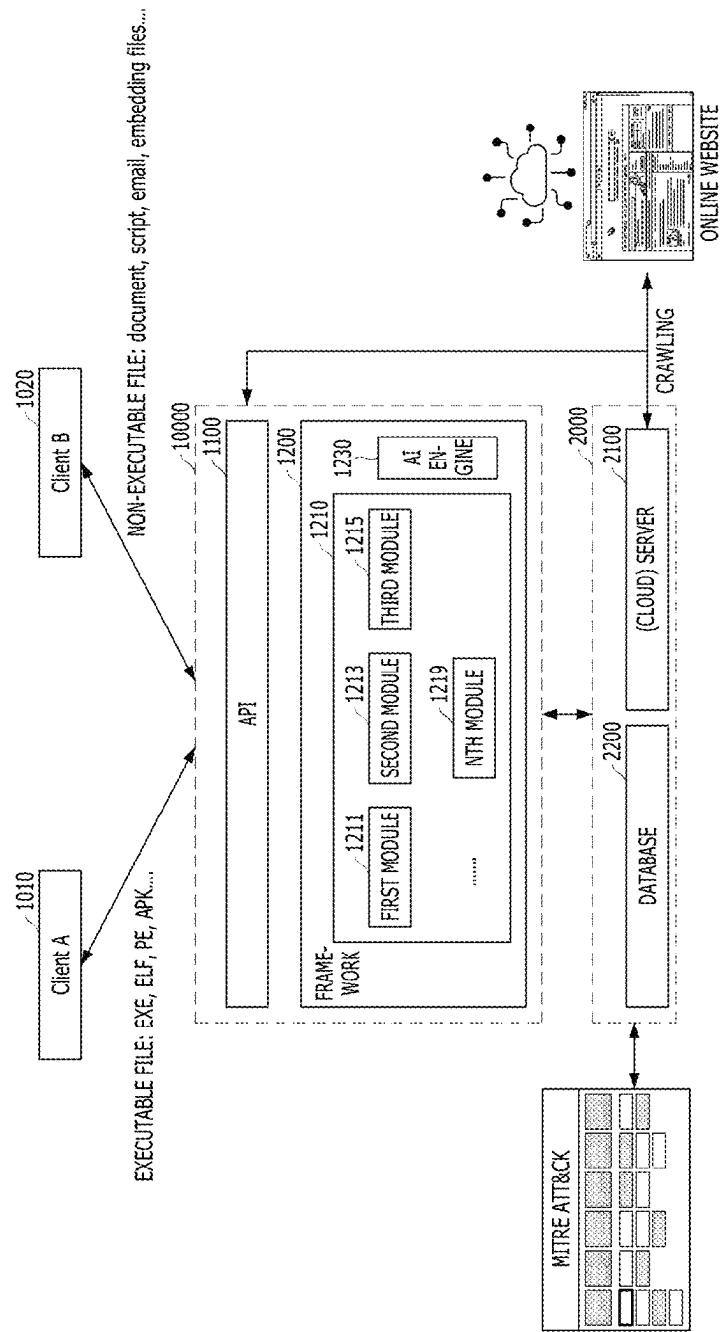


FIG. 4

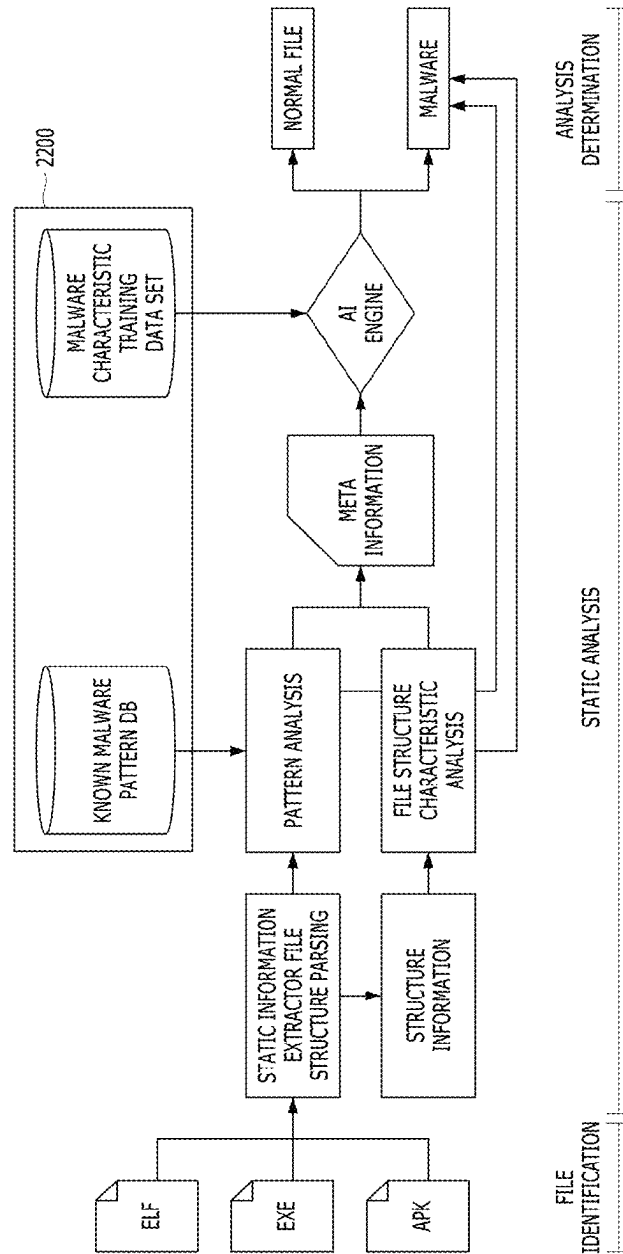


FIG. 5

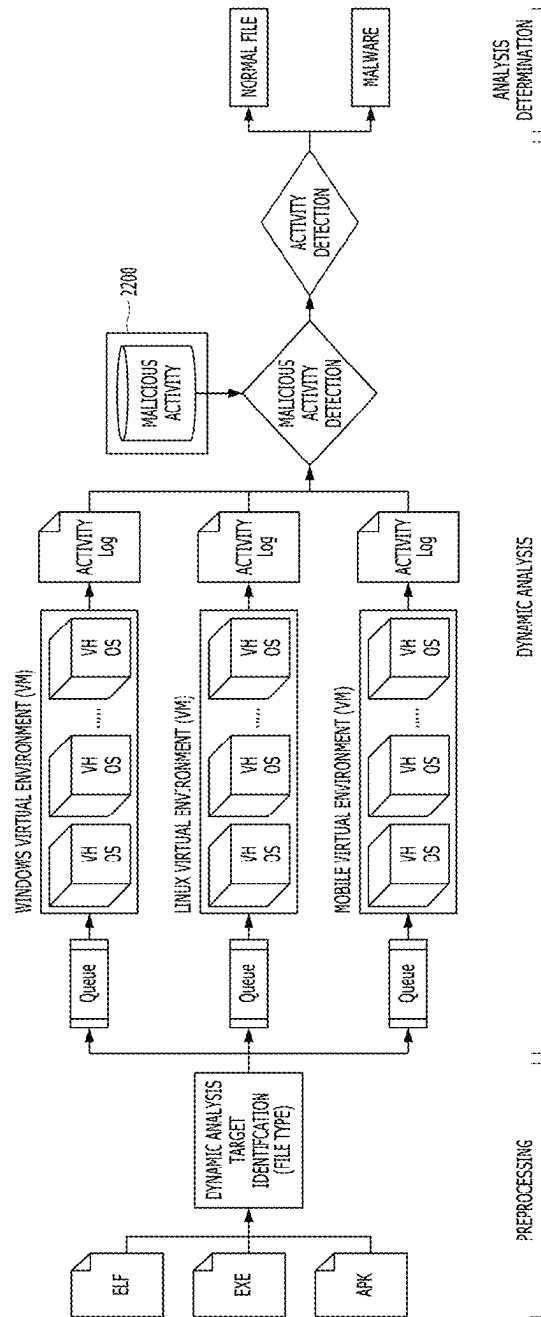
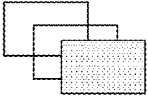


FIG. 6



EXE FILE



Disassembled code set

ATTACK TECHNIQUE
IDENTIFIER CODE
Technique/Tactic ID

FILE	OP CODE	T-ID	DESCRIPTION
malware.exe	MOV DWORD PTR SS: [EBP-4], 1 MOV DWORD PTR SS: [EBP-8], 2 MOV EDX, DWORD PTR SS: [EBP-8] LEA EAX, DWORD PTR SS: [EBP-4]	1022	CHANGE SYSTEM MAJOR REGISTRY
	PUSH EBP MOV EBP, ESP SUB ESP, 18 AND ESP, FFFFFFF0 MOV EAX, 0	1077	REGISTER STARTUP PROGRAM
	LEA EAX, DWORD PTR SS: [EBP-4] MOV EDX, DWORD PTR SS: [EBP-8] LEA EAX, DWORD PTR SS: [EBP-4] LEA EAX, DWORD PTR SS: [EBP-4]	1034	TURN OFF WINDOWS FIREWALL
	PUSH EBP MOV EBP, ESP MOV EAX, DWORD PTR SS: [EBP+B] ADD EAX, DWORD PTR SS: [EBP+C] POP EBP RETN	1090	ADD NEW USER
	CMP DWORD PTR SS: [EBP-4], 2 JNZ SHORT if. 00401035 PUSH if. 0040C008 CALL if.printf ADD ESP, 4 JMP SHORT if. 00401042	2011	CREATE BACKDOOR
	CMP DWORD PTR SS: [EBP-B], 1 JE SHORT switch. 00401027 CMP DWORD PTR SS: [EBP-B], 2 JE SHORT switch. 00401036 CMP DWORD PTR SS: [EBP-B], 3 JE SHORT switch. 00401045 JMP SHORT switch. 00401054	3744	SUSPEND OPERATION OF SECURITY PROGRAM

FIG. 7

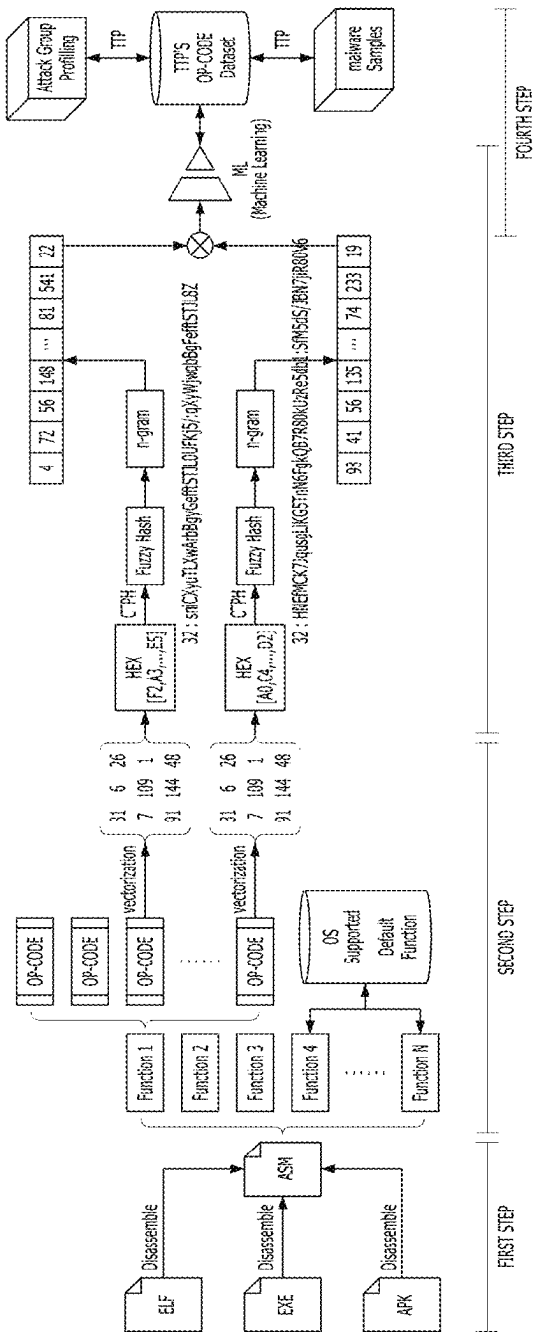


FIG. 8

OP-CODE	OP-CODE → CRC-16	ASM-CODE	ASM-CODE → CRC-32
push	0x45E9	55	0xC9034AF6
mov	0x10E3	8B EC	0x3012FD2C
lea	0xAACE	8D 45 0C	0x9214A6AA
push	0x45E9	50	0xB969BE79
push	0x45E9	6A 00	0xDECB91D2
push	0x45E9	FF 75 08	0x1D4AE87E
push	0x45E9	6A 01	0xA9CCA144
call	0x09F8	FF 15 B0 20 40 00	0x284055E2
pop	0x7117	59	0xC0B506DD
push	0x45E9	50	0xB969BE79
call	0x09F8	E8 AB FF FF FF	0x4452C315
add	0x8B0B	83 C4 10	0xAE137EE5
pop	0x7117	5D	0xC7D8C2C4
retn	0x12B3	C3	0xD06F7C87

FIG. 9

OP-CODE Vector	ASM-CODE Vector
17897	201 3 74 246
4323	48 18 253 44
43726	146 20 166 170
17897	185 105 121 44
17897	222 203 145 210
17897	29 74 232 126
17897	169 204 161 68
2552	40 64 85 226
28951	192 181 6 221
17897	185 105 190 121
2552	68 82 195 21
35595	174 19 126 229
28951	199 216 194 196
4787	208 111 124 135

FIG. 11

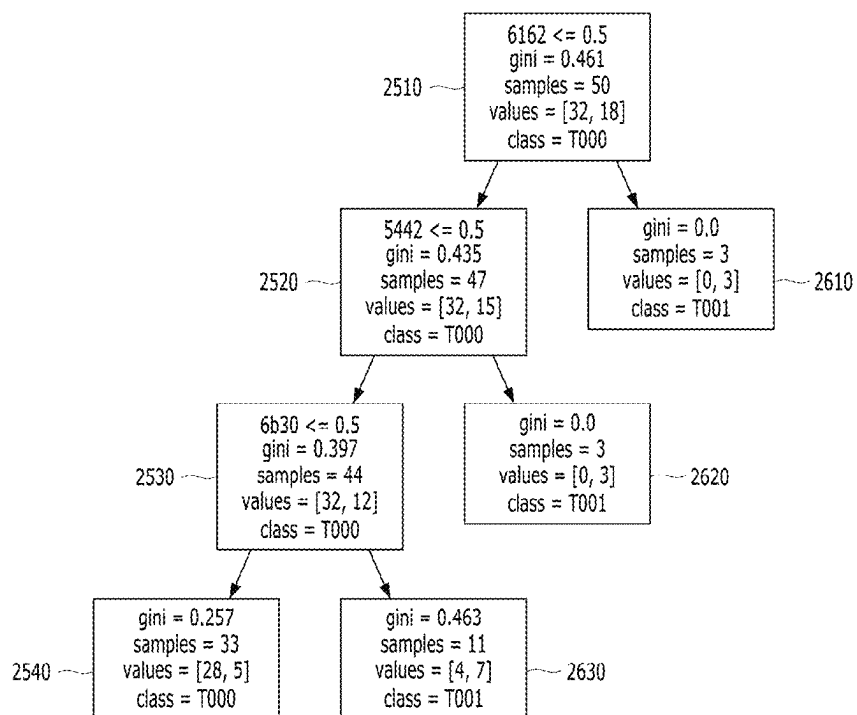


FIG. 12

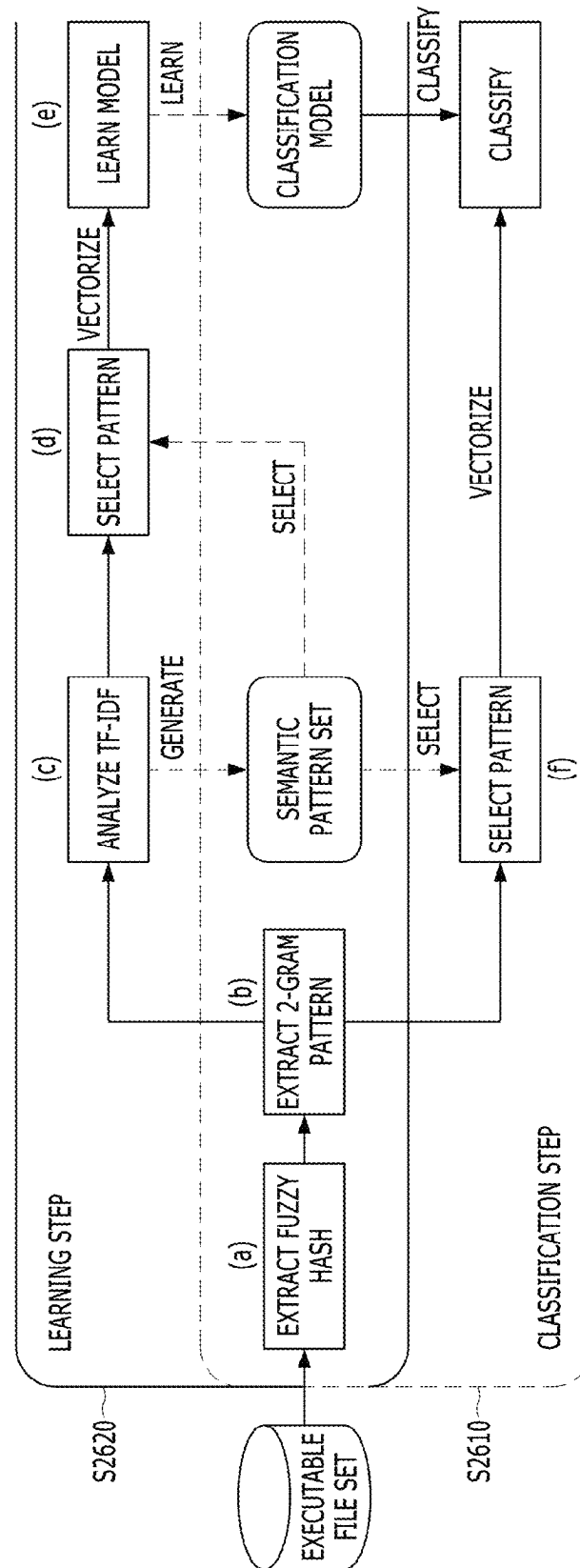


FIG. 13

Label		SHA-256 (size)	N-gram
T-ID	Attacker (or Group)	(OP-CODE + ASM-CODE)'s FUZZY HASH	
T10XX	TA504	(OP-CODE + ASM-CODE)'s	{ "3736":1, "3645":1, "4563":1, "6344":1, "4472":1, "7255":1
		32 : 76EcDrURBPQk58t...	
	TA504	E3EC2AA04AFECC6...	{ "3736":1, "3645":1, "4563":1, "6344":1, "4472":1, "7255":1, "5552":1, "5242":1, "4250":1, }
		32 : 76EcDrURBPQk58tGcoHE...	

FIG. 14

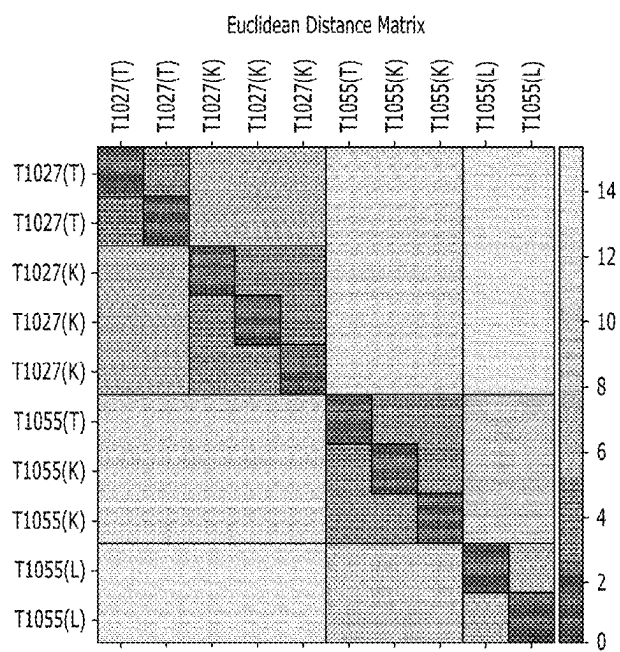


FIG. 15

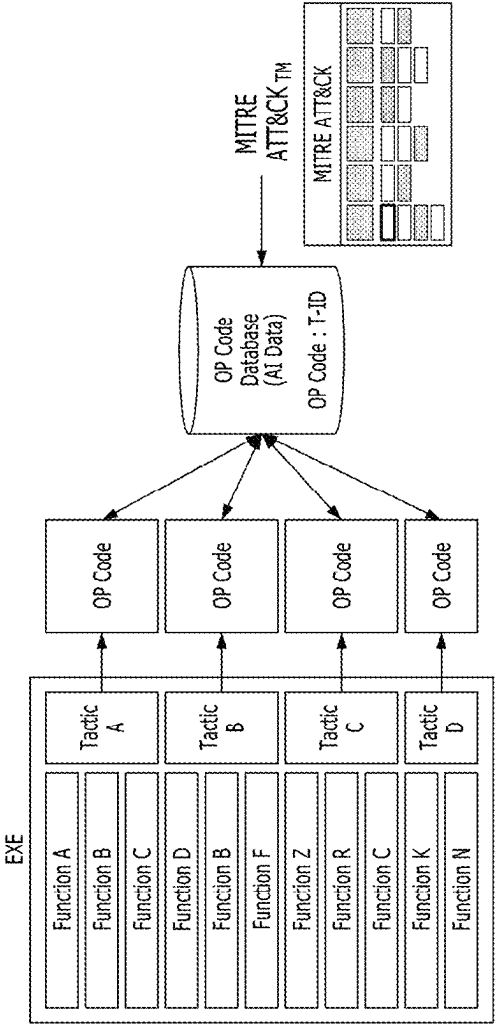


FIG. 16

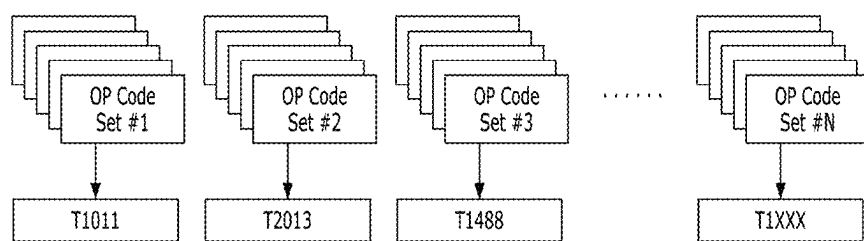


FIG. 17

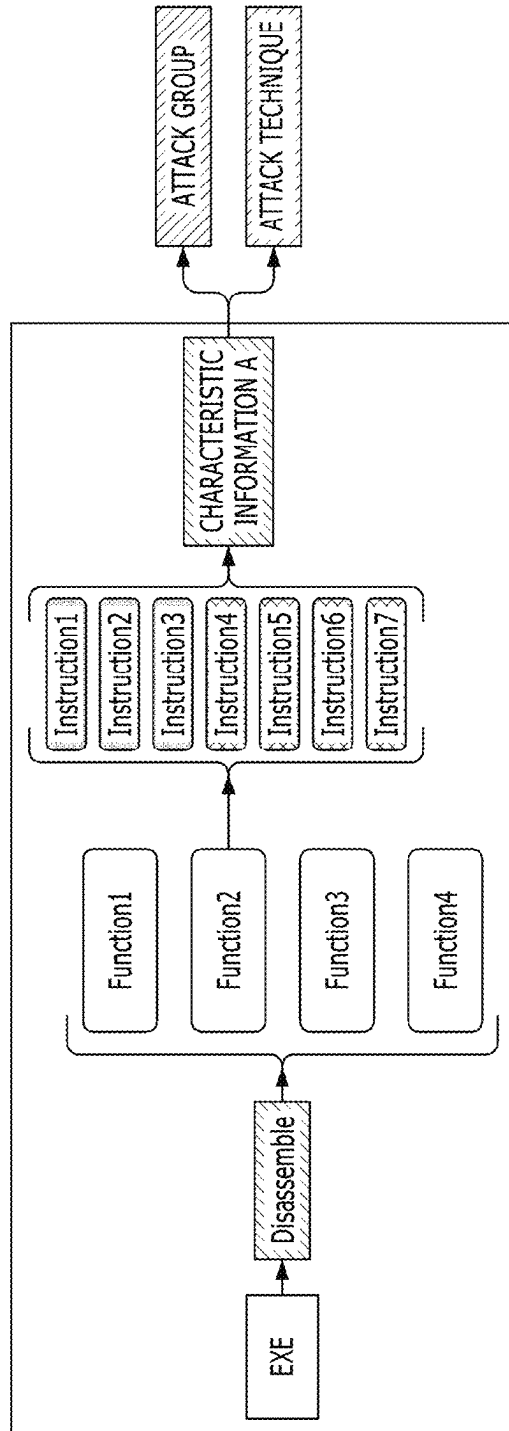


FIG. 18

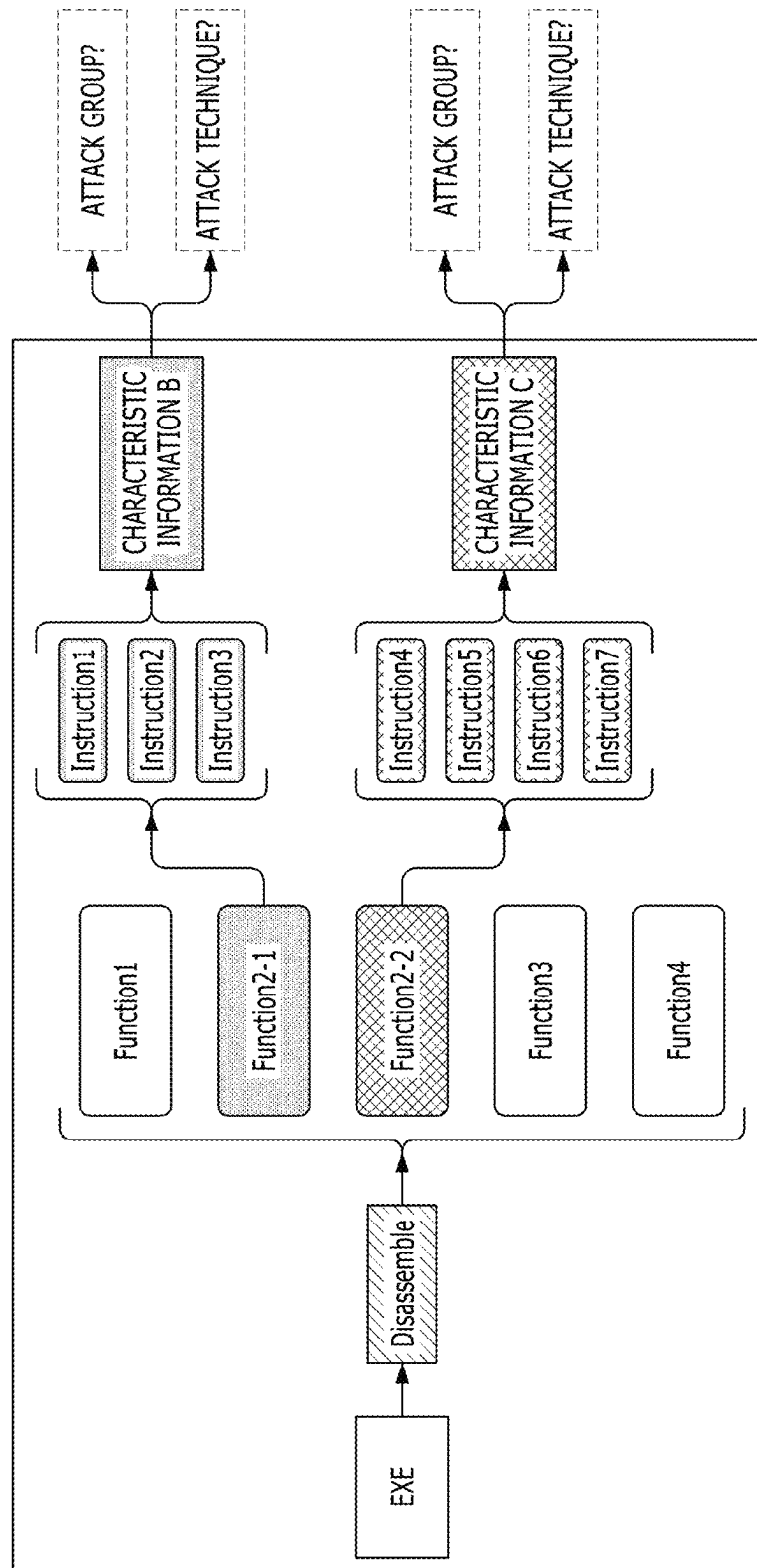


FIG. 19

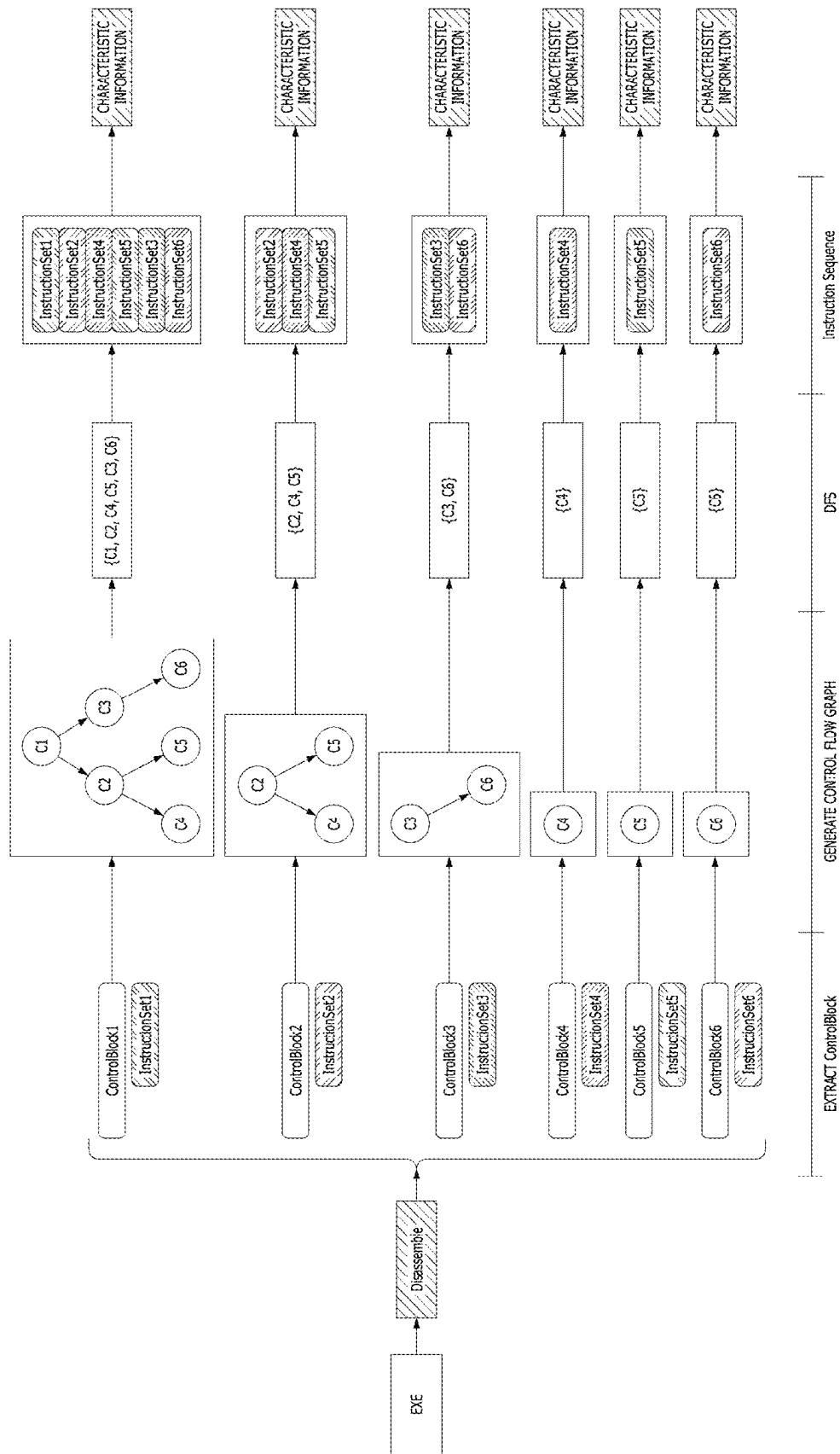


FIG. 20

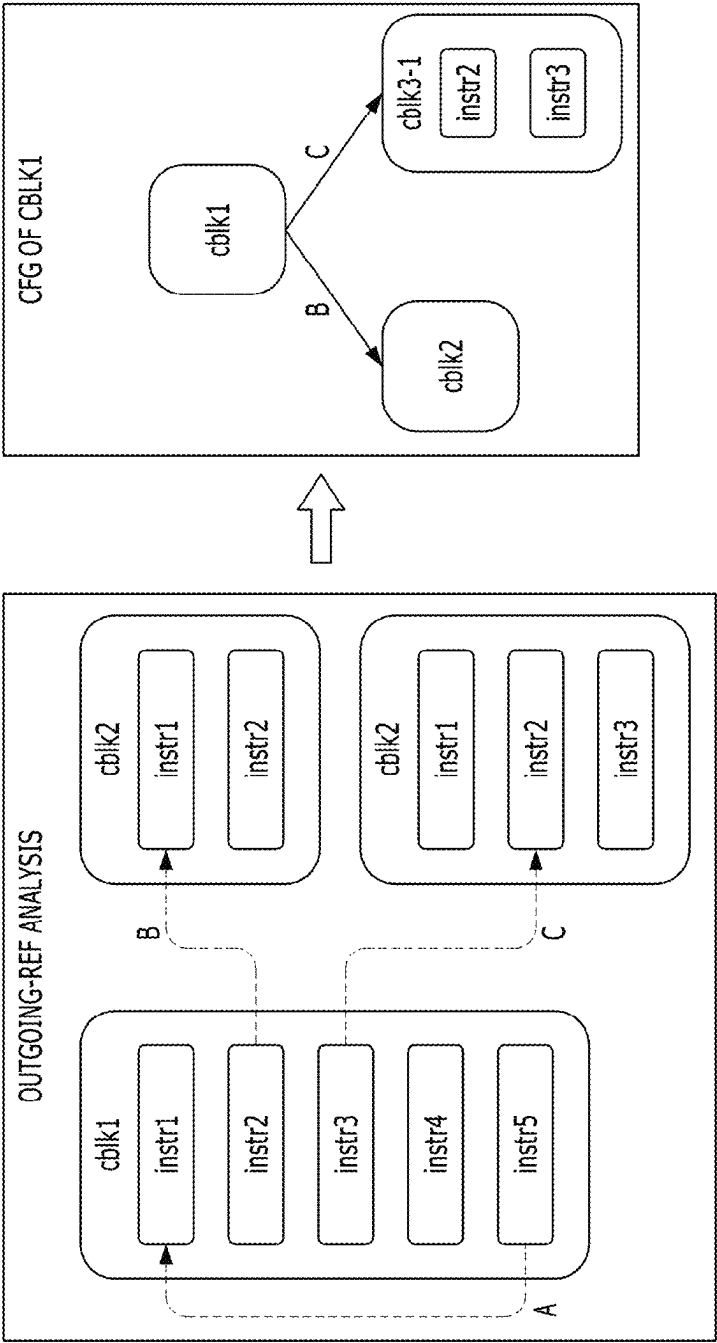


FIG. 24

00000000100003f40 <_foo1>:	
100003f40: 55	pushq %rbp
100003f41: 48 89 e5	movq %rsp, %rbp
100003f44: bf 02 00 00 00	movl \$2, %edi
100003f49: e8 a2 ff ff ff	callq 0x100003ef0 <_foo_out>
100003f4e: bf 09 00 00 00	movl \$9, %edi
100003f53: e8 98 ff ff ff	callq 0x100003ef0 <_foo_out>
100003f58: 5d	popq %rbp
100003f59: c3	retq
100003f5a: 66 0f 1f 44 00 00	nopw (%rax,%rax)

FIG. 25

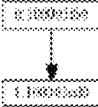
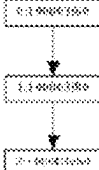
	app1	app2	app3
CFG			
Instruction Sequence	movl movl movl movl cmpl movl movl movl imull leaq movb movl addl movl movl movl cmpl movl movl imull leaq movb movl addl movl	movl movl movl cmpl movl movl movl imull leaq movb movl addl movl movl movl cmpl movl movl imull leaq movb movl addl movl	movl movl movl cmpl movl movl movl imull leaq movb movl addl movl movl cmpl movl movl imull leaq movb movl addl movl
Fuzzy hash	3:55Xv8t8733x8v8t873:1A:Ad	3:55Xv8t8733x8v8t873:1B:Ad	3:55Xv8t8733x8v8t873:1C:Ad

FIG. 26

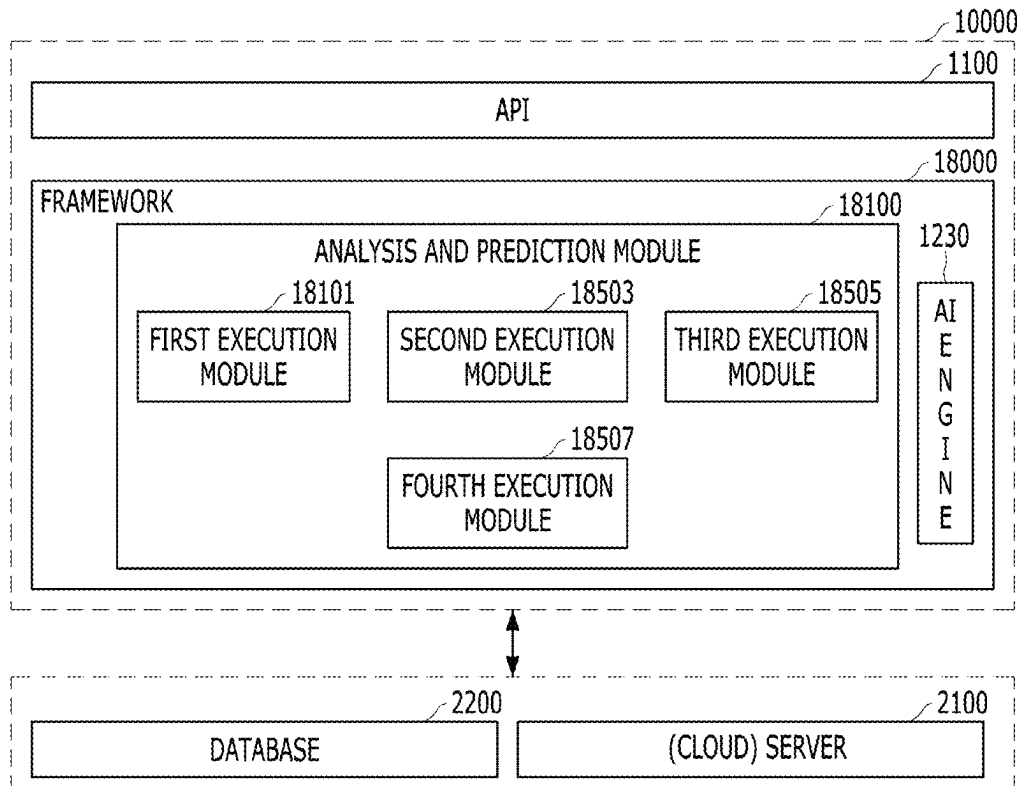


FIG. 27

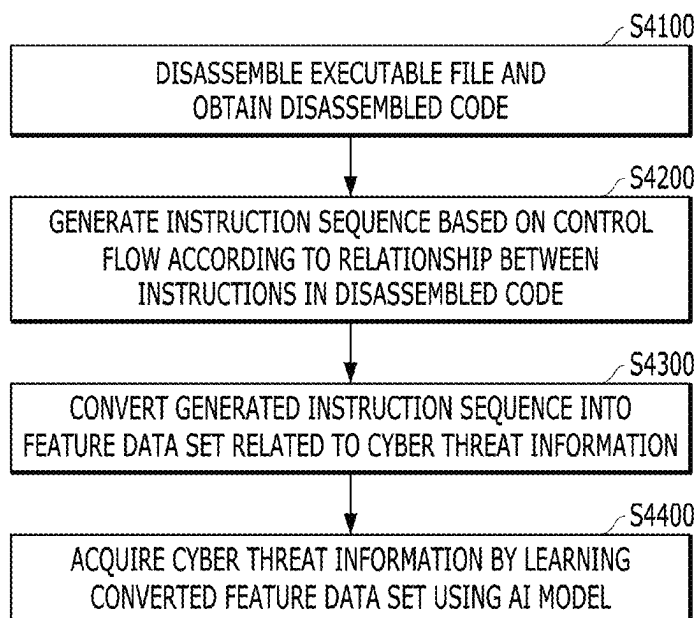


FIG. 28

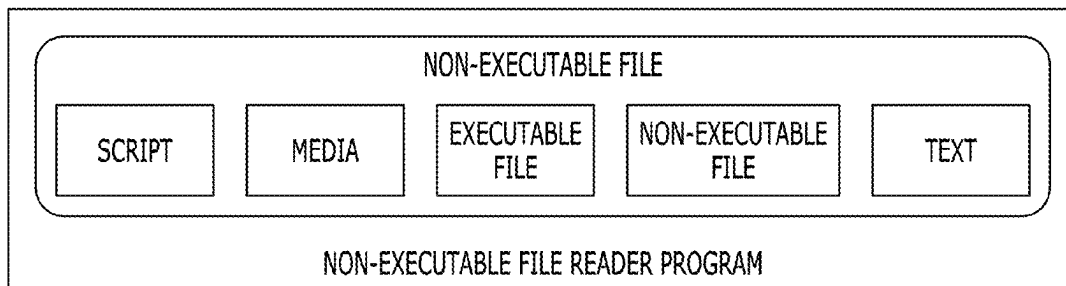


FIG. 29

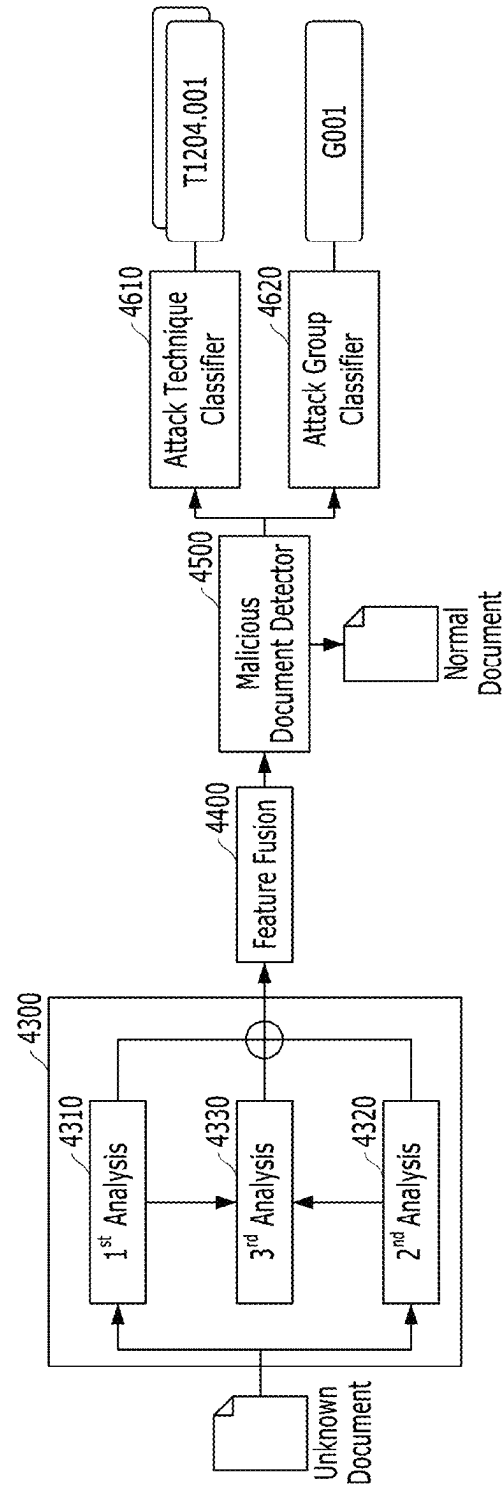


FIG. 30

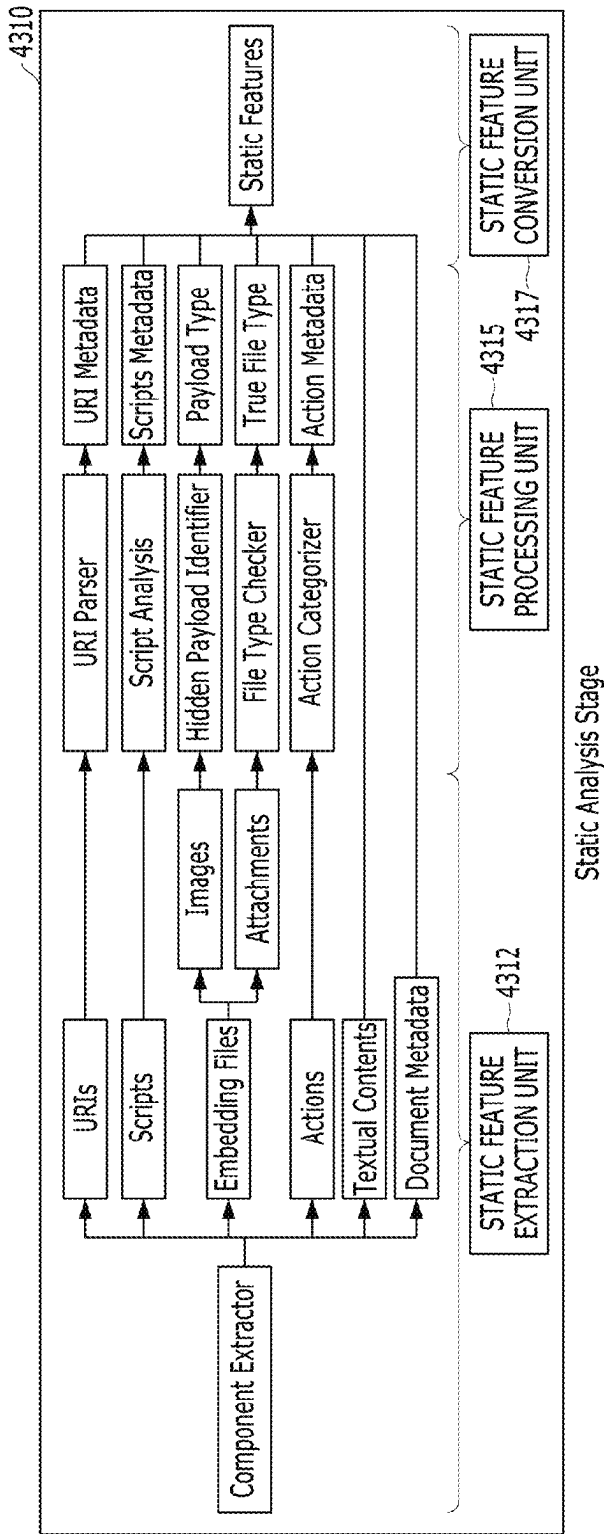


FIG. 31

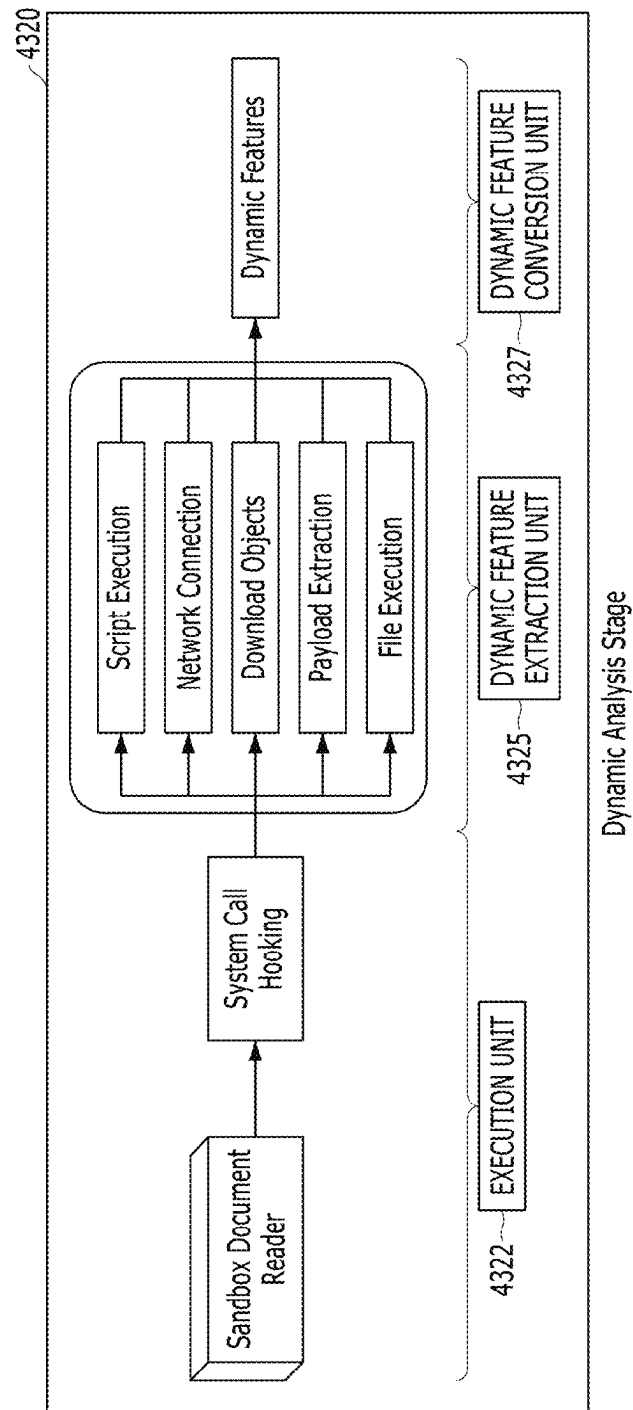


FIG. 32

DIVISION	CATEGORY	SYSTEM CALL API	EXAMPLE OF EXTRACTED PARAMETER	
1	SCRIPT EXECUTION/ OPENING	• WinExec • System	COMMAND LINE COMMAND	powershell.exe -file C:\Temp\WinVbs.ps1
2	SERVER CONNECTION	• Socket • Connect	COMMUNICATION TYPE	AF_INET
			PORT NUMBER	8080
			IP ADDRESS	10.0.0.42
3	DOWNLOAD	• Send / Sendto • Recv / Recvfrom	PACKET DATA	GET /vbscript/WinVbs.ps1
4	FILE EXTRACTION	• Fopen / Fwrite • CreateFile / WriteFile	TARGET FILE PATH	C:\Temp\WinVbs2.ps1
			CREATED CONTENT	echo W connection .X
5	FILE EXECUTION/ OPENING	• CreateProcess • ShellExecute	EXECUTION PROGRAM AND TARGET FILE	C:\Program Files\Microsoft Office\root\Office16\WINWORD.EXE C:\Temp\WinVbs.docx

FIG. 33

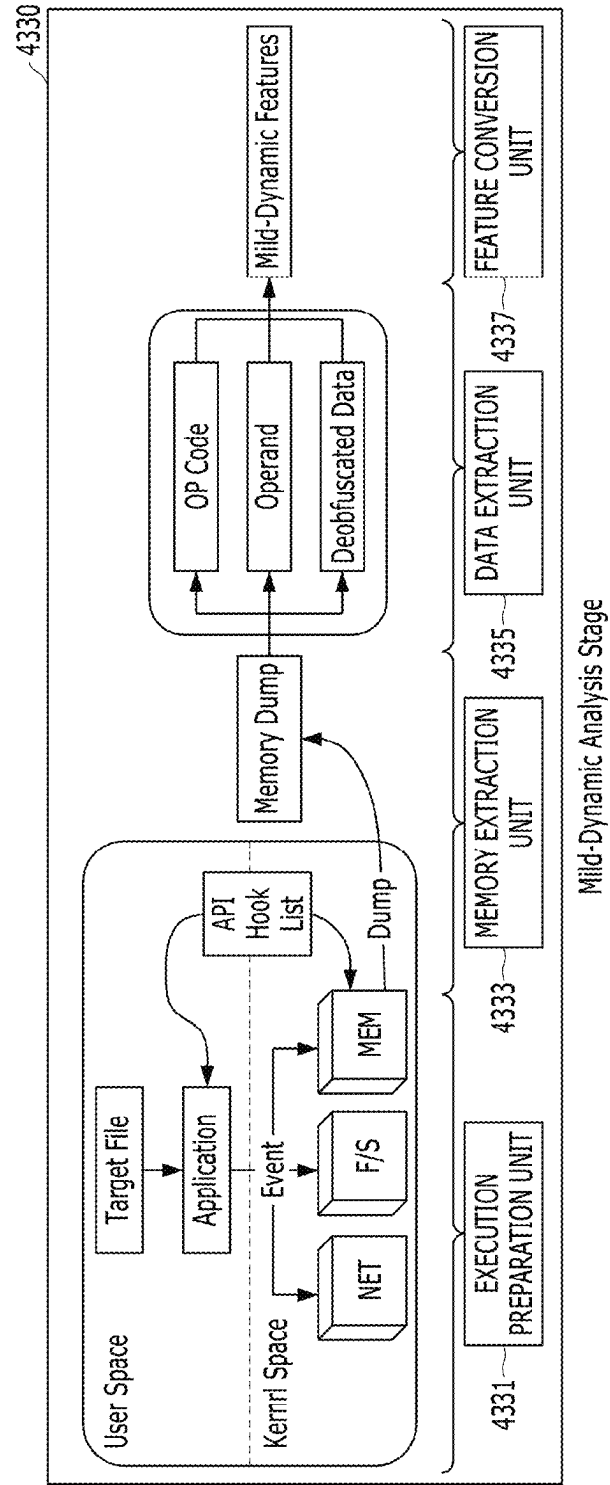
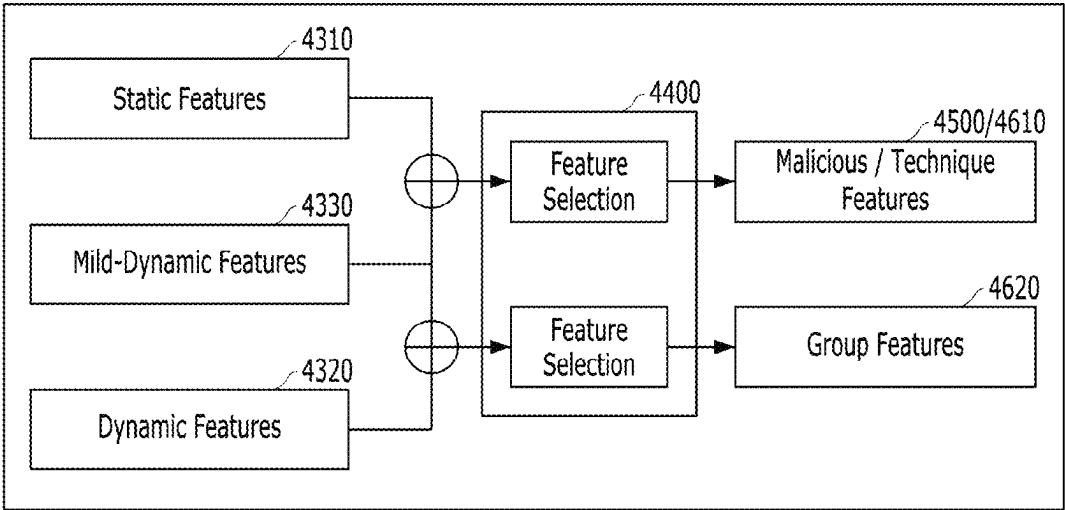


FIG. 34

DIVISION	API		
Windows OS Native API	NtQueryVirtualMemory	NtAllocateVirtualMemory	NtFreeVirtualMemory
	NtProtectVirtualMemory	NtReadVirtualMemory	NtQueryInformationProcess
	NtQueryInformationThread	NtQueryObject	NtQueryKey
	NtDuplicateObject	NtWriteFile	NtFsControlFile
	NtSetInformationFile	NtClose	NtDelayExecution
	NtWaitForSingleObject	NtOpenThread	NtResumeThread
HTML DOM Parser API	CDocument.write	CHyperlink.SetUrlComponent	GIFrameElement.CreateElement
	CWindow.AddTimeoutCode	CScriptElement.put_src	CElement.put_innerHTML
	CimgElement.put_src		
VBS Script Engine API	vbobj.StringConcat	vbobj.CreateObject	vbobj.GetObject
	vbobj.Invoke	vbobj.GetIDFromName	vbobj.CallByName
	vbobj.Shell	vbobj.Import	vbobj.Open
	vbobj.Print	vbobj.Close	

FIG. 35



Feature Fusion

FIG. 36

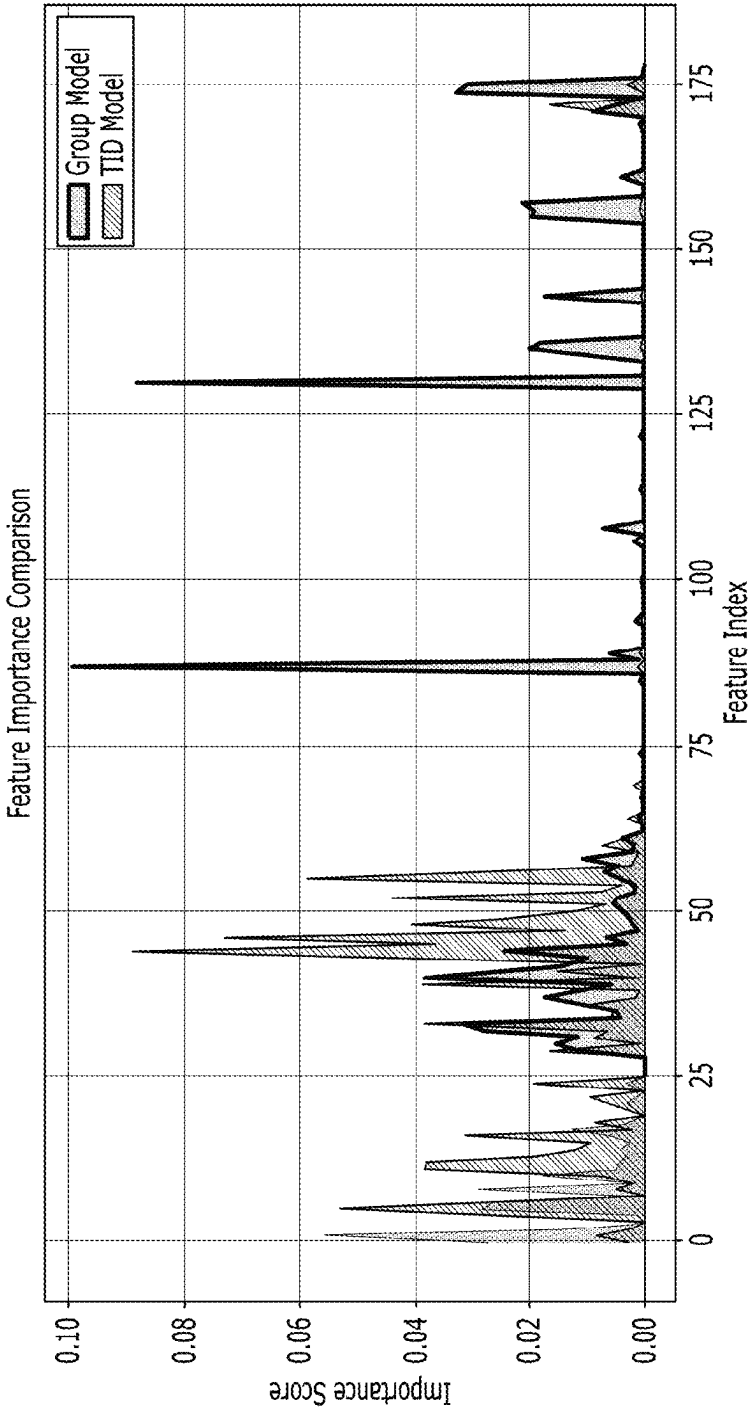


FIG. 37

DIVISION	SINGLE LABEL (Single Label)			MULTIPLE LABELS (Multi Label)	
CLASS LABEL (DEPENDENT VARIABLE)	0	T1059.005		[1, 1, 0]	T1059.005 T1564.007
OUTPUT EXAMPLE	0	T1059.005	62%	T1059.005	95%
	1	T1564.007	35%	T1564.007	95%
	2	T1204.002	2%	T1204.002	3%

FIG. 38

NO.	TECHNIQUE ID	NAME	DESCRIPTION
1	T1059.001	Command and Scripting Interpreter, PowerShell	PERFORM MALICIOUS ACTION USING POWERSHELL SCRIPT
2	T1059.003	Command and Scripting Interpreter, Windows Command Shell	PERFORM MALICIOUS ACTION USING WINDOWS COMMAND SHELL COMMAND
3	T1059.005	Command and Scripting Interpreter, Visual Basic	PERFORM MALICIOUS ACTION USING VISUAL BASIC PROGRAMMING LANGUAGE
4	T1059.007	Command and Scripting Interpreter, JavaScript	PERFORM MALICIOUS ACTION USING JAVASCRIPT
5	T1564.007	acts: VBA Stomping	ACTION OF HIDING VBA PAYLOAD DATA INSIDE DOCUMENT
6	T1204.001	ution: Malicious Link	ACTION OF INDUCING USER TO CLICK ON MALICIOUS LINK SO THAT MALICIOUS PROCEDURE IS PERFORME
7	T1204.002	ution: Malicious File	ACTION OF INDUCING USER TO CLICK ON MALICIOUS LINK SO THAT MALICIOUS PROCEDURE IS PERFORMED
8	T1027	Files ro Information	ACTION OF MAKING FILE ANALYSIS DIFFICULT BY PERFORMING COMPRESSION, ENCRYPTION, ENCODING, OBFUSCATION, ETC.
9	T1027.002	Obfuscated Files or Information : Software Packing	ACTION OF MAKING ANALYSIS DIFFICULT BY OBFUSCATING EXECUTABLE FILE
10	T1027.003	Obfuscated Files or Information : Software Packing	ACTION OF CONCEALING MALICIOUS SCRIPT OR DATA IN IMAGE, VIDEO, OR TEXT FILE
11	T1140	Deobfuscate/Decode Files or Information	ACTION OF RESTORING (DECODING) MALICIOUS PAYLOAD OBFUSCATED OR BASE64-ENCODED FOR CONCEALMENT
12	T1546.001	Event Triggered Execution: Change Default File Association	ACTION OF MODIFYING BASIC EXECUTION PROGRAM OF FILE EXTENSION
13	T11190	lic-Facing Application	ACTION OF ATTACKING USING VULNERABILITY OF APPLICATION THAT PERFORMS INTERNET COMMUNICATION (MICROSOFT EXCHANGE VULNERABILITY: CVE-2020-0688)
14	T1112	dfy Registry	ACTION OF APPROACHING AND MODIFYING REGISTRY

FIG. 40

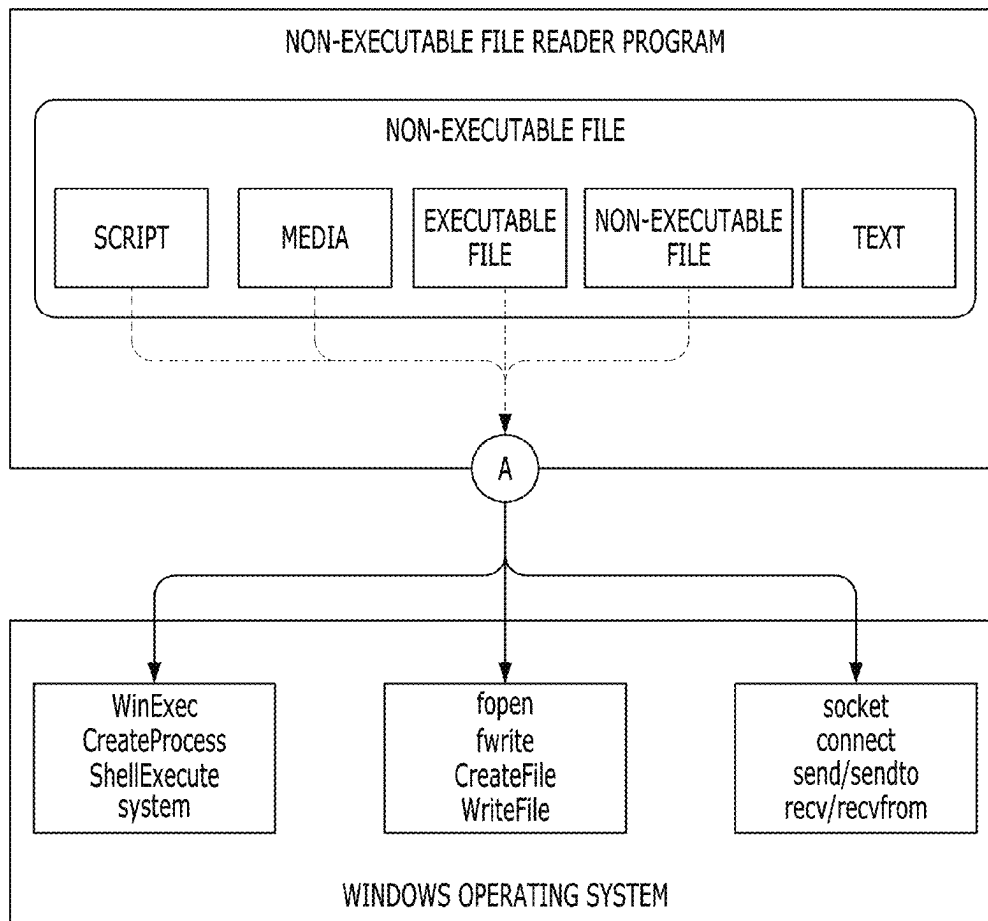


FIG. 41

```
1 | int WSAAPI send)
2 |     [in] SOCKET s,
3 |     [in] const char *buf,
4 |     [in] int len,
5 |     [in] int flags
6 | );
```

FIG. 42

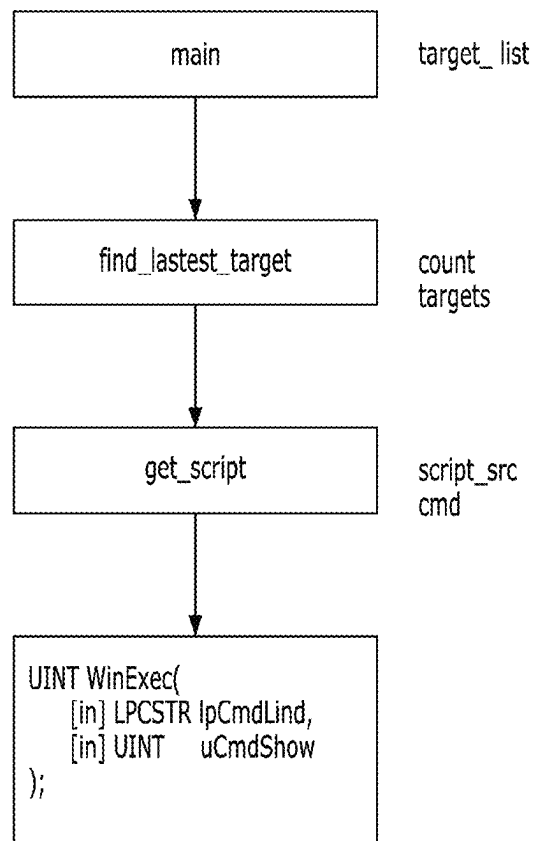


FIG. 43

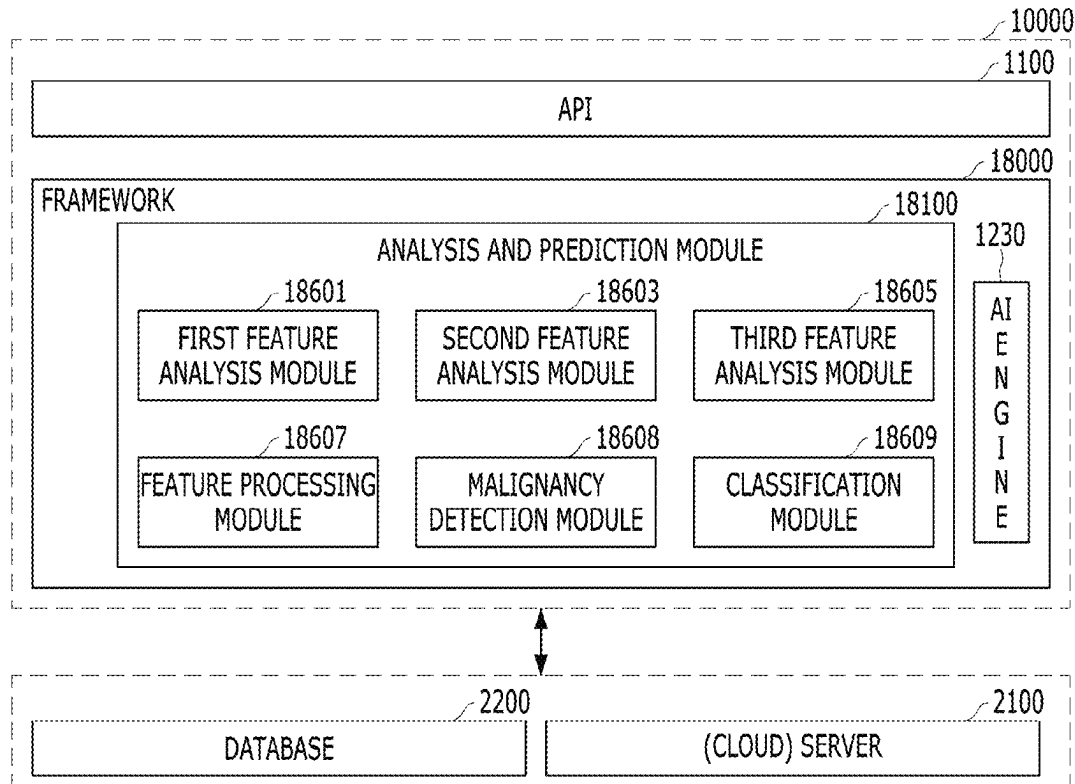


FIG. 44

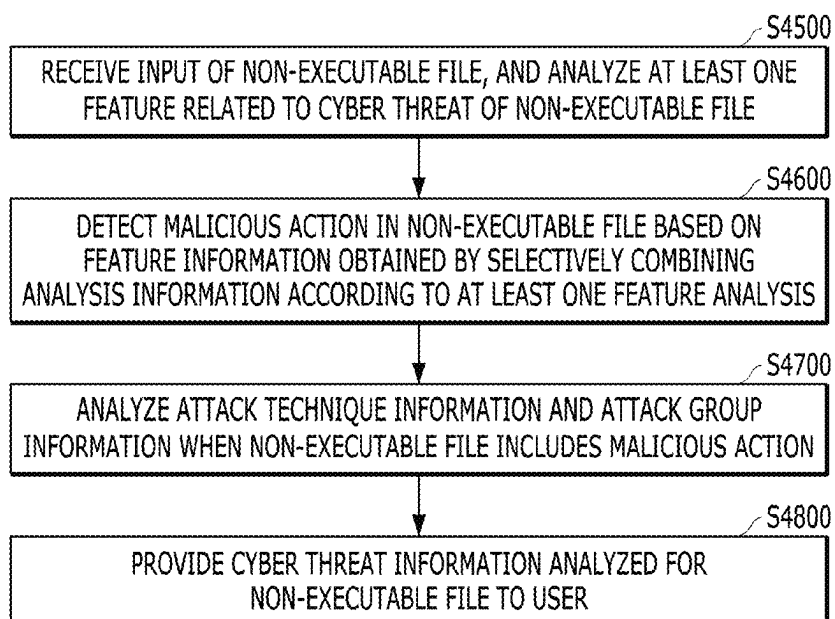


FIG. 45

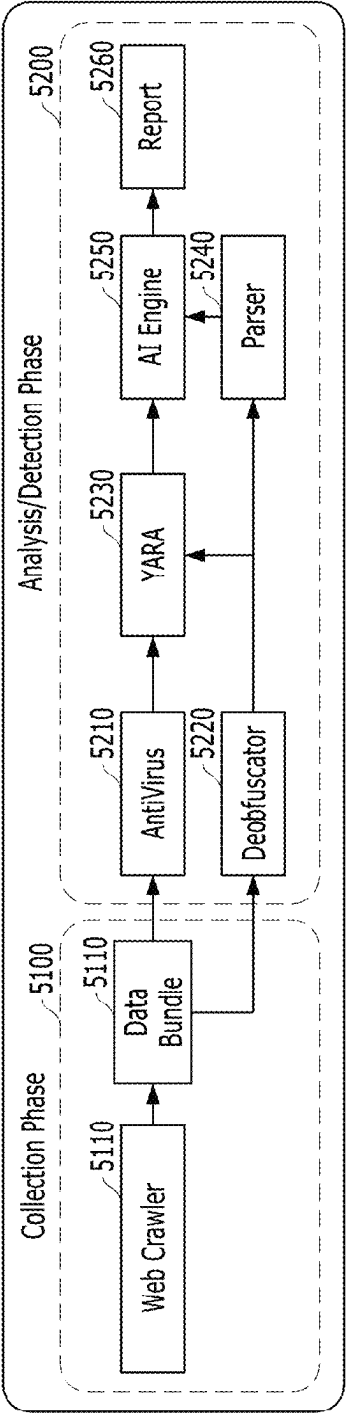


FIG. 46

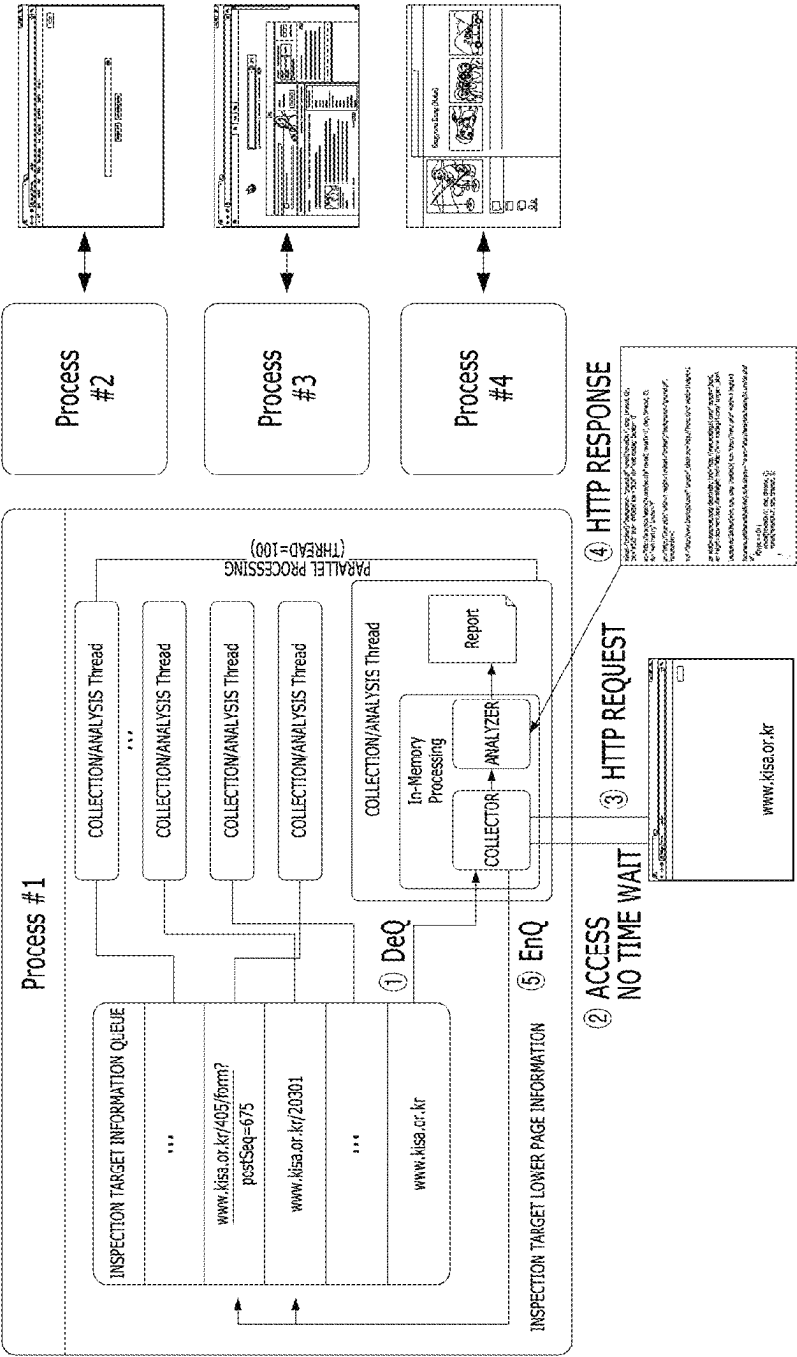


FIG. 47

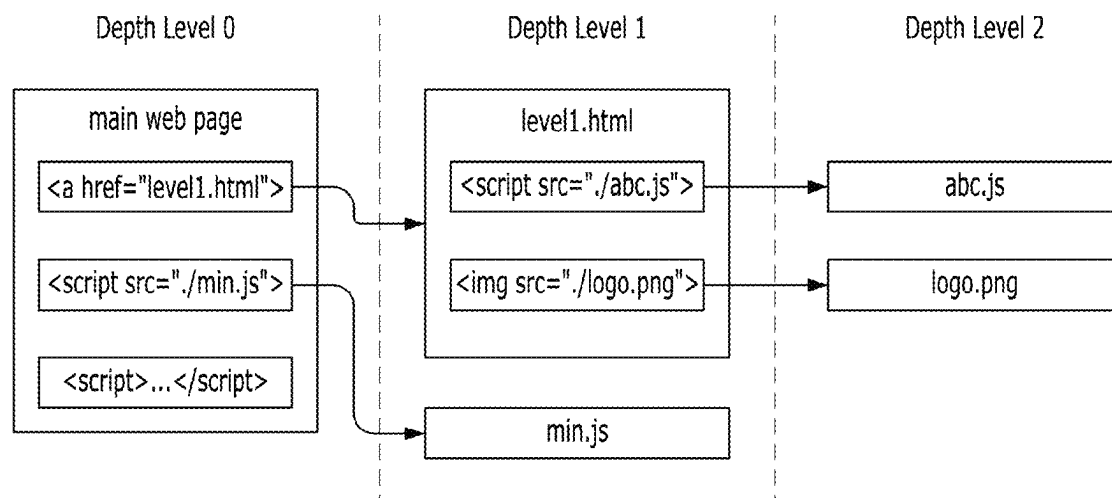


FIG. 48

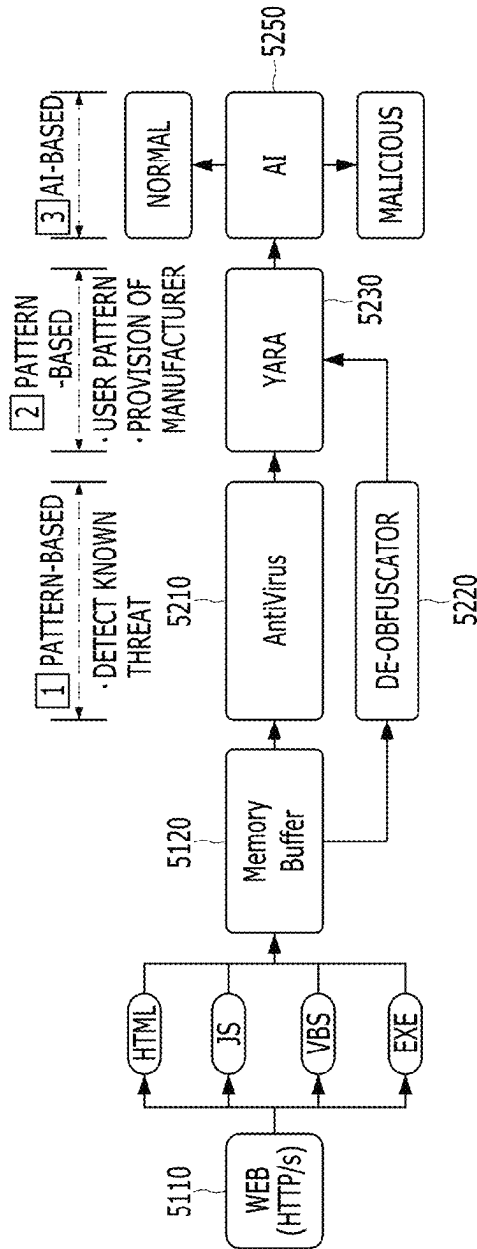
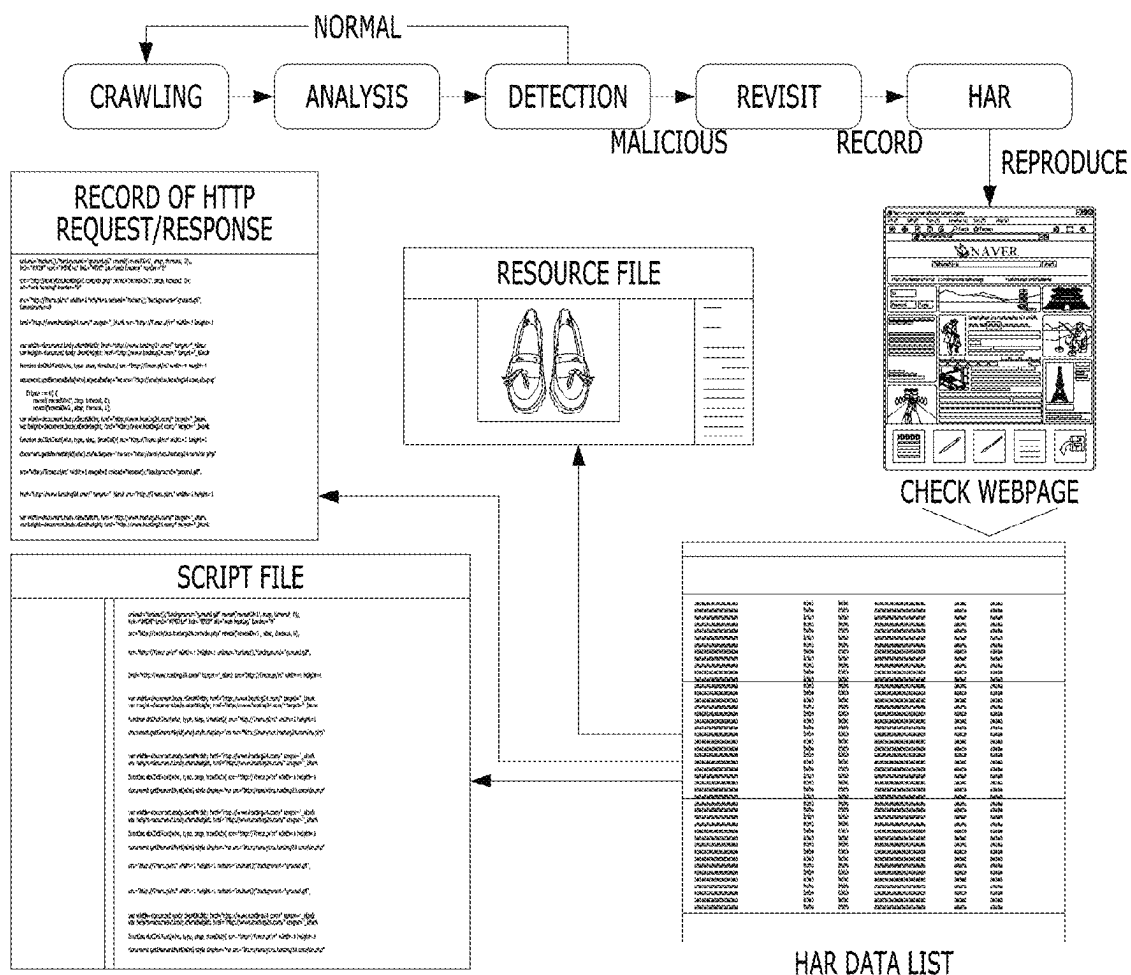


FIG. 49



※ HAR (HTTP ARCHIVE) FORMAT FILE IS FILE IN WHICH INTERACTION BETWEEN WEB BROWSER AND SITE IS LEFT AS LOG

FIG. 50

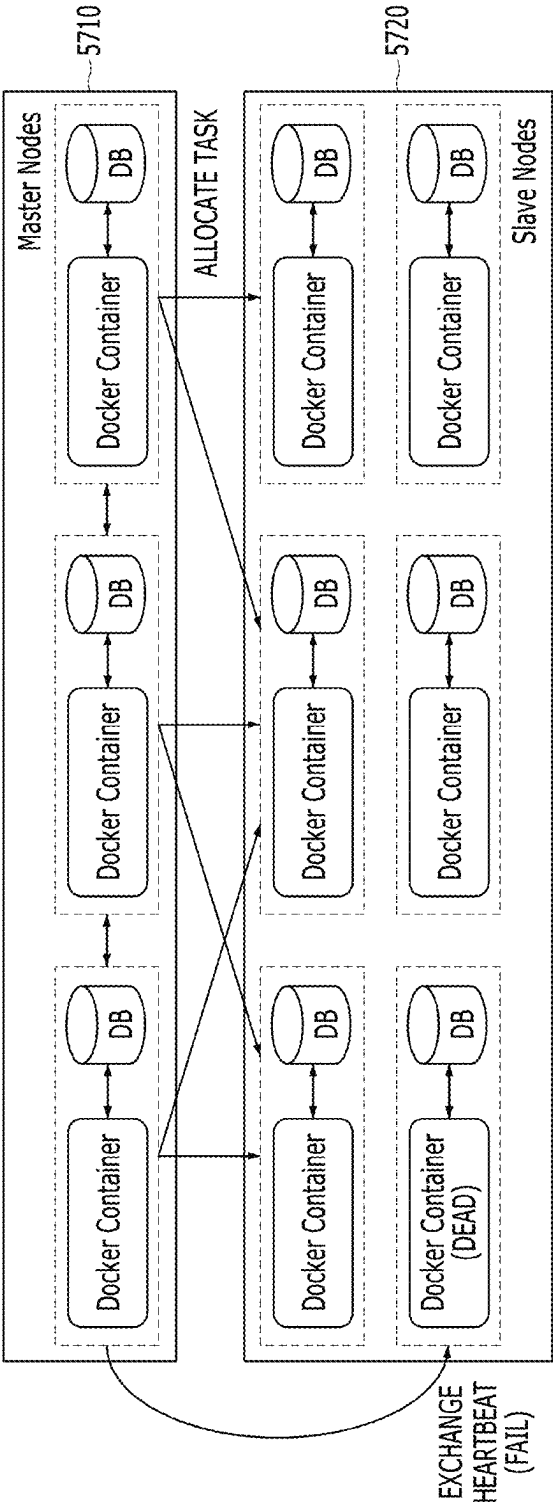


FIG. 51

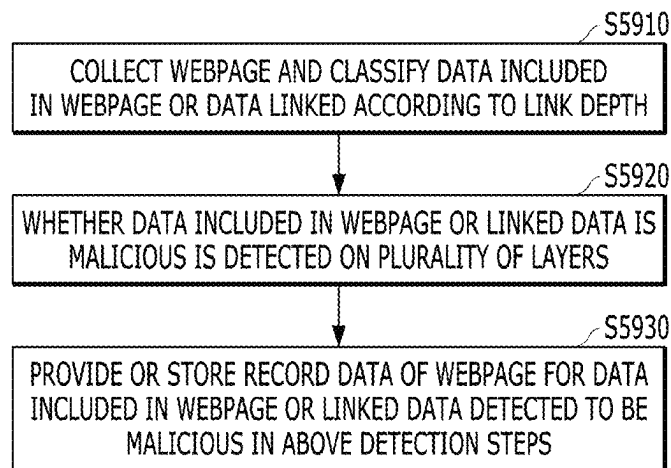


FIG. 52

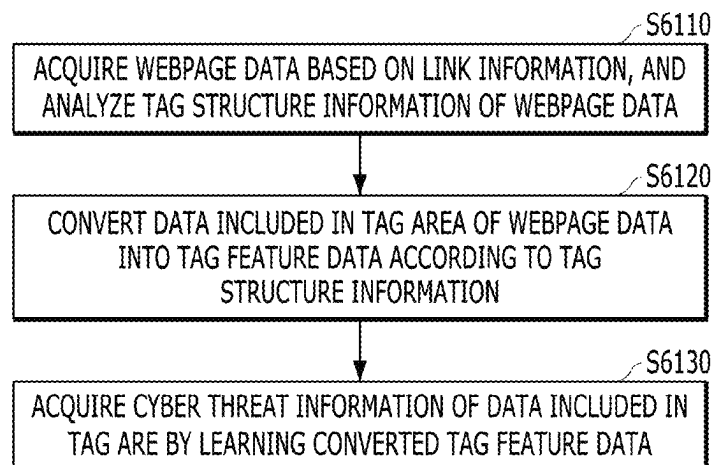


FIG. 53

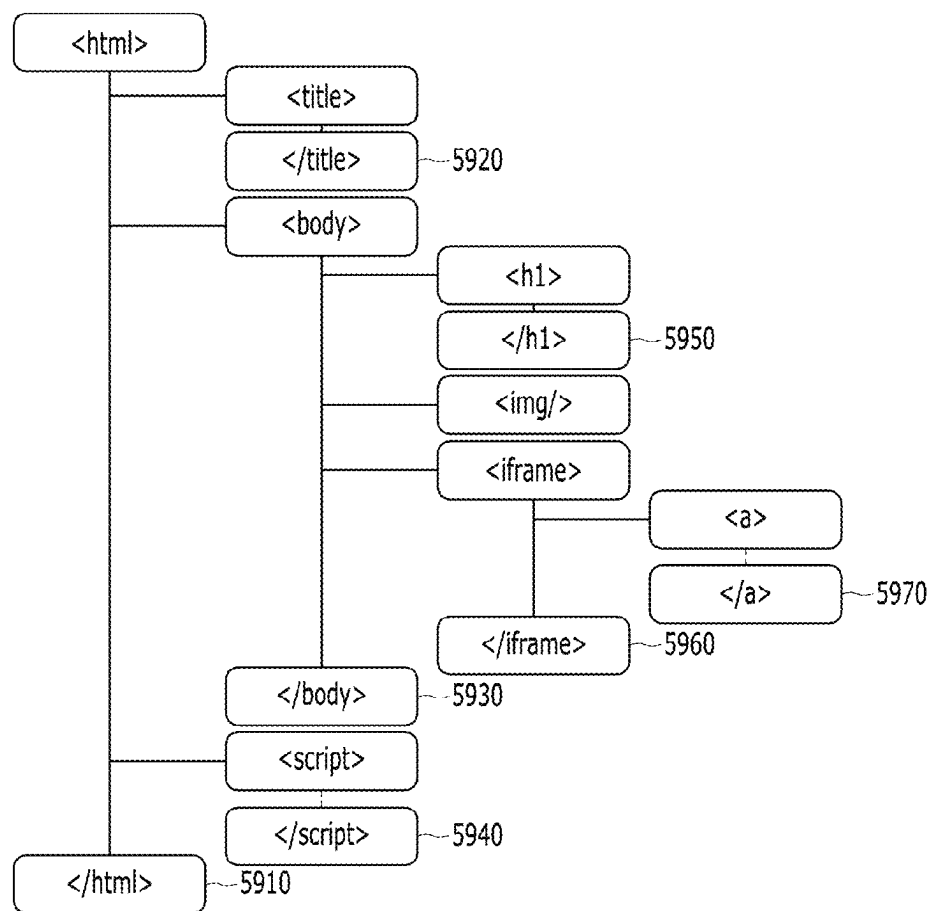


FIG. 54

Tag Data	Contents	5980
<body>	onload="teclear();"background="ground.gif", link="#ff2ff" text="#ff001e" link="#fff2ff"	
<image>	src="http://analytics.hosting24.com/do.php" alt="web hosting" border="0"	
<iframe>	src="http://Trenz.pl/rc" width=1 height=1 frameborder=0	
<a>	href="http://www.hosting24.com/" target="_blank"	
<SCRIPT>	var width=document.body.clientWidth; var height=document.body.clientHeight; function doClickText(who, type, step, timeOut){ document.getElementById(who).style.display="none"; if(type == 0) { reveal('revealDiv1', step, timeout, 0); reveal('revealDiv2', step, timeout, 1); } ... }	
</SCRIPT>		

FIG. 55

6110 Tag Data	6120 Preprocessing	6130 Fuzzy Hash	6140 TAG FEATURE DATA
<body>	onload="<func>({ background="<name>.gif", link="<hex>" text="<hex>" link="<hex>"	3:ktRXlK6G6JM6BbHIUN4bHIB4bHI1.2fbi M6Bjm4jn4jb	{'6B74': 1, '7452': 1, '526C': 1, '6C58': 1, '584C': 1, '4C63': 1, ... '346A': 2, '6A6E': 1, '6E34': 1, '6A62': 1}
<image>	src="<http><url><ext:php>" alt="<string>" border="0"	3:TYHXBZCzcfVNXc2RYHjZWxKqI7HI5on: TYHXq6n0/Hj/9n	{'5459': 2, '5948': 3, '4858': 2, '5842': 1, '425A': 1, '5A43': 1, ... '2F39': 1, '396E': 1}
<iframe>	src="<http><url>" width=1 height=1 frameborder=0	3:TYHXBZCtGPF/O+clhIALV:TYHXqqF/ nEhfh	{'5459': 2, '5948': 2, '4858': 2, '5842': 1, '425A': 1, '5A43': 1, ... '6E45': 1, '4568': 1, '6866': 1, '6668': 1}
<a>	href="<http><url>" target="<string>"	3:FHxZCtX+HIWxKqI4.F^F+Tn	{'4648': 1, '4878': 1, '785A': 4, '5A43': 1, '4374': 1, '7458': 1, ... '3646': 1, '462B': 1, '2B54': 1, '546E': 1}
<SCRIPT> </SCRIPT>	var<name>=document.body.clientWidth; var<name>=document.body.clientHeight; function <func> (who, type, step, timeout) { document.getElementById(who).style.display= "<string>"; if(type == 0) { reveal(<string>, step, timeout, 0); reveal(<string>, step, timeout, 1); } ... }	24:7wt5mlAp3oKpOyyplipophypolaPxhRR VarPRYb:7251hoKuyyc2zyalaPxhURPR2	{'3777': 1, '7774': 1, '7435': 1, '356D': 1, '6D6C': 1, '6C41': 1, ... '6855': 1, '5552': 1, '5232': 1}

FIG. 56

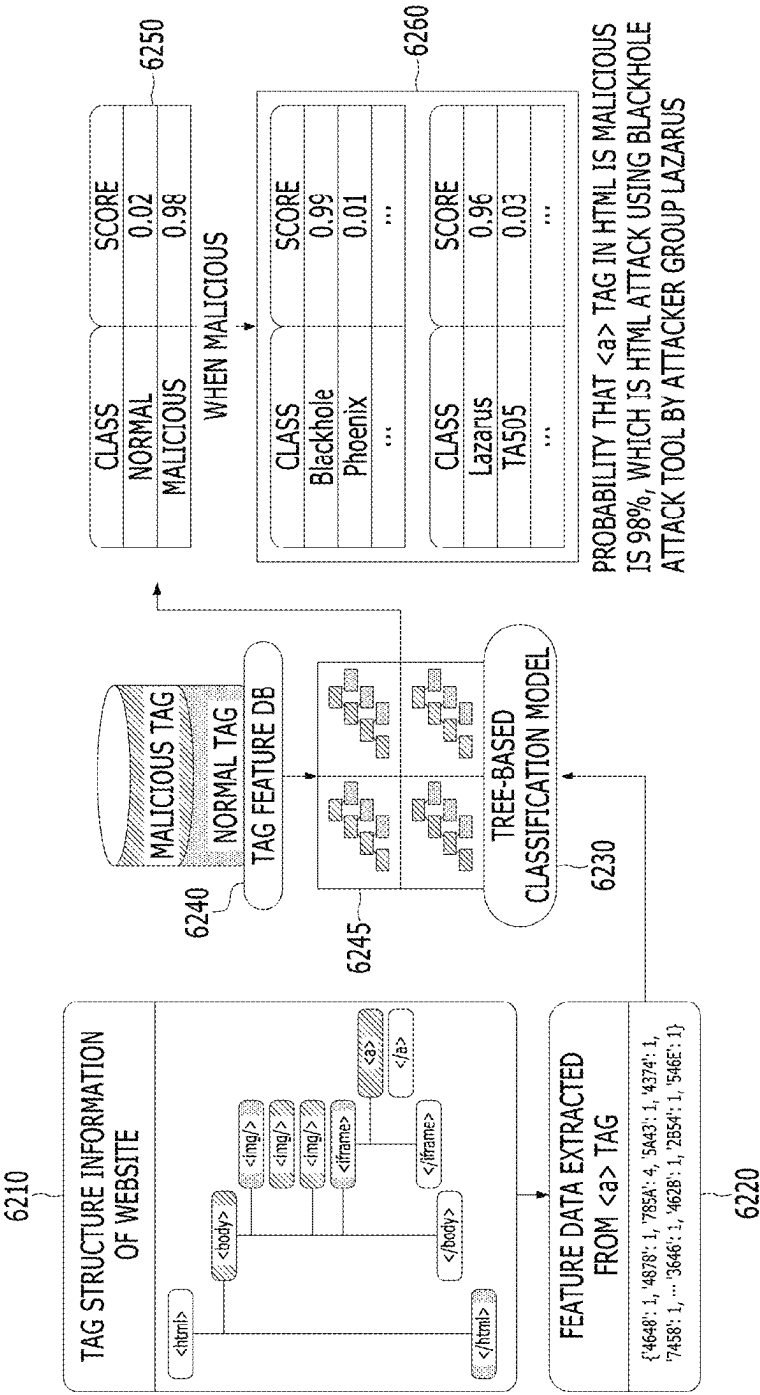


FIG. 57

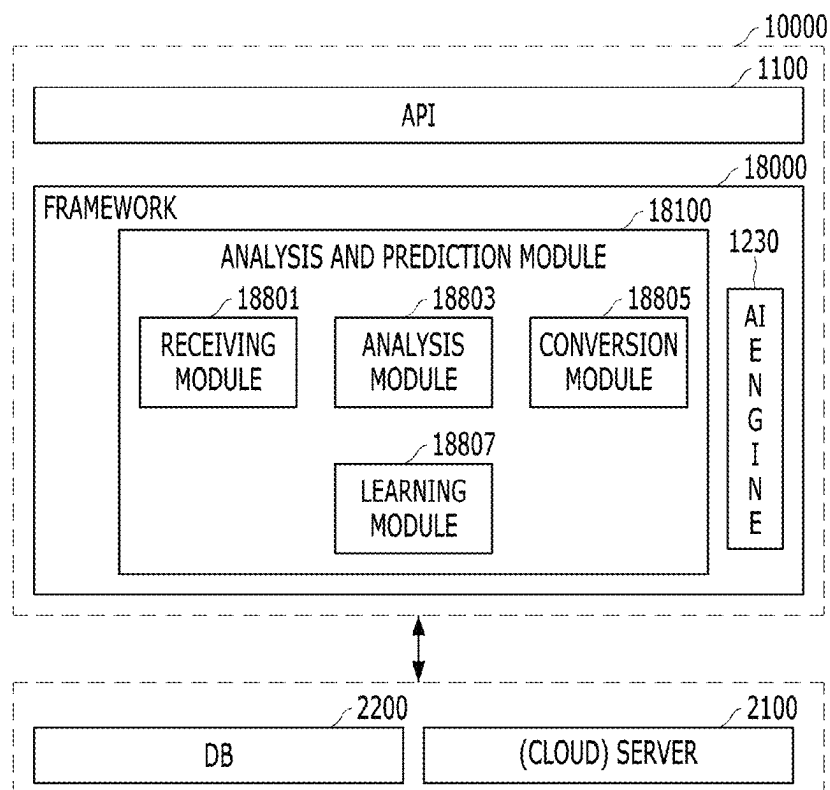


FIG. 58

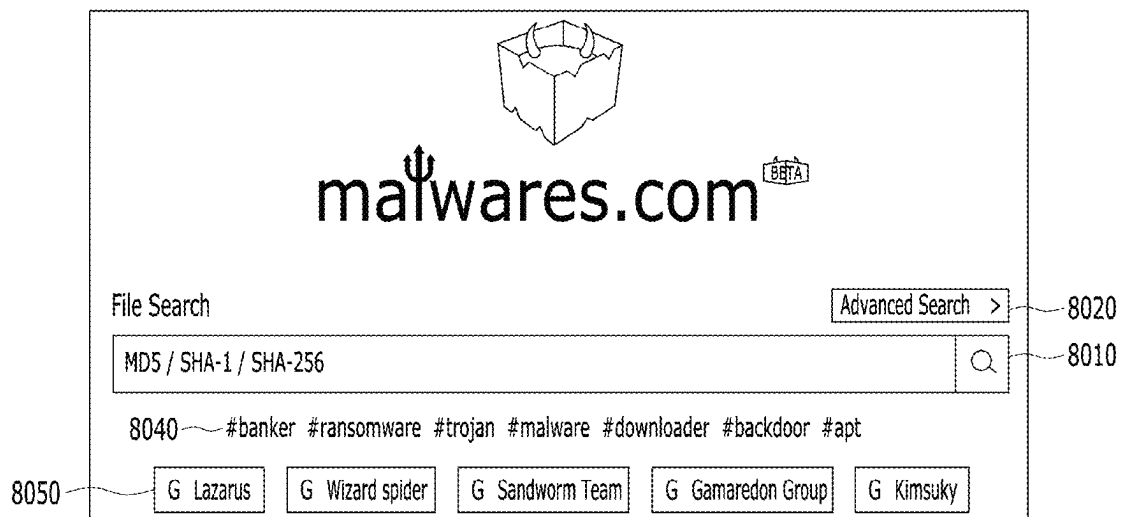


FIG. 59

Advanced Search Guide	
8021	AI · FORM : NUMBER AI MALICIOUS SCORE, RANGE SEARCH (>, <, =)
8022	Tag · FORM : STRING FILE FEATURE INFORMATION (ransomware, banker, trojan)
8023	FILE TYPE · FORM : STRING FILE TYPE (exe_32bit, pdf, doc, xls)
8024	FILE SIZE · FORM : NUMBER FILE SIZE (Bytes), RANGE SEARCH (>, <, =)
8025	ATTACK GROUP · FORM : STRING ATTACK GROUP NAME
8026	ATTACK TECHNIQUE ID · FORM : STRING ATTACK TECHNIQUE CODE (T1059, T1059.007, T1204, T1548)
8027	THREAT TYPE · FORM : STRING THREAT TYPE (trojan, riskware, downloader, phishing)
8028	ATTACK TARGET COUNTRY CODE · FORM : STRING ATTACK TARGET COUNTRY CODE (kr, cn, us, jp)
8029	ATTACK TARGET INDUSTRY · FORM : STRING ATTACK TARGET INDUSTRY (energy, financial, technology, aerospace)
8030	CVE · FORM : STRING VULNERABILITY CODE (cve-2023-22409, cve-2019-5181)

FIG. 60

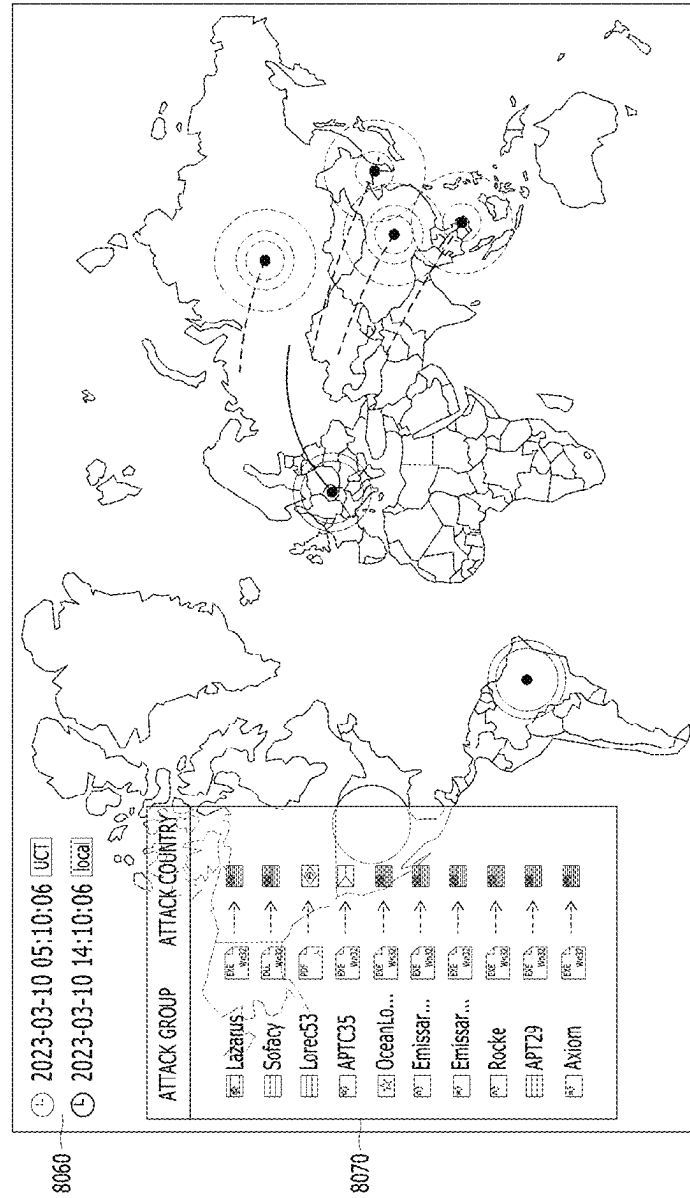


FIG. 61

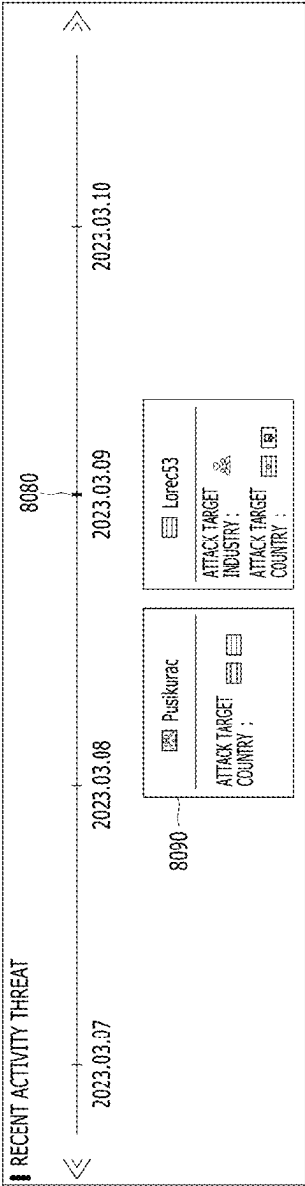


FIG. 62

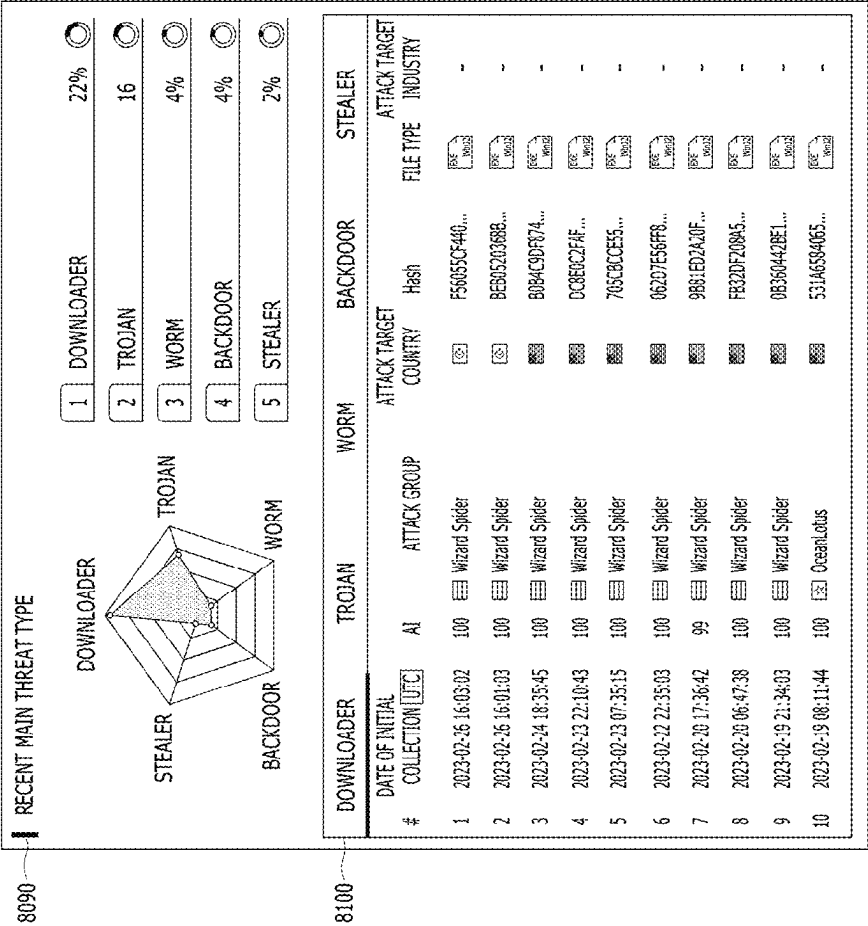
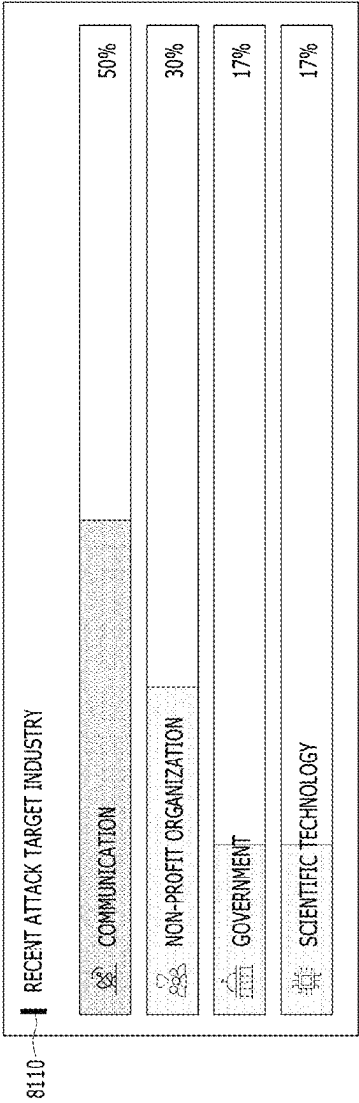


FIG. 63



(a)

8120

COMMUNICATION		NON-PROFIT ORGANIZATION		GOVERNMENT		SCIENTIFIC TECHNOLOGY	
#	DATE OF INITIAL COLLECTION UTC	AI	ATTACK GROUP	ATTACK TARGET COUNTRY	Hash	FILE TYPE	
1	2023-02-16 15:52:39	0	Mummy Spider		4E6A6F66AE9...		
2	2023-02-16 06:52:39	100	Mummy Spider		D91572FF64C...	EYE W632	
3	2023-02-09 18:19:26	12	Lorec53		12FD8AB950...	EYE W632	

(b)

FIG. 64

8130 RECENT APT ATTACK Top 10 [F:File] [I:IP] [U:URL] [D:Domain]

#	LAST ACTIVITY TIME	ATTACK GROUP	File	IP	URL	Domain	ATTACK TARGET INDUSTRY	VIEW DETAILS
1	2023-03-08 22:11:13	 raspite	[F:85B936960F...] [F:15292172A8...]	[I:85B936960F...] [I:57971205]	-	[D:mygoodluck...] [D:henherhero...]	-	NEW DETAILS
2	2023-03-08 17:16:29	 wizard spier	[F:8756F7B20E...] [F:D50C068A3E...]	-	-	-		NEW DETAILS
3	2023-03-08 17:14:23	 lazarus	[F:52A1CA4E65...] [F:133820EBAC...]	[I:212.143.21.43] [I:86.124.128.1...]	[U:totalmateria...] [U:kattachicvi...]	[D:dlinc.com] [D:imoreholm...]		NEW DETAILS
4	2023-03-08 16:40:01	 mummy spider	[F:9EB50FB862...] [F:4A133E34E4...]	-	-	-	-	NEW DETAILS
5	2023-03-08 16:16:18	 lore53	[F:6FBEDEA566...] [F:5FD291193F...]	[I:95.214.24.244] [I:193.142.146...]	[U:update-check...] [U:nirsoft.me]	[D:windows-ser...] [D:amitcan-spor...]	-	NEW DETAILS
6	2023-03-08 15:27:32	 emissary panda	[F:50B406F48A...] [F:101171CC6F...]	-	-	-	-	NEW DETAILS
7	2023-03-08 08:49:21	 solacy	[F:0FC4DA88F5...] [F:07D1F7BF5A...]	[I:45.124.132.1...] [I:109.248.148...]	[U:iboxmit.com] [U:myinvestgrou...]	[D:homeabouth...] [D:handsincevar...]	-	NEW DETAILS
8	2023-03-08 02:20:30	 pusikura	[F:AA056BD16...] [F:8D88075854...]	[I:109.234.36.55]	-	[D:qstorm.chick...] [D:fattoy.hopto...]	-	NEW DETAILS

FIG. 65

IoC (Indicators of Compromise) USED IN ATTACK GROUP Top. 50									
Respite		File 76		IP 10		URL 0		Domain 7	
#	DATE OF INITIAL COLLECTION	AI	Hash (SHA-256)	FILE TYPE	ATTACK COUNTRY	TARGET INDUSTRY	SUMMARY		
30	2023-02-23 11:57:07	100	A92F808C34C1A87D514480B8F8B84A1C239477C895051A07D0... #bbsgm #genetic #malicious #backdoor #variant #application #batter #apt #malware #batter #backdoor #trojan #pua #exploit #downloader #miner			.	View		
49	2023-02-22 02:36:39	100	7CF532E23230728EC1367A6FA79C120068646F5795807963810502F... #trojan #malicious #variant #batter #genetic #batter #apt #malware #batter #backdoor #trojan #pua #exploit #downloader #miner			.	View		
48	2023-02-19 18:29:54	100	E84A06E48108CA8B159E70E06FFC11A08332320173A204813850C35... #variant #batter #backdoor #malicious #trojan #malware #apt #batter #backdoor #trojan #backdoor #pua #exploit #downloader #miner			.	View		
47	2023-02-19 18:27:55	100	F4EE5A208078F71682094E9278487238FC36945580012105577D598... #malicious #variant #batter #backdoor #trojan #malware #apt #batter #backdoor #trojan #backdoor #pua #exploit #downloader #miner			.	View		
46	2023-02-14 18:18:14	100	72A007C7F9648025E0F5D5C3065671520C58871279C7577F421204E7... #application #genetic #equationing #malicious #committer #trojan #pua #exploit #downloader			.	View		
45	2023-01-16 01:39:07	100	3356555588FEC18D3A4949E1B31FF5E12863D9447769424148704F5530... #equationing #application #genetic #malicious #committer #trojan #pua #exploit #downloader #trojan #backdoor #pua #trojan #miner #exploit		.	.	View		

8140

FIG. 66

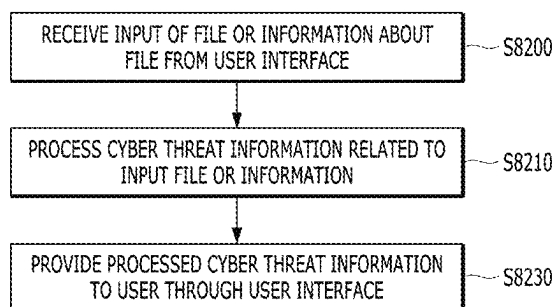


FIG. 67

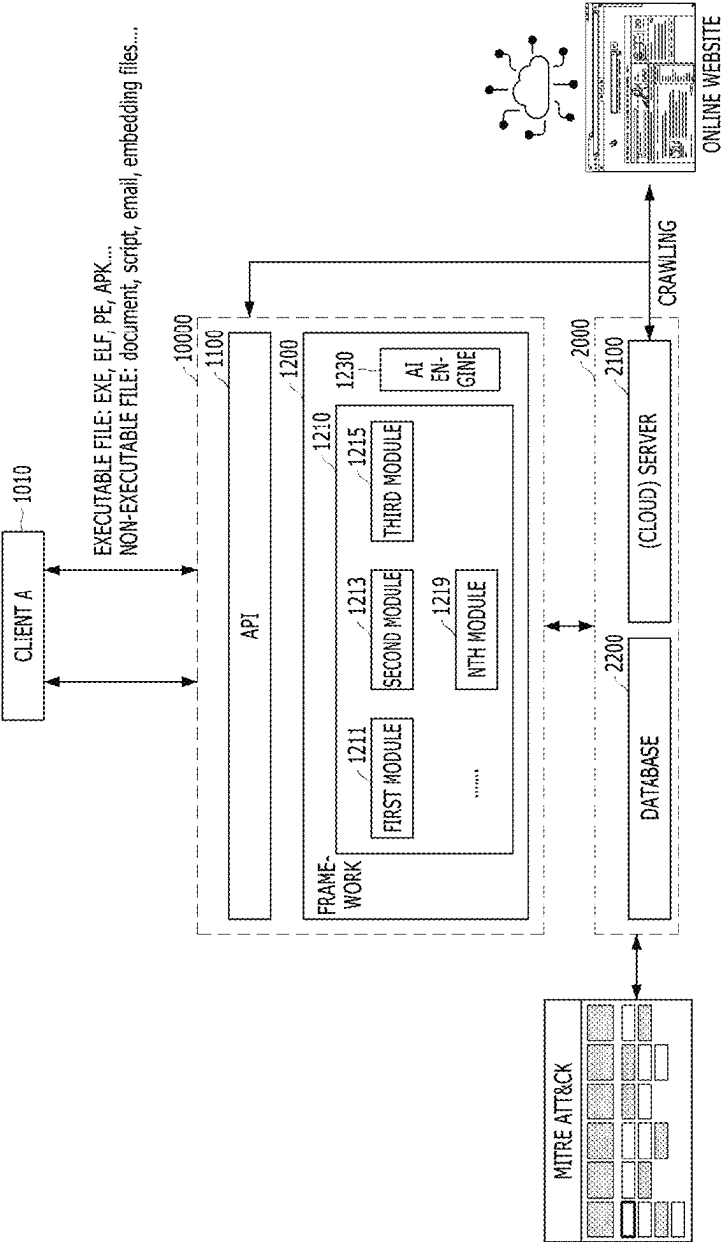


FIG. 68

8300	RECENT APT ATTACK LIST										MALWARE LIST USED IN APT ATTACK CAN BE CHECKED IN REAL TIME.										2023-03-10 06:37:39 UTC 2023-03-10 15:37:39 UTC																																																			
	8310										8320										8330																																																			
	KNOWN ATTACK GROUP										UNKNOWN ATTACK GROUP																																																													
8331	8332										8333										8334										8335										8336										8337										8338											
	DATE OF INITIAL COLLECTION										ATTACK GROUP										AI										HASH (SHA-256)										FILE TYPE										ATTACK TARGET COUNTRY										ATTACK TARGET INDUSTRY										SUMMARY	
8341	2023-03-02 19:21:06										Gemaredon Group										0										56128266863510EAC6F7CE3904CF8002D94377ED7A2E508B125C756...										trojan #tro																															

FIG. 69

RECENT APT ATTACK LIST MALWARE LIST USED IN APT ATTACK CAN BE CHECKED IN REAL TIME.									
8320 UNKNOWN ATTACK GROUP									
DATE OF INITIAL COLLECTION	ATTACK GROUP	AI	Hash (SHA-256)	FILE TYPE	ATTACK COUNTRY	TARGET INDUSTRY	SUMMARY		
2023-03-09 19:11:16	0001 *Gratifying-Newt-757	0	204747C59375712D815F7674A485254540872E08E608080228D9131184... #trojan #trojan #trojan	000	000	000	View		
2023-03-09 19:11:16	0002 *Gratifying-Newt-757	0	88460995C701C5408803144241412654916248584C9150C5F07A68C12... #trojan #trojan #trojan	000	000	000	View		
2023-03-09 17:23:46	0003 *Purring-Newt-892	0	5438708640F951770872671537F1668029C3678794408605146E8F03C9... #trojan #trojan #trojan	000	000	000	View		
2023-03-09 17:21:55	0004 *Gratifying-Newt-708	75	0AD2004538820369F6FC4FC404F91588007A4F70B5943D5E7A7AE... #trojan #trojan #trojan	000	000	000	View		
2023-03-09 17:08:15	0005 *Gratifying-Newt-860	99	540C3D131130F487294466C4612218C79FF0D0A8F4659D867EB338C3... #trojan #trojan #trojan	000	000	000	View		
2023-03-09 16:01:00	0006 *Alluring-Leopard-523	0	52FF4B0F281D6585747878814F704F7796552940722341848F01758874C... #trojan #trojan #trojan	000	000	000	View		
2023-03-09 16:01:00	0007 *Alluring-Leopard-523	0	66FFC04525FE56560007E2D6A8F9104246E00457287CC4886C104888... #trojan #trojan #trojan	000	000	000	View		
2023-03-09 16:01:00	0008 *Alluring-Leopard-523	0	2B51A852ED187F790C8C71188505714880018D4048F0A10C15095B7... #trojan #trojan #trojan	000	000	000	View		
2023-03-09 16:01:00	0009 *Alluring-Leopard-523	0	732F0CC40A6A50F1D26188E8E6505C5165C2914C7F974275C4B572A... #trojan #trojan #trojan	000	000	000	View		
2023-03-09 16:01:00	0010 *Alluring-Leopard-523	0	9F66820386F697878A8217C380843D3880E396407F9100C40C530FD... #trojan #trojan #trojan	000	000	000	View		

8343

8344

FIG. 70

8351

A992FBD89C34C1AA8F7DC51AAA8C6B3F1BBADA7C2394377C5B95D51AD7DD4501 ×

100

#bingonl

#generic

#malicious

#blackmoon

#variant

#application

#babar

#apt

#malware

#clicker

#hacktool

#backdoor

#trojan

#pua

#exploit

#downloader

#miner

8350

SUMMARY

VIEW DETAILS

8353

OUTLINE

DATE OF INITIAL COLLECTION

2023-02-23 11:57:07

14 DAYS 17 HOURS

LAST ACTIVITY DATE

2023-02-28 00:26:24

10 DAYS 4 HOURS

FILE TYPE

exe_32bit

FILE SIZE

70,144 bytes

8354

Hash

MD5

8A75B0EF65CD434E12598D6D656E89A7

SHA-1

92AC075CED9909B1F23416C881E924B0857D57CC

SHA-256

A992FBD89C34C1AA8F7DC51AAA8C6B3F1BBADA7C2394377C5B95D51AD7DD4501

8355

THREAT TYPE

Trojan

8356

PATTERN DETECTION NAME

Gen:Variant.Application.Babar.17593

8357

ATTACK GROUP

Emissary Panda

Raspite

8358

ATTACK TARGET COUNTRY

100%

FIG. 72

CAMPAIGN : threat--1340c3fc-5570-5a22-80fc-fb24cc8f1947 (2023-01-11 03:40:26 ~ 2023-01-11 05:59:55(UCT)) 8371					
[F] File [I] IP [U] URL [D] Domain					
Ⓢ ATTACK GROUP	Ⓢ ATTACK TARGET COUNTRY	Ⓢ ATTACK TECHNIQUE	Ⓢ ATTACK TECHNIQUE	Ⓢ CVE	Ⓢ IoC
PKPLUG MUSTANG PANDA	🇰🇼 KUWAIT	🏢 FINANCIAL-SERVICES 🏛️ GOVERNMENT 📡 TELECOMMUNICATIONS 🏢 SERVICES			{F} 3BF18AEE007E8935F...
	🇹🇷 TURKEY				{F} 918DE40E8BA7E9C1...
	🇸🇦 UNITED KINGDOM				{F} E8ADA48C075B6CA4...
	🇯🇴 JORDAN				{F} 9A888034495D103AE...
	🇮🇹 ITALY				
	🇩🇪 GERMANY				
CAMPAIGN : threat--3960caca-c343-5d7f-8702-621995233791 (2022-08-30 16:17:03 ~ 2023-02-28 02:26:04(UCT)) 8372					
[F] File [I] IP [U] URL [D] Domain					
Ⓢ ATTACK GROUP	Ⓢ ATTACK TARGET COUNTRY	Ⓢ ATTACK TECHNIQUE	Ⓢ ATTACK TECHNIQUE	Ⓢ CVE	Ⓢ IoC
PKPLUG MUSTANG PANDA	🇰🇼 KUWAIT	🏢 FINANCIAL-SERVICES 🏛️ GOVERNMENT 📡 TELECOMMUNICATIONS 🏢 SERVICES			{F} 3BF18AEE007E8935F...
	🇹🇷 TURKEY				{F} 918DE40E8BA7E9C1...
	🇸🇦 UNITED KINGDOM				{F} E8ADA48C075B6CA4...
	🇯🇴 JORDAN				{F} 9A888034495D103AE...

FIG. 73

CODE ~ 8380 ~ 8381		Techniques ~ 8382		Offset	bytes	opcode	operand	8384
[Start Offset ~ End Offset [L]]		[T027] ~ 8385		401040	55	push	%edx	
[0x401040 0x401096 53 Lines] ~ 8383				401041	80 bc	mov	%esp, %edx	
[0x401096 0x40114F 122 Lines]		Defense Evasion		401043	83 ec 14	sub	\$0x14, %esp	
[0x401200 0x40141F 122 Lines]		Obfuscated Files or Information		401046	56	push	%esi	
[0x401510 0x401647 116 Lines]		[T0021]		401047	57	push	%edi	
[0x401680 0x40171E 36 Lines]		[T0062]		401048	66 2c 15 00 00	mov	\$0x12c, %esi	
[0x401770 0x401885 97 Lines]		[T0062]		40104d	80 fb	mov	%eax, %edi	
[0x401880 0x401CC5 105 Lines]		[T0063]		40104f	89 75 fb	mov	%esi, (%ebx, %edi)	
[0x401CD0 0x401D4F 52 Lines]		[T0063]		401052	8b 36 4c 00 00	mov	\$0x436, %ecx	
[0x402060 0x402A21 146 Lines]		[T0063] [T0547 006]		401057	85 d	test	%ecx, %eax	
[0x402430 0x402B56 111 Lines]		[T0063]						

FIG. 74

SIMILAR FILE 8380									
8391		8392		8393		8394		8395	
SIMILAR FILE Top 5 (LAST 3 MONTHS)									
DATE OF INITIAL COLLECTION (UTC)	ATTACK GROUP	AI	Hash (SHA-256)	FILE TYPE	ATTACK COUNTRY	TARGET COUNTRY	TARGET INDUSTRY	SUMMARY	
2023-03-15 03:17:59	(88) Lazarus Group	100	0A7026542337752396504C7062751412590183403... Trojan-Fragment-Adware-Fraudulent-Endpoint-Executable-Program [T1106]: [T1597] [T1597/000] [T1597] [T1597/001] [T1112]	(86) EXE	(88)	(88)	.	View	
2023-03-12 15:40:45	(88) "Blowing-Nest-098"	100	84833D8EE18858F71AF9C3338CE40C5020051125108... Fraudulent-Endpoint-Program-Endpoint-Executable-Program	(86) EXE	(88)	(88)	.	View	
2023-03-20 14:54:31	(88) Lazarus Group	100	893D5116381388825062009870348063177C094910... Trojan-Fragment-Adware-Fraudulent-Endpoint-Executable-Program [T1106]: [T1597] [T1597/000] [T1597] [T1597/001] [T1112]	(86) EXE	(88)	(88)	.	View	
2023-02-21 03:41:40	(87) Lazarus	100	264420000775492D159CC498045ED1583040396086C... Fraudulent-Endpoint-Adware-Endpoint-Executable-Program	(86) EXE	(88)	(88)	.	View	
2023-02-19 08:16:25	(87) Lazarus	100	61238945CC6A85F046FF84A30ED735AC398232646... Trojan-Fragment-Fraudulent-Adware-Endpoint-Executable-Program	(86) EXE	(88)	(88)	.	View	

FIG. 76

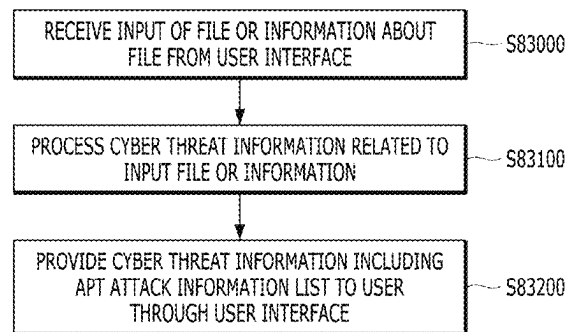


FIG. 77

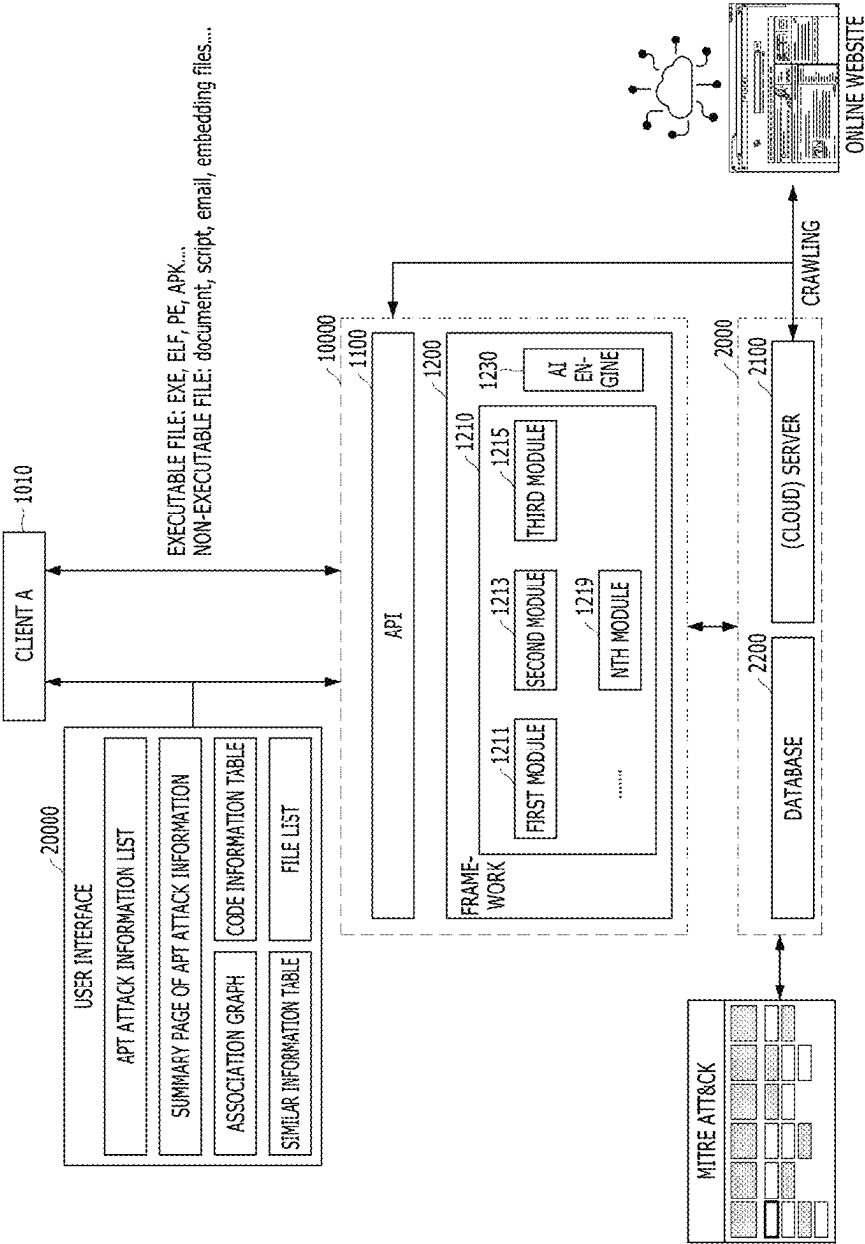


FIG. 79

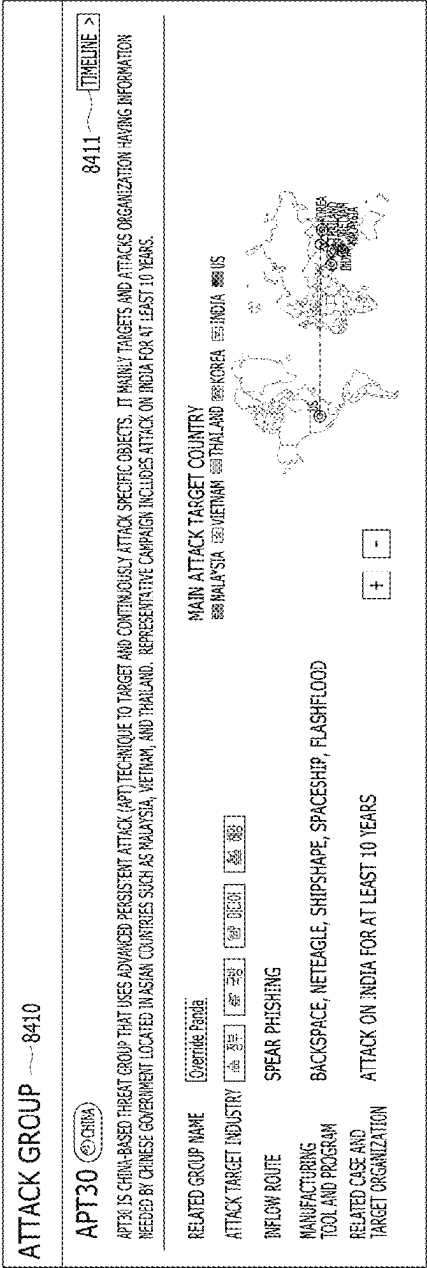


FIG. 80

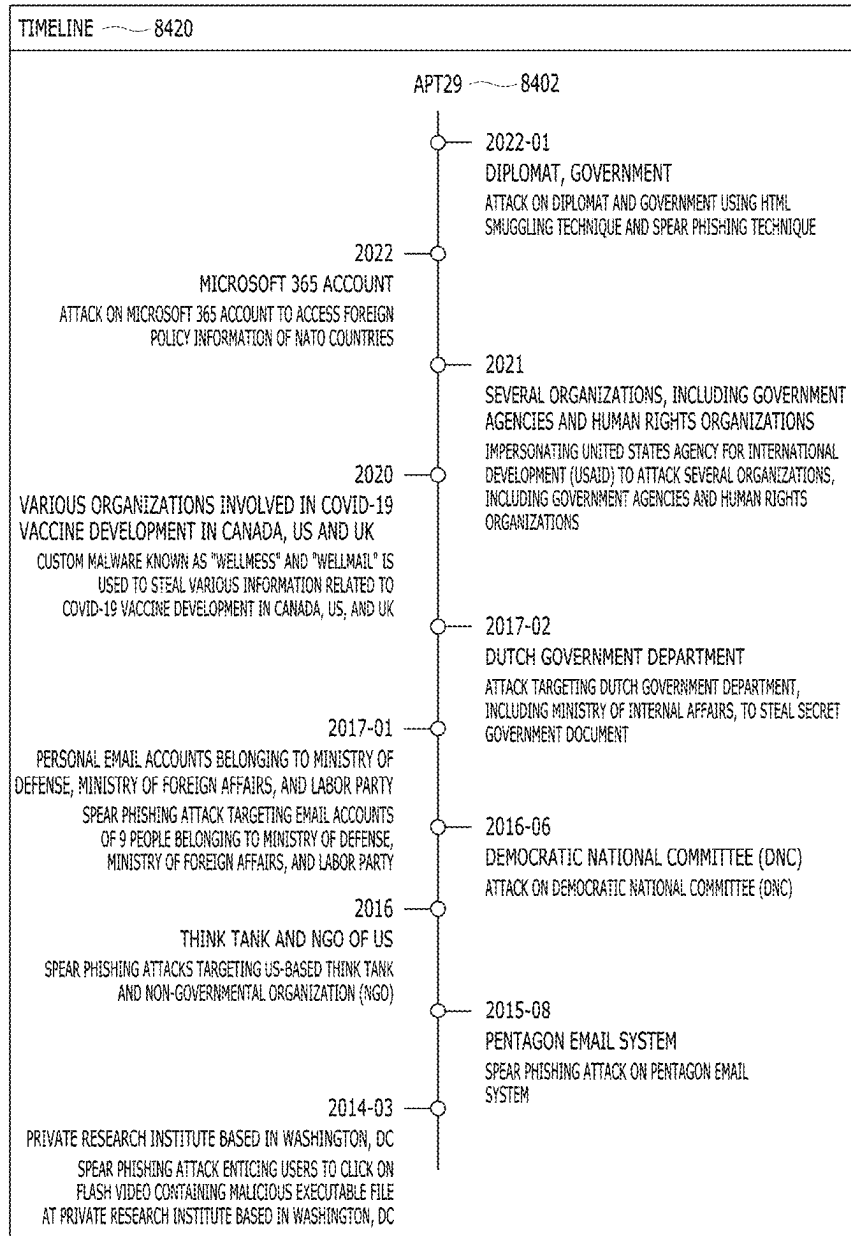


FIG. 81

ATTACK GROUP

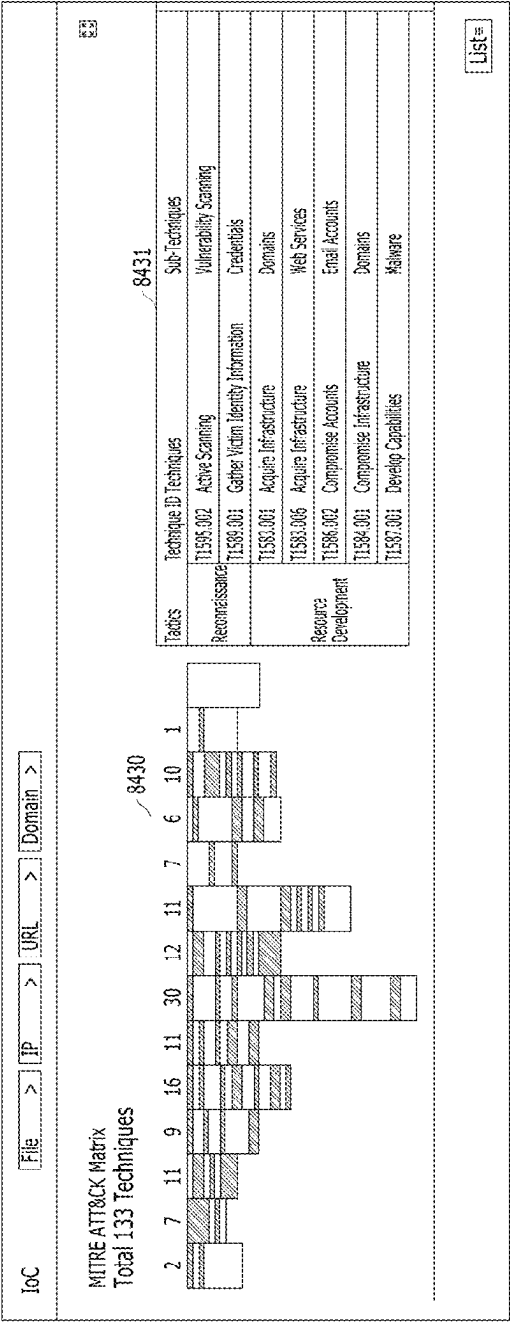


FIG. 82

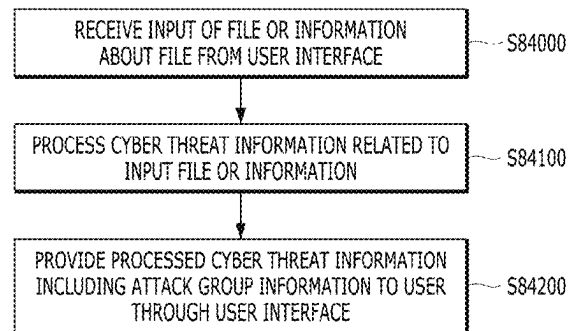


FIG. 83

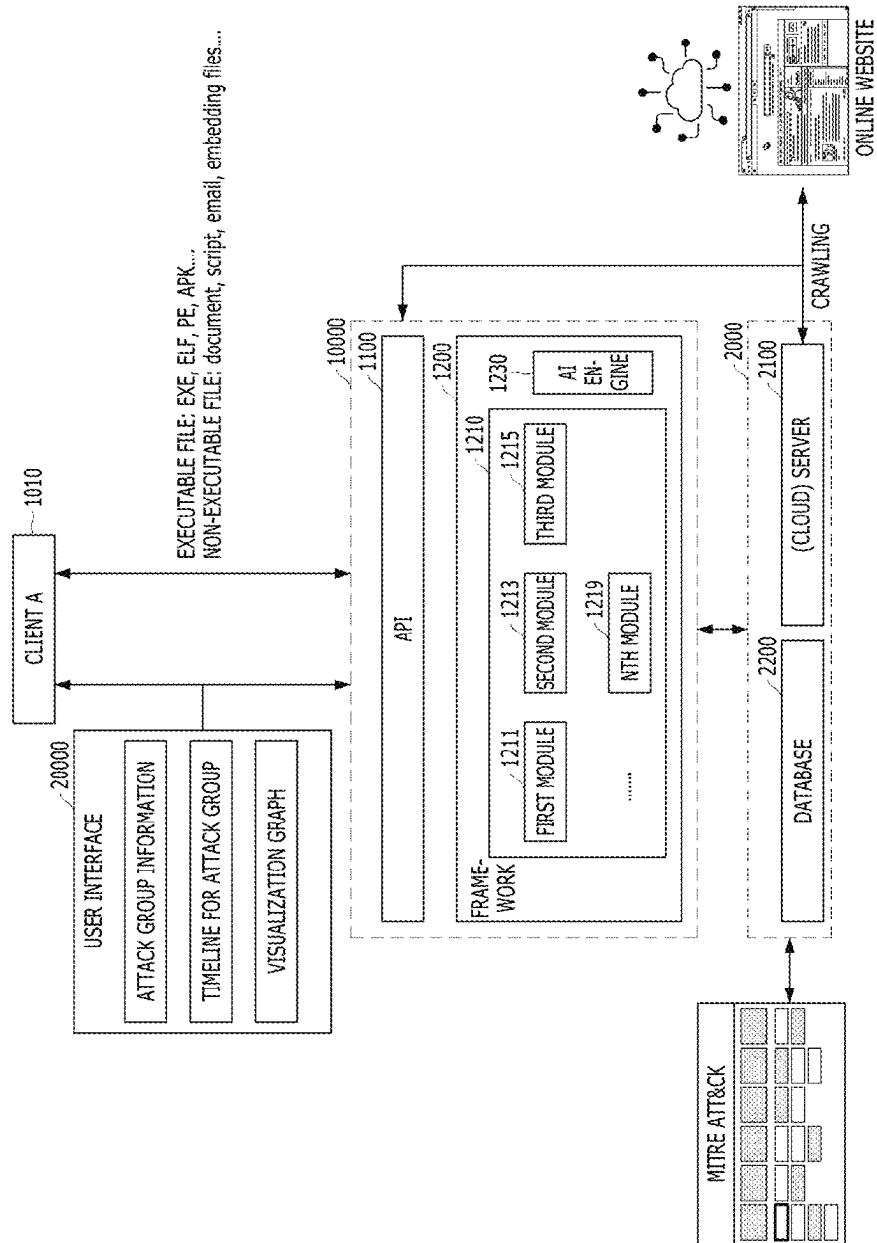


FIG. 84

INTEGRATED MALWARE DATASET PACKAGE 8500

THIS PACKAGE CONSISTS OF EXECUTABLE AND NON-EXECUTABLE FILE MALWARE DATASETS.
RATIO OF EXECUTABLE DATASET TO NON-EXECUTABLE DATASET IS 6:4.

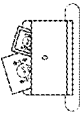
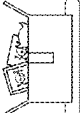
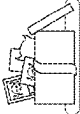
 8501 1,000 CASES EXE-DLL-ELF-APK PDF-DOC(X)-XLS(X)-PPT(X)-HWP DATASET 1,000 RAW MALWARE FILES CONFIGURATION AMOUNT OF PAYMENT W 1,500,000 ☐ CHECK PRODUCT PURCHASE CONDITIONS AND AGREE TO CANCELLATION AND REFUND POLICY PURCHASE
 8502 3,000 CASES EXE-DLL-ELF-APK PDF-DOC(X)-XLS(X)-PPT(X)-HWP DATASET 3,000 RAW MALWARE FILES CONFIGURATION AMOUNT OF PAYMENT W 4,500,000 ☐ CHECK PRODUCT PURCHASE CONDITIONS AND AGREE TO CANCELLATION AND REFUND POLICY PURCHASE
 8503 5,000 CASES EXE-DLL-ELF-APK PDF-DOC(X)-XLS(X)-PPT(X)-HWP DATASET 5,000 RAW MALWARE FILES CONFIGURATION AMOUNT OF PAYMENT W 7,500,000 ☐ CHECK PRODUCT PURCHASE CONDITIONS AND AGREE TO CANCELLATION AND REFUND POLICY PURCHASE
 8504 10,000 CASES EXE-DLL-ELF-APK PDF-DOC(X)-XLS(X)-PPT(X)-HWP DATASET 10,000 RAW MALWARE FILES CONFIGURATION AMOUNT OF PAYMENT W 15,000,000 ☐ CHECK PRODUCT PURCHASE CONDITIONS AND AGREE TO CANCELLATION AND REFUND POLICY PURCHASE

FIG. 85

EXECUTABLE MALWARE DATASET PACKAGE 8510

THIS PACKAGE CONSISTS OF EXECUTABLE FILE MALWARE DATASETS
IT CONSISTS OF 32-BIT, 64-BIT EXE AND DLL FILES THAT RUN ON WINDOWS OPERATING SYSTEMS AND 32-BIT, 64-BIT, ELF AND SO FILES THAT RUN ON LINUX OPERATING SYSTEMS

 1,000 CASES EXE DLL ELF APK DATASET CONFIGURATION 1,000 RAW MALWARE FILES AMOUNT OF PAYMENT W 1,000,000 ☐ CHECK PRODUCT PURCHASE CONDITIONS AND AGREE TO CANCELLATION AND REFUND POLICY PURCHASE	 3,000 CASES EXE DLL ELF APK DATASET CONFIGURATION 3,000 RAW MALWARE FILES AMOUNT OF PAYMENT W 3,000,000 ☐ CHECK PRODUCT PURCHASE CONDITIONS AND AGREE TO CANCELLATION AND REFUND POLICY PURCHASE	 5,000 CASES EXE DLL ELF APK DATASET CONFIGURATION 5,000 RAW MALWARE FILES AMOUNT OF PAYMENT W 5,000,000 ☐ CHECK PRODUCT PURCHASE CONDITIONS AND AGREE TO CANCELLATION AND REFUND POLICY PURCHASE	 10,000 CASES EXE DLL ELF APK DATASET CONFIGURATION 10,000 RAW MALWARE FILES AMOUNT OF PAYMENT W 10,000,000 ☐ CHECK PRODUCT PURCHASE CONDITIONS AND AGREE TO CANCELLATION AND REFUND POLICY PURCHASE
---	---	---	--

FIG. 86

NON-EXECUTABLE MALWARE DATASET PACKAGE 8520

THIS PACKAGE CONSISTS OF NON-EXECUTABLE FILE MALWARE DATASETS WIDELY USED IN RECENT APT ATTACKS. IT CONSISTS OF HANGUL (HWP), WORD (DOCX), POWERPOINT (PPTX), EXCEL (XLSX), AND PDF DOCUMENT FILES.

1,000 CASES PDF-DOCX-XLSX-PPTX-HWP DATASET 1,000 RAW MALWARE FILES AMOUNT OF PAYMENT W 2,000,000 ☐ CHECK PRODUCT PURCHASE CONDITIONS, AND AGREE TO CANCELLATION AND REFUND POLICY PURCHASE	3,000 CASES PDF-DOCX-XLSX-PPTX-HWP DATASET 3,000 RAW MALWARE FILES AMOUNT OF PAYMENT W 6,000,000 ☐ CHECK PRODUCT PURCHASE CONDITIONS, AND AGREE TO CANCELLATION AND REFUND POLICY PURCHASE	5,000 CASES PDF-DOCX-XLSX-PPTX-HWP DATASET 5,000 RAW MALWARE FILES AMOUNT OF PAYMENT W 10,000,000 ☐ CHECK PRODUCT PURCHASE CONDITIONS, AND AGREE TO CANCELLATION AND REFUND POLICY PURCHASE	10,000 CASES PDF-DOCX-XLSX-PPTX-HWP DATASET 10,000 RAW MALWARE FILES AMOUNT OF PAYMENT W 20,000,000 ☐ CHECK PRODUCT PURCHASE CONDITIONS, AND AGREE TO CANCELLATION AND REFUND POLICY PURCHASE
--	--	---	---

FIG. 87

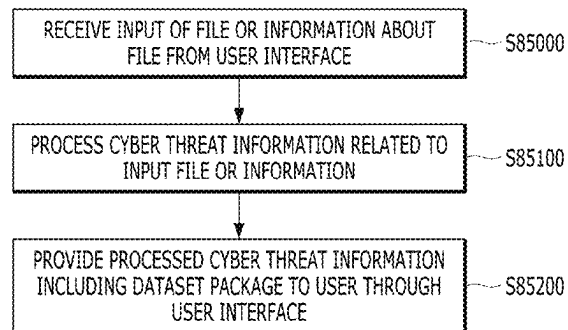


FIG. 88

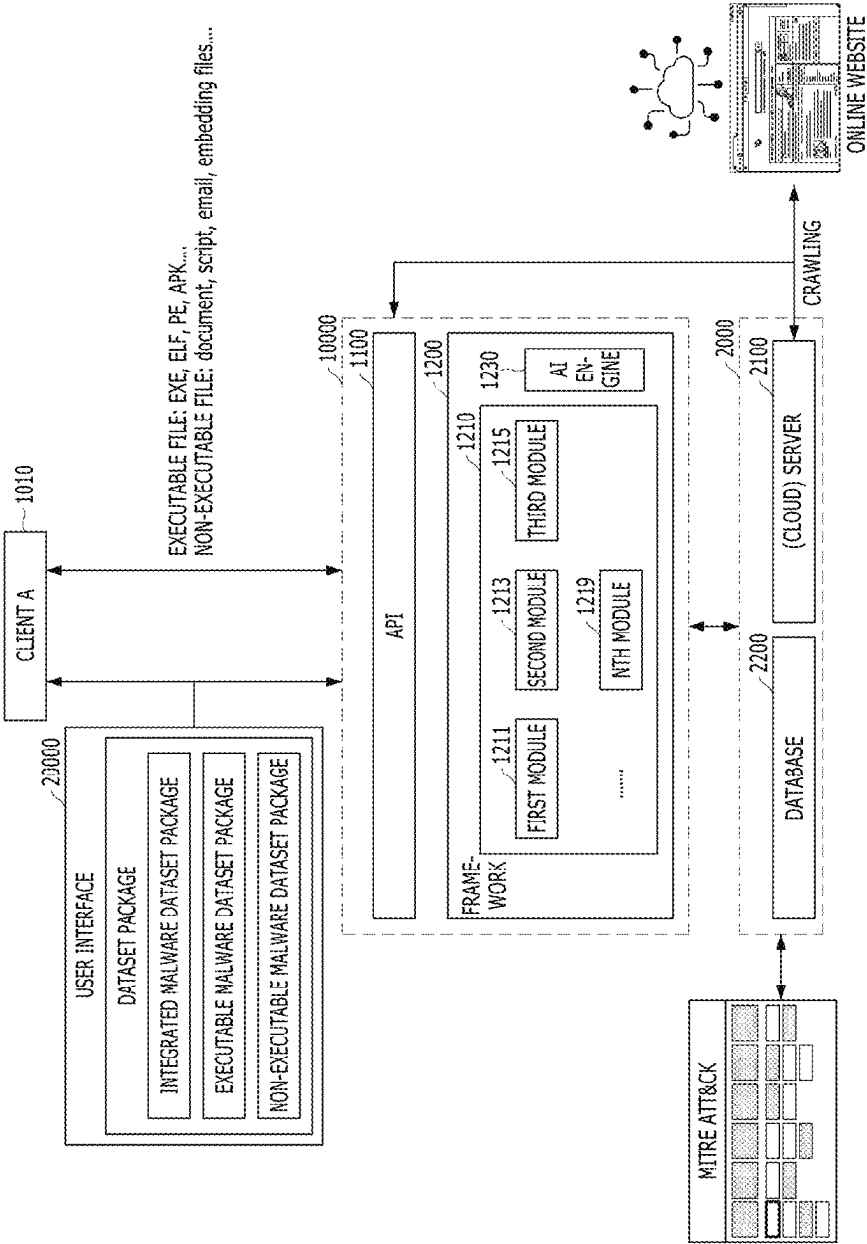


FIG. 90

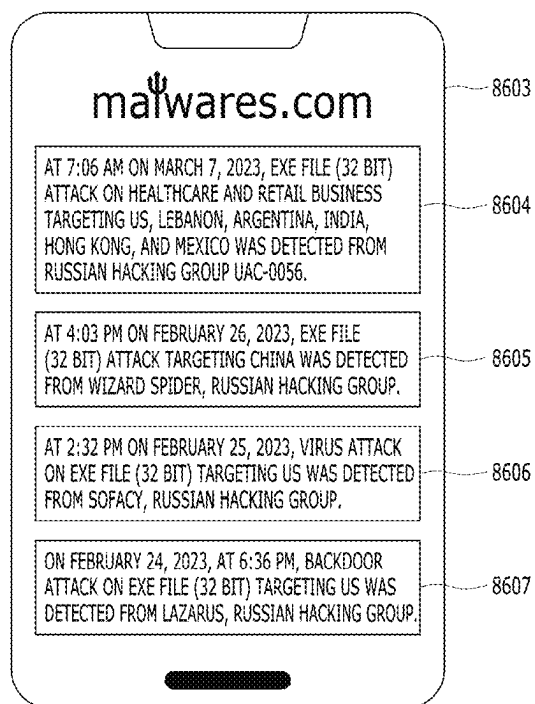


FIG. 91

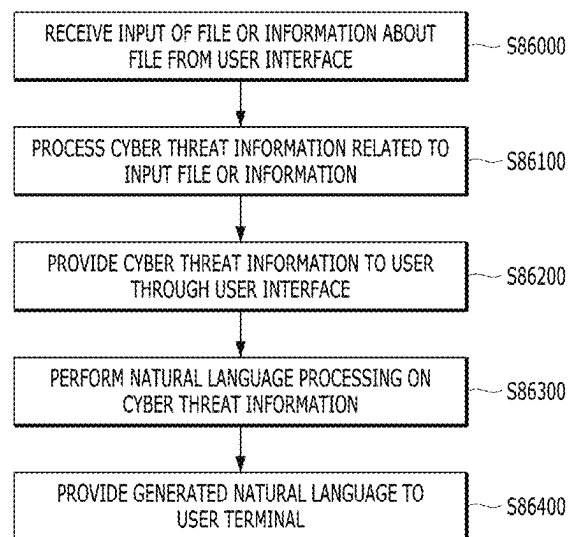


FIG. 93

malwares.com
178.63.254.36
MDS / SHA-1 / SHA-256

ASSOCIATED CAMPAIGN INFORMATION

Threat: --30791e0f-2339-5666-A308-Fd781f36ba7 (2023-05-23 16:33:44 ~ 2023-03-23 16:34:00)

ATTACK GROUP	ATTACK TARGET COUNTRY	ATTACK TARGET INDUSTRY
Razors [3%] Fancy Bear [6%]	[3%] [30%]	FINANCE [30%] NATIONAL DEFENSE [5%] MEDIA [5%] ENERGY [5%] ENTERTAINMENT [5%] AEROSPACE INDUSTRY [5%]

IP	DATE OF LAST CONFIRMATION	DATE OF FIRST CONFIRMATION	Protocol	Tag	GROUPS
	2022-04-16 01:33:21	2020-06-16 01:33:21	-	C&C, Malware	REPUTATION INFORMATION INQUIRY

IoC File > IP > URL > Domain >

Threat: --30791e0f-2339-5666-A308-Fd781f36ba7 (2023-05-23 16:33:44 ~ 2023-03-23 16:34:00)

ATTACK GROUP	ATTACK TARGET COUNTRY	ATTACK TARGET INDUSTRY
Razors [3%] Fancy Bear [6%]	[3%] [30%]	FINANCE [30%] NATIONAL DEFENSE [5%] MEDIA [5%] ENERGY [5%] ENTERTAINMENT [5%] AEROSPACE INDUSTRY [5%]

IP	DATE OF LAST CONFIRMATION	DATE OF FIRST CONFIRMATION	Protocol	Tag	GROUPS
	2022-04-16 01:33:21	2020-06-16 01:33:21	-	C&C, Malware	REPUTATION INFORMATION INQUIRY

IoC File > IP > URL > Domain >

FIG. 94

[illegible]

[illegible]

FIG. 96

[illegible]

FIG. 97

malwares.com

NDS / SHA-1 / SHA-256

asassasss.autos

ASSOCIATED CAMPAIGN INFORMATION

Threat--30791e0f-2339-5b66-A308-Fd78f56ba7 (2023-03-23 16:33:44 ~ 2023-03-23 16:34:00 UTC)

ATTACK GROUP

ATTACK TARGET COUNTRY

ATTACK TARGET INDUSTRY

RAZZOS

30%

30%

FINANCE 30%

WITNESS OFFENSE 1%

45% MEDIA 5%

2% ENERGY 2%

ENTERTAINMENT 2%

PERSONALITY INDUSTRY 1%

DATE OF LAST CONFIRMATION	DATE OF FIRST CONFIRMATION	Protocol	Tag	Grounds
2022-04-16 01:33:21 UTC	2020-04-16 01:33:21 UTC	-	CAC, Malware	REPUTATION INFORMATION INQUIRY

IoC

File

>

IP

>

URL

>

Domain

>

Threat--30791e0f-2339-5b66-A308-Fd78f56ba7 (2023-03-23 16:33:44 ~ 2023-03-23 16:34:00 UTC)

ATTACK GROUP

ATTACK TARGET COUNTRY

ATTACK TARGET INDUSTRY

RAZZOS

30%

30%

FINANCE 30%

WITNESS OFFENSE 1%

45% MEDIA 5%

2% ENERGY 2%

ENTERTAINMENT 2%

PERSONALITY INDUSTRY 1%

DATE OF LAST CONFIRMATION	DATE OF FIRST CONFIRMATION	Protocol	Tag	Grounds
2022-04-16 01:33:21 UTC	2020-04-16 01:33:21 UTC	-	CAC, Malware	REPUTATION INFORMATION INQUIRY

IoC

File

>

IP

>

URL

>

Domain

>

1

**CYBER THREAT INFORMATION
PROCESSING APPARATUS, CYBER THREAT
INFORMATION PROCESSING METHOD,
AND STORAGE MEDIUM STORING CYBER
THREAT INFORMATION PROCESSING
PROGRAM**

CROSS-REFERENCE TO RELATED
APPLICATIONS

This application claims the benefit of Korean Patent Application No. 10-2023-0047990, filed on Apr. 12, 2023, which is hereby incorporated by reference as if fully set forth herein.

BACKGROUND OF THE INVENTION

Field of the Invention

The disclosed embodiments relate to a cyber threat information processing apparatus, a cyber threat information processing method, and a storage medium storing a cyber threat information processing program.

Discussion of the Related Art

The damage from cybersecurity threats, which are gradually becoming more sophisticated, centering on new or variant malware, has been increasing. In order to reduce such damage even a little and to respond at an early stage, countermeasure technology has been advancing through multi-dimensional pattern composition, various types of complex analysis, etc. However, recent cyber-attacks tend to increase day by day rather than being adequately responded to within a control range. These cyberattacks threaten finance, transportation, environment, health, etc. that directly affect lives of people beyond the existing information and communication technology (ICT) infrastructure.

One of basic technologies to detect and respond to most existing cybersecurity threats is to create a database of patterns for cyberattacks or malware in advance, and utilize appropriate monitoring technologies where data flow is required. Existing technology has evolved based on a method of identifying and responding to threats when a data flow or code matching a monitored pattern is detected. Such conventional technology has an advantage of being able to rapidly and accurately perform detection when a data flow or code matches a previously secured pattern. However, the technology has a problem in that, in the case of a new or mutant threat for which a pattern is not secured or is bypassed, detection is impossible or it takes a significantly long time for analysis.

The related art is focused on a method of advancing technology to detect and analyze malware itself even when artificial intelligence (AI) analysis is used. However, there is no fundamental technology to counter cybersecurity threats, and thus there is a problem in that it is difficult to address new malware or new variants of malware with this method alone, and there is a limitation.

For example, there is a problem in that only the technology for detecting and analyzing previously discovered malware itself cannot address decoy information or fake information for deceiving a detection or analysis system thereof, and confusion occurs.

In the case of mass-produced malware having enough data to be learned, characteristic information thereof can be sufficiently secured, and thus it is possible to distinguish

2

whether code is malicious or a type of malware. However, in the case of advanced persistent threat (APT) attacks, which are made in relatively small numbers and attack precisely, since training data does not match in many cases, and targeted attacks make up the majority, even when the existing technology is advanced, there are limitations.

In addition, conventionally, methods and expression techniques for describing malware, attack code, or cyber threats have differed depending on the position or analysis perspective of an analyst. For example, a method of describing malware and attack activity has not been standardized worldwide, and thus there has been a problem in that, even when the same incident or the same malware is detected, explanations of experts in the field are different, and thus confusion had occurred. Even a malware detection name has not been unified, and thus, for the same malicious file, it has been impossible to identify an attack performed correctly, or attacks have been differently organized. Therefore, there has been a problem in that identified attack techniques cannot be described in a normalized and standardized manner.

A conventional malware detection and analysis method focuses on detection of malware itself, and thus has a problem in that, in the case of malware performing significantly similar malicious activity, when generating attackers are different, the attackers cannot be identified.

In connection with the above problems, the conventional method has a problem in that it is difficult to predict a type of cyber threat attack occurring in the near future by such an individual case-focused detection method.

SUMMARY OF THE INVENTION

The present disclosure is to address at least the above-mentioned problems and/or disadvantages and to provide at least the advantages described below. Accordingly, an aspect of the present disclosure is to provide a cyber threat information processing apparatus, a cyber threat information processing method, and a storage medium storing a cyber threat information processing program capable of detecting and addressing malware not exactly matching data learned by AI and addressing a variant of malware.

Another aspect of the present disclosure is to provide a cyber threat information processing apparatus, a cyber threat information processing method, and a storage medium storing a cyber threat information processing program capable of identifying malware, an attack technique, an attacker, and an attack prediction method in a significantly short time even for a variant of malware.

Another aspect of the present disclosure is to provide a cyber threat information processing apparatus, a cyber threat information processing method, and a storage medium storing a cyber threat information processing program capable of providing information about malware, for which a malware detection name, etc. is not unified or a cyberattack technique cannot be accurately described, in a normalized and standardized scheme.

Another aspect of the present disclosure is to provide a cyber threat information processing apparatus, a cyber threat information processing method, and a storage medium storing a cyber threat information processing program capable of identifying different attackers creating malware that performs significantly similar malicious activity and predicting a cyber threat attack occurring in the future.

Additional aspects will be set forth in part in the description which follows and, in part, will be apparent from the description, or may be learned by practice of the presented embodiments of the disclosure.

In accordance with an aspect of the present invention, the above and other objects can be accomplished by the provision of A method of processing cyber threat information, the method comprising: receiving input of a file or information on the file from a user through at least one interface; processing cyber threat information related to the received or input file or the information on the file; providing the processed cyber threat information to the user through a user interface; and performing natural language processing on the processed cyber threat information.

The method further comprises providing natural language obtained by the natural language processing to the user in a feed form.

In accordance with another aspect of the present invention, there is an apparatus for processing cyber threat information, the apparatus comprising: a database configured to store cyber threat information; and a server comprising a processor, wherein: the server receives input of a file or information on the file from a user through at least one interface, and the processor: processes cyber threat information related to the input file or the information on the file; provides the processed cyber threat information to the user through a user interface; and performs natural language processing on the processed cyber threat information.

Natural language obtained by the natural language processing is provided to the user in a feed form.

In accordance with a further aspect of the present invention, there is a computer-readable storage medium storing a cyber threat information processing program that executes computer instructions for: receiving input of a file or information on the file from a user through at least one interface; processing cyber threat information related to the received or input file or the information on the file; providing the processed cyber threat information to the user through a user interface; and performing natural language processing on the processed cyber threat information, wherein natural language obtained by the natural language processing is provided to the user in a feed form.

According to embodiments disclosed below, it is possible to detect and address malware not exactly matching data learned by machine learning and address a variant of malware.

According to the embodiments, it is possible to identify malware, an attack technique, and an attacker in a significantly short time even for a variant of malware, and furthermore to predict an attack technique of a specific attacker in the future.

According to the embodiments, it is possible to accurately identify a cyberattack implementation method based on whether such malware exists, an attack technique, an attack identifier, and an attacker, and provide the cyberattack implementation method as a standardized model. According to the embodiments, it is possible to provide information about malware, for which malware detection names, etc. are not unified or a cyberattack technique cannot be accurately described, using a normalized and standardized scheme.

In addition, it is possible to provide a means capable of predicting a possibility of generating previously unknown malware and attackers who can develop the malware, and predicting a cyber threat attack occurring in the future.

According to the embodiments, it is possible to more clearly detect and recognize different attack techniques or different attack groups generated according to differences in an execution process even when execution results of executed files are the same.

According to the embodiments, it is possible to identify cyber threat information, attack techniques, and attack

groups for various file types included in a file even when the file is a non-executable file, not an executable file.

According to the embodiments, it is possible to monitor a webpage, identify a webpage including a malicious action or information, and furthermore, identify cyber threat information, an attack technique, and an attack group included in the webpage.

BRIEF DESCRIPTION OF THE DRAWINGS

The accompanying drawings, which are included to provide a further understanding of the disclosure and are incorporated in and constitute a part of this application, illustrate embodiment(s) of the disclosure and together with the description serve to explain the principle of the disclosure. In the drawings:

FIG. 1 is a diagram illustrating an embodiment of a cyber threat information processing method;

FIG. 2 is a diagram illustrating an embodiment of a cyber threat information processing apparatus;

FIG. 3 is a diagram illustrating another embodiment of a cyber threat information processing apparatus;

FIG. 4 is a diagram illustrating an example of performing static analysis according to a disclosed embodiment;

FIG. 5 is a diagram illustrating an example of performing dynamic analysis according to a disclosed embodiment;

FIG. 6 is a diagram illustrating an example of disassembling malware to determine that a file includes malicious activity as an example of in-depth analysis;

FIG. 7 is a diagram illustrating a flow of processing cyber threat information according to a disclosed embodiment;

FIG. 8 is a diagram illustrating values obtained by converting opcode and ASM code into normalized code according to a disclosed embodiment;

FIG. 9 is a diagram illustrating vectorized values of opcode and ASM code according to a disclosed embodiment;

FIG. 10 is a diagram illustrating an example of converting a block unit of code into a hash value according to a disclosed embodiment;

FIG. 11 is a diagram illustrating an example of an ensemble machine learning model according to a disclosed embodiment;

FIG. 12 is a diagram illustrating a flow of learning and classifying data by machine learning according to a disclosed embodiment;

FIG. 13 is a diagram illustrating an example of performing labeling by identifying an attack identifier and an attacker using training data according to a disclosed embodiment;

FIG. 14 is a diagram illustrating a result of identifying an attack identifier according to an embodiment;

FIG. 15 is a diagram illustrating an example of matching an attack technique with code extracted from binary code according to a disclosed embodiment;

FIG. 16 is a diagram illustrating an example of matching an attack technique with a code set including opcode according to a disclosed embodiment;

FIG. 17 is a diagram for describing an example of identifying an attack technique and an attack group in units of functions;

FIG. 18 is a diagram for describing an example of identifying an attack technique and an attack group when a function is separated;

FIG. 19 is a diagram disclosing an example of obtaining characteristic information related to a cyber threat according to an embodiment;

5

FIG. 20 illustrates a process of obtaining a control flow using a branch instruction series according to an embodiment;

FIG. 21 is a diagram illustrating the case of generating an instruction sequence by combining instructions of a ControlBlock according to an instruction combining principle illustrated according to a second example;

FIG. 22 is a diagram for describing another example of generating instruction sequences including characteristic information using instructions in a ControlBlock;

FIG. 23 is a diagram for describing still another example of generating instruction sequences including characteristic information using instructions in a ControlBlock;

FIG. 24 is a diagram for describing yet another example of generating instruction sequences including characteristic information using instructions in a ControlBlock;

FIG. 25 is a diagram disclosing an example of generating an instruction sequence according to the above-described examples;

FIG. 26 is a diagram illustrating another embodiment of the disclosed cyber threat information processing apparatus; and

FIG. 27 is a diagram illustrating another embodiment of the disclosed cyber threat information processing method.

FIG. 28 is a diagram conceptually illustrating a structure of a non-executable file and a reader program for the non-executable file;

FIG. 29 is a diagram disclosing a block diagram of an embodiment capable of obtaining cyber threat information of a non-executable file;

FIG. 30 is a diagram disclosing an example of performing a first type of analysis of a file by being included in a file analysis unit in an exemplary diagram capable of obtaining cyber threat information of the file;

FIG. 31 is a diagram disclosing an example of performing a second type of analysis of a file by being included in the file analysis unit in an exemplary diagram capable of obtaining cyber threat information of the file;

FIG. 32 is a diagram illustrating an object extracted by dynamic execution of a non-executable file and extracted information by a second type of analysis for a file according to an embodiment;

FIG. 33 is a diagram disclosing an example of performing a third type of analysis of a file by being included in the file analysis unit in an exemplary diagram capable of obtaining cyber threat information of the file;

FIG. 34 is a diagram illustrating API hooking list information when a third analysis unit performs mild-dynamic analysis according to an embodiment;

FIG. 35 is a diagram for describing a feature processing unit in an embodiment capable of obtaining cyber threat information of a non-executable file;

FIG. 36 is an exemplary diagram comparing importance of feature information extracted from a non-executable file according to a disclosed embodiment;

FIG. 37 is an exemplary diagram for describing a classification model of an attack technique classifier according to a disclosed embodiment;

FIG. 38 is a diagram illustrating an attack technique identified by selectively combining various analytical techniques for a non-executable file according to a disclosed embodiment;

FIG. 39 is an exemplary diagram for describing a classification model of an attack group classifier according to a disclosed embodiment;

6

FIG. 40 is a diagram illustrating execution of the reader program of the non-executable file described above and system calls;

FIG. 41 is a diagram for describing an example of hooking a system call on program code according to an embodiment;

FIG. 42 is a diagram disclosing an example capable of tracing cyber threat information through dynamic analysis according to an embodiment;

FIG. 43 is a diagram illustrating another embodiment of the disclosed cyber threat information processing apparatus; and

FIG. 44 is a diagram illustrating another embodiment of the disclosed cyber threat information processing method.

FIG. 45 is a diagram disclosing an example of receiving input of or collecting webpage information and identifying malicious information based thereon in an embodiment;

FIG. 46 is a diagram illustrating an operation of a web collection unit (Web Crawler) according to an embodiment;

FIG. 47 is a diagram disclosing an example of storing and managing webpage data according to depth information of a disclosed embodiment;

FIG. 48 is a diagram disclosing an example of determining whether webpage data is malicious according to analysis of a plurality of steps or layers according to an embodiment;

FIG. 49 is a diagram illustrating a concept of analyzing webpage data and providing detected information according to an embodiment;

FIG. 50 is a diagram disclosing an example in which the above-disclosed embodiment operates on a computer; and

FIG. 51 is a diagram disclosing an embodiment of a method of processing cyber threat information included in a webpage.

FIG. 52 is a diagram disclosing an embodiment of a method of processing cyber threat information;

FIG. 53 is a diagram illustrating structure information based on tags of hypertext markup language (HTML) data as a method of processing cyber threat information according to an embodiment;

FIG. 54 is a diagram disclosing an example of obtaining feature information related to a cyber security threat from structure information based on a tag of HTML data as a method of processing cyber threat information according to an embodiment;

FIG. 55 is a diagram illustrating a process of processing and converting a part that may include cyber threat information, except for HTML grammar, in an HTML document according to an embodiment;

FIG. 56 is a diagram conceptually illustrating an example of a cyber threat information processing method according to an embodiment;

FIG. 57 is a diagram illustrating an example of an apparatus for processing cyber threat information included in a tag of a webpage according to an embodiment;

FIG. 58 illustrates an example of processing and providing cyber threat information according to the disclosed embodiments;

FIG. 59 illustrates another example of processing and providing cyber threat information according to the disclosed embodiments;

FIG. 60 illustrates another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments;

FIG. 61 illustrates another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments;

FIG. 79 is a diagram disclosing another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

FIG. 97 is a diagram disclosing another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

FIG. 98 is a diagram disclosing another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

DETAILED DESCRIPTION OF THE EMBODIMENTS

Hereinafter, embodiments will be described in detail with reference to the accompanying drawings. In the embodiments, a framework, a module, an application program interface, etc. may be implemented as a device coupled with a physical device or may be implemented as software.

When an embodiment is implemented as software, the software may be stored in a storage medium, installed in a computer, etc., and executed by a processor.

Embodiments of a cyber threat information processing apparatus and a cyber threat information processing method are disclosed in detail as follows.

FIG. 1 is a diagram illustrating an embodiment of a cyber threat information processing method. The embodiment of the cyber threat information processing method is described as follows.

A file input to a cyber threat information processing apparatus is preprocessed (S1000).

Identification information capable of identifying a file may be obtained through preprocessing of the file. An example of performing preprocessing of a file is as follows.

Various types of meta information may be obtained from a received file, including source information of the file, collection information for obtaining the file, and user information of the file. For example, when the file includes a uniform resource locator (URL) or is included in an e-mail, it is possible to obtain collection information for the file. The user information may include information about a user generating, uploading, or finally saving the file, etc. In a preprocessing process, as meta information of the file, it is possible to obtain internet protocol (IP) information, country information based thereon, API key information, for example, API information of a user requesting analysis, etc.

It is possible to extract a hash value of the file in the preprocessing process. When the hash value is previously known to the cyber threat information processing apparatus, a type of file or a degree of risk may be identified based on the hash value.

When the file is not previously known, analysis information for identifying the file type may be obtained by inquiring about pre-stored information or, if necessary, the hash value and file information on an external reference website. For example, information according to file type may be obtained from a site such as cyber threats analysis system (C-TAS) operated by Korea Internet & Security Agency, cyber threat alliance (CTA) operating system (OS), or Virus Total as the external reference website.

For example, it is possible to search for the file from the site by using a hash value of a hash function such as Message-Digest algorithm 5 (MD5), Secure Hash Algorithm 1 (SHA1), or SHA 256 of the file. In addition, the file may be identified using a search result.

As an example of performing file analysis, when an input file is transmitted through a mobile network, network transmission packet recombination technology, etc. is used for packets transmitted through network traffic, so that, when the input file is suspicious mobile malware, the file may be saved. The packet recombination technology recombines a series of packets corresponding to one piece of executable

code in the collected network traffic, and when a file transmitted by the recombined packets is suspicious mobile malware, this file is saved.

When the suspicious mobile malware is not extracted from the transmitted file in this step, it is possible to directly access a download URL in the file to download and save the suspicious mobile malware.

Malicious activity analysis information related to the input file is generated (S2000).

The malicious activity analysis information related to the input file may include static analysis information for analyzing information about the file itself or dynamic analysis information for determining whether malicious activity is performed by executing information obtained from the input file.

The analysis information in this step may include in-depth analysis information that uses information processed from an executable file related to the input file or performs memory analysis related to the file.

In-depth analysis may include AI analysis to accurately identify malicious activity.

The analysis information in this step may include correlation analysis information capable of estimating a correlation for attack activity or an attacker by correlating analysis information previously stored in relation to the file or generated analysis information with each other.

In this step, a plurality of pieces of analysis information may be aggregated to provide an overall analysis result.

For example, static analysis information, dynamic analysis information, in-depth analysis information, correlation analysis information, etc. for a single file may be integrated and analyzed for accurate attack technique and attacker identification. Integrated analysis removes an overlap between pieces of analysis information, and common information between pieces of analysis information may be used to increase accuracy.

For example, cyber threat infringement information (indicator of compromise, IoC) collected through several analyses and pathways may be standardized through normalization or enrichment of information.

In the embodiment of acquiring the analysis information, it is unnecessary to calculate all the analysis information described above in order. For example, any one of acquisition of the static analysis information and acquisition of the dynamic analysis information may be performed, and the dynamic analysis information may be acquired before the static analysis information.

The in-depth analysis information does not have to proceed after static analysis or dynamic analysis is performed, and correlation analysis may be performed without the in-depth analysis information.

Accordingly, the processing order for acquiring the analysis information may be changed, or acquisition may be selectively performed. In addition, the process of acquiring the analysis information and the process of generating the prediction information described above may be performed in parallel based on the information acquired from the file. For example, even when dynamic analysis is not completed, correlation analysis information may be generated. Similarly, dynamic analysis or in-depth analysis may be performed simultaneously.

In this case, the preprocessing process (S1000) exemplified above is for obtaining or identifying the information of the file, and thus, when static analysis, dynamic analysis, in-depth analysis, or correlation analysis is performed individually or in parallel, each preprocessing process may be performed as a part of each analysis step.

11

A detailed embodiment of this step will be described below.

Prediction information of malicious activity related to the input file may be generated (S3000).

In order to increase analysis accuracy, a data set of the various types of information analyzed above may be used to generate prediction information for whether malicious activity occurs, attack technique, an attacker group, etc.

The prediction information may be generated through AI analysis of a previously analyzed data set. The generation of the prediction information is not an essential step, and when an appropriately analyzed data set is prepared for AI analysis and a condition is satisfied, prediction information for malicious attack activity may be generated in the future.

An embodiment performs machine learning based on AI based on various types of analysis information. An embodiment may generate prediction information based on a data set for the analyzed information. For example, additional analysis information may be generated based on data learned by AI, and the regenerated analysis information may be used again as input data of AI as new training data.

Here, the prediction information may include malware creator information, malware tactic information, malware attack group prediction, malware similarity prediction information, and malware spread degree prediction information.

The generated prediction information may include first prediction information for predicting a risk level of the malware itself and second prediction information for predicting the attacker, attack group, similarity, spread degree, etc. of the malware.

Predictive analysis information including the first prediction information and the second prediction information may be stored in a server or a database.

A detailed embodiment thereof will be described below.

After post-processing of the analysis information or prediction information, cyber threat information related to the input file is provided (S4000).

The embodiment determines a type of malware and a risk level of the malware based on the analysis information or the prediction information. In addition, the embodiment creates profiling information for the malware. Therefore, it is possible to save a result of performing self-analysis on the file or a result of performing additional and predictive analysis through file analysis. The generated profiling information includes an attack technique for malware or labeling for an attacker.

The cyber threat information may include information on which preprocessing is performed, generated or identified analysis information, generated prediction information, aggregate information of these pieces of information, or information determined based on these pieces of information.

As for the provided cyber threat information, analysis information stored in a database in relation to the input file may be used, or the analyzed or predicted information may be included.

According to an embodiment, when a user inquires about not only malicious activity for an input file but also cyber threat information for a previously stored file or malicious activity, information thereon may be provided.

Such integrated analysis information may be stored in a standardized format in a server or database in response to the corresponding file. Such integrated analysis information may be stored in a standardized format and used for searching for or inquiring about cyber threat information.

An additional example of inquiring about the cyber threat information by the user will be described in detail below.

12

In addition, the illustrations of various user interfaces that provide real-time cyber threat information according to the embodiments of the present invention will be described below.

FIG. 2 is a diagram illustrating an embodiment of a cyber threat information processing apparatus. The embodiment of this figure conceptually illustrates the cyber threat information processing apparatus, and the embodiment of the cyber threat information processing apparatus will be described below with reference to this figure.

The disclosed cyber threat information processing apparatus includes a server 2100 and a database 2200, which are physical devices 2000, and a platform 10000 including an API running on the physical devices 2000. Hereinafter, the platform 10000 is referred to as a cyber threat intelligence platform (CTIP) or simply an intelligence platform 10000.

The server 2100 may include an arithmetic unit such as a central processing unit (CPU) or a processor, and may store or read data in the database 2200.

The server 2100 calculates and processes input security-related data, and executes a file to generate various security events and process related data. In addition, the server 2100 may control input/output of various cybersecurity-related data and store data processed by the intelligence platform 10000 in the database 2200.

The server 2100 may include a network device for data input or a network security device. The central processing unit, processor, or arithmetic unit of the server 2100 may execute a framework illustrated in the following drawings or a module within the framework.

The intelligence platform 10000 according to an embodiment provides an API for processing cyber threat information. For example, the intelligence platform 10000 may receive a file or data input from a network security device connected to a network or cyber malicious activity prevention programming software that scans for and detects malicious activity.

For example, the intelligence platform 10000 according to the embodiment may provide functions such as a security information and event management (SIEM) API that provides a security event, an environmental data retrieval (EDR) API that provides data about an execution environment, and a firewall API that monitors and controls network traffic according to a defined security policy. In addition, the intelligence platform 10000 may provide a function of an API of intrusion prevention systems (IPS) that perform a function similar to that of a firewall between internal and external networks.

An API 1100 of the intelligence platform 10000 according to an embodiment may receive files including malware that perform cybersecurity attack activities from various client devices 1010, 1020, and 1030.

The intelligence platform 10000 according to an embodiment may include a preprocessor (not illustrated), an analysis framework 1210, a prediction framework 1220, an AI engine 1230, and a postprocessor (not illustrated).

The preprocessor of the intelligence platform 10000 performs preprocessing to analyze cyber threat information on various files received from the client devices 1010, 1020, and 1030.

For example, the preprocessor may process a received file to obtain various types of meta information from the received file, including source information of the file, collection information for obtaining the file, and user information of the file. For example, when the file includes a URL or is included in an e-mail, it is possible to obtain collection information for the file. The user information may include

13

information about a user generating, uploading, or finally saving the file, etc. In a preprocessing process, as meta information of the file, it is possible to obtain IP information, country information based thereon, API key information, etc.

The preprocessor (not illustrated) of the intelligence platform **1000** may extract a hash value of the input file. When the hash value is previously known to the cyber threat information processing apparatus, the file type may be identified based thereon.

When the file is not previously known, analysis information for identifying the file type may be obtained by inquiring about the hash value and file information from reference Internet sites for cyber threat information such as operating C-TAS, an operating system of CTA, and Virus Total.

As described above, the hash value of the input file may be a hash value of a hash function such as MD5, SHA1, or SHA 256.

The framework **1210** may generate analysis information on the malware from the input file. The framework **1210** includes N modules (N is a natural number) (**1211**, **1213**, **1215**, . . . , **1219**) exemplarily illustrated in this drawing, which respectively analyze cyber threat information in various ways, such as static analysis, dynamic analysis, in-depth analysis, and correlation analysis.

Here, these various modules analyze cyber threat information included in the input files or predict cyber threat information.

The static analysis module included in the framework **1210** may analyze malware-related information on the file itself for the analysis information of the malicious activity related to the input file.

The dynamic analysis module included in the framework **1210** may analyze malware-related information by performing various activities based on various types of information obtained from the input file.

The in-depth analysis module included in the framework **1210** may analyze malware-related information by using information obtained by processing an executable file related to the input file or by performing memory analysis related to an executable file. The in-depth analysis module may include AI analysis to accurately identify malicious activity.

The correlation analysis module included in the framework **1210** may include correlation analysis information capable of estimating a correlation with attack activity or an attacker by correlating the previously stored analysis information or the generated analysis information in relation to the input file.

The framework **1210** may mutually combine the information analyzed from the static analysis module, the dynamic analysis module, the in-depth analysis module, and the correlation analysis module with analysis results for the characteristics and activities of the malware, and provide the combined final information to the user.

For example, the framework **1210** may perform integrated analysis of static analysis information, dynamic analysis information, in-depth analysis information, correlation analysis information, etc. for a single file to accurately identify the attack technique and attacker. The framework **1210** removes an overlap between pieces of analysis information and uses information common to pieces of analysis information to increase accuracy.

The framework **1210** may standardize the information provided, for example, by normalizing or enriching cyber threat infringement information (IoC) collected through

14

various analyses and paths. In addition, it is possible to generate analysis information on the final standardized malware or malicious activity.

The static analysis module, the dynamic analysis module, the in-depth analysis module, and the correlation analysis module of the framework **1210** may perform machine learning or deep learning techniques according to AI analysis on analysis target data to increase accuracy of the analyzed data.

The AI engine **1230** may perform an AI analysis algorithm to generate analysis information of the framework **1210**.

Such information may be stored in the database **2200**, and the server **2100** may provide analysis information on malware or malicious activity stored in the database **2200** as cyber threat intelligence information according to a user or client request.

The framework **1210** may include a plurality of prediction information generation modules according to prediction information, such as a first prediction information generation module and a second prediction information generation module. The framework **1210** may generate prediction information about whether malicious activity occurs, an attack technique, an attacker group, etc. by using the data set of the various types of information analyzed above in order to increase analysis accuracy.

The framework **1210** may generate prediction information for malicious activity related to the input file by performing an AI analysis algorithm using the AI engine **1230** based on the data set for the analysis information analyzed by the framework **1210**.

The AI engine **1230** generates additional analysis information by learning the data set for the analysis information through AI-based machine learning, and the additionally generated analysis information may be used again as AI input data as new training data.

The prediction information generated by the framework **1210** may include malware creator information, malware tactic information, malware attack group prediction, malware similarity prediction information, and malware spread degree prediction information.

As described above, the framework **1210** generating prediction information related to various malware or attack activities may store the generated prediction information in the database **2200**. In addition, the generated predicted information may be provided to the user according to a user request or attack symptom.

As described above, the server **2100** may provide the cyber threat information related to the input file after post-processing the analysis information or prediction information stored in the database **2200**.

The processor of the server **2100** determines the type of malware and the risk level of the malware based on the generated analysis information or prediction information.

The processor of the server **2100** may generate profiling information about the malware. The database **2200** may store a result of performing self-analysis on a file through file analysis or a result of performing additional and predictive analysis.

The cyber threat information provided to the user by the server **2100** may include information on which the preprocessing is performed, generated or identified analysis information, generated prediction information, aggregate information of these pieces of information, or information determined based on these pieces of information.

15

As for the provided cyber threat information, analysis information stored in a database in relation to the input file may be used, or the analyzed or predicted information may be included.

According to an embodiment, when a user inquires about not only malicious activity for an input file but also cyber threat information for a previously stored file or malicious activity, information thereon may be provided.

Such integrated analysis information may be stored in a standardized format in a server or database in response to the corresponding file. Such integrated analysis information may be stored in a standardized format and used for searching for or inquiring about cyber threat information.

An embodiment may analyze an input file and identify an attack activity from the analyzed file. The embodiment can identify the attacking activity in the file by matching the malicious code of the file with the detailed elements of the attacking activity commonly recognized by cyber security expert groups.

According to an embodiment, it is possible to identify the attack activity or attack technique (TTP) based on the database storing cyber threat information in the file and the matching relation for each attack activity or attack technique (TTP).

As an example of a database storing the attack activity of such a security expert group, a database storing information of MITRE ATT&CK, etc. may be exemplified. MITRE ATT&CK is a database on an actual security attack technique or activity, and by displaying specific security attack techniques or activities as components in a matrix format, attack techniques and activities may be identified in a specific data set format.

MITRE ATT&CK classifies content of attack techniques of hackers or malware for each attack stage and expresses the content as a matrix of common vulnerabilities and exposures (CVE) code.

The embodiment identifies specific attack activity among various attack activities by analyzing cyber threat information in the file, and allows an identified type of attack activity to be matched with attack code recognized by expert groups and actually performed, so that attack activity identification may be expressed by professional and commonly recognized elements.

In this embodiment, the server **2100** and the intelligence platform **10000** are described as different elements for convenience in describing the embodiment, but the intelligence platform **10000** may be performed by at least one processor in the server **2100**.

Meanwhile, the embodiments of processing cyber threat information may be included as hardware or software in various types of high-performance computing servers or distributed cloud servers and function as a part of the servers.

In this case, cyber threat information can be processed and provided from data or files included not only in communication between user clients and servers, but also in communication between servers or communication between servers and devices such as small terminals and vehicles according to the disclosed embodiments.

Since the embodiments disclosed below can be implemented with a miniaturized computing device or software, they are not limited to a specific location and may even be included in space vehicles such as satellites.

For example, it is possible to process data according to the embodiment below to determine what kind of cyber threat information is contained in data or files received by a satellite or a spacecraft.

16

In the following embodiments, when a device or software receives data, files, or information received from the outside, the embodiments in which cyber threat information is processed from the received data, files, or information and the result is provided to the user are disclosed in detail.

FIG. 3 is a diagram illustrating another embodiment of a cyber threat information processing apparatus.

The intelligence platform **10000** may include an API **1100**, a framework **18000**, an analysis and prediction module **18100** that executes various algorithms and execution modules, and an AI engine **1230**.

Here, an embodiment in which the intelligence platform **10000** analyzes and provides cyber threat information by receiving or collecting files is disclosed.

The intelligence platform **10000** may receive executable files from the client **1010** of a specific user. Here, the executable files such as EXE, ELF (Executable and Linkable Format), PE (Portable Executable), APK (Android Application Package), and the like are illustrated.

The intelligence platform **10000** may receive a non-executable file from the client **1020** of a specific user. Here, non-executable files are document files, script files, e-mails, etc. other than executable files that are directly executed. The non-executable files may also refer to embedded files that may include malicious codes or executable files.

Meanwhile, the server **2100** that operates the intelligence platform **10000** may itself directly collect various executable files or non-executable files such as external websites through internet.

The intelligence platform **10000** or the server **2100** running the intelligence platform **10000** analyzes cyber threat information from files received from users or directly collected, and provides various information so that the users can efficiently recognize attack activities or attack techniques (TTPs).

Hereinafter, embodiments in which cyber threat information processing devices such as the intelligence platform **10000** or server **2100** analyze executable files or non-executable files, and provide cyber threat information to users are sequentially disclosed.

Here, an embodiment in which a cyber threat information processing device such as the intelligence platform **10000** or the server **2100** analyzes an executable file is disclosed.

FIG. 4 illustrates an example of performing static analysis according to a disclosed embodiment. An example of a static analysis method for processing executable files according to an embodiment will be described with reference to the drawings.

As described, the type of file may be identified in a preprocessing step before performing static analysis or in an initial step of static analysis. This figure illustrates the case in which ELF, EXE, and ARK files are identified as types of files for convenience. However, application of the embodiment is not limited thereto.

Static analysis or detection of malware may be performed based on a process of comparing the characteristics of the file itself with a previously identified pattern database.

A static information extractor may obtain structure information by parsing a structure of the input file.

A pattern in the structure of the parsed file may be compared with a pattern of malware previously stored in the database (DB) **2200**.

The structure characteristics and patterns of the parsed file may be meta information of the parsed file.

Although not illustrated in the example disclosed above, a machine learning engine may be used in the static analysis

of the disclosed embodiment. The database 2200 may store a data set including the learned characteristics of the previously stored malware.

The AI engine may learn meta information obtained from the parsed file through machine learning, and compare the meta information with a data set previously stored in the database 2200 to determine whether the file is malware.

Structural characteristics of a file analyzed as malware through static analysis may be saved again as a data set related to the malware.

FIG. 5 illustrates an example of performing dynamic analysis according to a disclosed embodiment. An example of a dynamic analysis method for processing executable files according to an embodiment will be described with reference to the drawings.

As described, the type of file may be identified in a preprocessing step before performing dynamic analysis or in an initial step of the dynamic analysis. Similarly, in this example, the case where ELF, EXE, and ARK files are identified as types of files is illustrated for convenience.

Through preprocessing, a type of file subjected to dynamic analysis may be identified. The identified file may be executed in a virtual environment according to a sort and type of each file.

For example, when the identified file is an ELF file, the file may be executed in an operating system of a Linux virtual environment (virtual machine, VM) through a queue.

An event that occurs when the ELF file is executed may be recorded in an activity log.

In this way, Windows, Linux, and mobile operating systems are virtually built for each type of identification file, and then an execution event of a virtual system is recorded.

In addition, execution events of the malware previously stored in the database 2200 may be compared with recorded execution events. Although not illustrated above, in the case of dynamic analysis, execution events recorded through machine learning may be learned, and it may be determined whether the learned data is similar to execution events of previously stored malware.

In the case of dynamic analysis, a virtual environment needs to be constructed according to the file, which can increase the size of the analysis and detection system.

FIG. 6 illustrates an example of disassembling malware to determine that a file includes malicious activity as an example of the in-depth analysis.

As described above, when the executable file is disassembled, opcode and ASM code, which are assembly language code types, may be obtained.

For example, a specific function A in an EXE executable file may be converted into disassembled code including opcode or disassembled code through a disassembler.

When the EXE executable file is malware causing malicious activity, disassembled code set causing the malicious activity may be obtained by disassembling a function or code segment that causes such activity.

The disassembled code set may include opcode set or a set combining opcode and ASM code corresponding to the malicious activity or malware.

Even when the malicious activity is the same, since a disassembly result of the executable file or an algorithm of the malware causing the activity to be performed is not exactly the same, whether the input malware corresponds to a specific disassembled code set may be identified through AI-based similarity analysis.

This malicious activity corresponding to a specific disassembled code set may be used to identify an attack technique

(TTP) by being matched with a professional and public tactic or attack technique such as MITRE ATT&CK.

Alternatively, an opcode set or a set combining opcode and ASM code in a specific disassembled code may be used to determine an attack technique by being matched with the attack technique elements defined in MITRE ATT&CK.

This figure illustrates an example in which the executable file, the disassembled code set of the executable file, and the attack technique corresponding to the attack technique elements in the MITRE ATT&CK correspond to each other.

FIG. 7 is a diagram illustrating a flow of processing cyber threat information according to a disclosed embodiment.

A case where the file identified in this figure is an executable binary file of ELF, EXE, and ARK will be described as an example. The processing of this step is related to the in-depth analysis described above.

First, a detailed example of a process of extracting the disassembled code including the opcode code as a first step will be described as follows.

When source code is compiled, an executable file is created.

The raw source code is generated as new data in a form suitable for processing by a machine by a compiler in each executable OS environment. The newly constructed binary data is in a form that is not suitable for human reading, and thus it is impossible for a human to understand the internal logic by interpreting the file created in the form of an executable file.

However, a reverse process is performed for vulnerability analysis of the security system and for various purposes to perform interpretation or analysis of machine language, which is referred to as a disassembly process as described above. The disassembly process may be performed according to a CPU of a specific operating system and the number of processing bits (32-bit, 64-bit, etc.).

Disassembled assembly code may be obtained by disassembling each of the illustrated ELF, EXE, and ARK executable files.

The disassembled code may include code in which opcode and ASM code are combined.

The embodiment may extract the opcode and ASM code from an executable file by analyzing the executable file based on a disassembly tool.

The disclosed embodiment does not use the extracted opcode and ASM code without change, and reconstructs the opcode array by reconstruction for each function. When the opcode array is rearranged, the data may be reconstructed so that the data may be sufficiently interpreted by including the original binary data. Through this rearrangement, the new combination of the opcode and the ASM code provides basic data that can identify the attacker as well as the attack technique.

A process (ASM) of processing assembly data as a second step will be described in detail as follows.

Assembly data processing is a process of analyzing similarity and extracting information based on data reconstructed in a human or computer-readable form after separating only the opcode and the necessary ASM code.

In this step, the disassembled assembly data may be converted into a certain data format.

Such conversion of the data format may be selectively applied without needing to apply all of the conversion methods described below to increase data processing speed and accurately analyze data.

Various functions may be extracted from the assembly data of the rearranged opcode and ASM code combination.

When one executable file is disassembled, it is possible to include, on average, about 7,000 to 12,000 functions, depending on the size of the program. Some of these functions are implemented by a programmer as needed, and some of the functions are provided by default in the operating system.

When the actual ASM code is analyzed, about 87% to 91% of the functions are basically provided by the operating system (OS supported), and the ASM code actually implemented by the programmer for the program logic is about 10%. The functions provided by the operating system are functions included in various DLL and SO files basically installed when the operating system is installed along with function names (default functions). These operating system-provided functions may be previously analyzed and stored to be filtered from analysis target data. By separating only code to be analyzed in this way, processing speed and performance may be increased.

In the embodiment, in order to accurately perform functional analysis of a program, the opcode may be processed by being separated into function units. The embodiment may perform the minimum unit of all semantic analysis based on a function included in assembly code.

In order to increase analysis performance and processing speed, the embodiment may filter out operator-level functions having inaccurate meaning, and remove functions having the information amount smaller than a threshold value from analysis. Whether or not to filter the functions and a degree of filtering may be set differently depending on the embodiment.

The embodiment may remove annotation data provided by the disassembler during output from the opcode organized according to the function. In addition, the embodiment may rearrange the disassembled code.

For example, the disassembled code output by the disassembler may have the order of [ASM code, opcode, and parameter].

The embodiment may remove parameter data from the assembly data and rearrange or reconstruct the disassembled code of the above order in the order of [opcode and ASM code]. The reassembled disassembled code is easy to process by being normalized or vectorized. In addition, the processing speed may be significantly increased.

In particular, in disassembled code having a combination of [opcode and ASM code], an ASM code segment has different data lengths, making comparison difficult. Therefore, in order to check uniqueness of the corresponding assembly data, the data may be normalized into a data format of a specific size. For example, in order to check uniqueness of the disassembled code of the [opcode and ASM code] combination, the embodiment may convert a data part into a data set of a specific length that is easy to normalize, for example, cyclic redundancy check (CRC) data.

As an example, in the disassembled code of the [opcode and ASM code] combination, it is possible to convert an opcode segment into CRC data of a first length and an ASM code segment into CRC data of a second length, respectively.

Normalized data converted from the opcode and the ASM code may maintain uniqueness of each code before the corresponding conversion, respectively. Vectorization may be performed on the normalized data in order to increase similarity determination speed of the normalized data converted with uniqueness.

As described above, normalization or vectorization processes as a data conversion process may increase data processing speed and selectively apply accurate data analysis.

Detailed examples of the normalization process and the vectorization process are again described in detail below.

As a third step, a process of analyzing data for analyzing the disassembled code will be described in detail as follows.

In this process, conversion of various data formats may be used to increase data processing speed and to accurately analyze data. Some of the conversion methods described below may be selectively applied without the need to apply all the methods.

This step is a step of analyzing the malware and similarity based on a data set for each function in converted disassembled code based on the converted data.

The embodiment may convert vectorized opcode and ASM code data sets back into byte data in order to perform code-to-code similarity operation.

Based on the byte data converted again, a block-unit hash value may be extracted, and a hash value of the entire data may be generated based on the block-unit unique value.

The hash value may be compared by extracting a hash value of a unit designated to extract a unique value of each block unit in order to efficiently perform block-unit comparison, which is a part of byte data.

A fuzzy hashing technique may be used to extract the hash value of the designated unit and compare similarity of two or more pieces of data. For example, the embodiment may determine similarity by comparing a hash value extracted in block units with a hash value in some units in a pre-stored malware using the CTPH method in fuzzy hashing.

In summary, the embodiment generates a unique value of disassembled code of the opcode and the ASM code in order to confirm uniqueness of each specific function based on the fact that the combination code of the opcode and the ASM code implements specific functions in units of functions. In addition, it is possible to perform a similarity operation by extracting a unique value in block units in the opcode and the ASM code of the disassembled code based on this unique value.

A detailed example of extracting a block-unit hash value will be disclosed with reference to the drawings below.

As described above, the embodiment may use a block-unit hash value when performing a similarity operation.

The extracted block-unit hash value includes String Data (Byte Data), and String Data (Byte Data) is numerical values enabling comparison of similarity between codes. When comparing bytes of billions of disassembled code data sets, a significantly long time may be consumed to obtain a single similarity result.

Therefore, according to the embodiment, String Data (Byte Data) may be converted into a numerical value. Based on the numerical value, similarity analysis can be rapidly performed using AI technology.

The embodiment may vectorize String Data (Byte Data) of the hash value of the extracted block unit based on N-gram data. The embodiment of this figure illustrates the case in which a block-unit hash value is vectorized into 2-gram data in order to increase the operation speed. However, in the embodiment, it may be unnecessary to convert the block-unit hash value into 2-gram data, and the block-unit hash value may be vectorized and converted into 3-gram, 4-gram, . . . , N-gram data. In N-gram data, as N increases, the characteristics of the data may be accurately reflected. However, the data processing time increases.

As described above, in order to increase the data processing speed and to accurately analyze data, byte conversion, hash conversion, and N-gram conversion below may be selectively applied.

The illustrated 2-gram conversion data has a maximum of 65,536 dimensions. As the dimension of the training data increases, a distribution of the data becomes sparse, which may adversely affect classification performance. In addition, as the dimension of the training data increases, temporal complexity and spatial complexity for learning the data increase.

The embodiment may address this problem by various natural language processing algorithms based on various text expressions. In this embodiment, Term Frequency-Inverse Document Frequency (TF-IDF) technique will be described as an example of such an algorithm.

As an example for processing the similarity of the training data in this step, when determining an attack identifier or class (T-ID) from high-dimensional data, the TF-IDF technique may be used to select a meaningful feature (pattern). In general, the TF-IDF technique is used to find documents having high similarity in a search engine, and equations for calculating this value are as follows.

$$tf(t, d) = \frac{f_{t,d}}{\sum_{r \in d} f_{r,d}} \quad [\text{Equation 1}]$$

Here, $tf(t,d)$ denotes a frequency of a specific word r in a specific document d , and has a higher value as the word repeatedly appears.

$$idf(t, D) = \log \frac{N}{|\{d \in D: t \in d\}|} \quad [\text{Equation 2}]$$

$idf(t, D)$ denotes a reciprocal value of a proportion of the document d including the specific word t , and has a lower value as the word appears more frequently in several documents.

$$td - idf(t, d, D) = tf(t, d) \times idf(t, D) \quad [\text{Equation 3}]$$

$tf-idf(t,d,D)$ is a value obtained by multiplying $tf(t,d)$ by $idf(t,D)$, and may quantify which word is more suitable for which document.

The TF-IDF method is a method of using a word frequency according to Equation 1 and an inverse document frequency (inverse number specific to the frequency of the document) according to Equation 2 to reflect a weight according to an importance of a word in a document word matrix as in Equation 3.

In an embodiment, a document including a corresponding word may be inferred as an attack identifier (T-ID) based on a characteristic or pattern of a word in block-unit code. Therefore, when the TF-IDF is calculated with respect to a pattern extracted from the block-unit code, a pattern that appears frequently within a specific attack identifier (T-ID) may be extracted, or code having a pattern unrelated to the specific attack identifier (T-ID) may be removed.

For example, assuming that a specific pattern A is a pattern expressed in all attack identifiers (T-IDs), a TF-IDF value for the specific pattern A may be measured low. In

addition, it may be determined that such a pattern is an unnecessary pattern to distinguish an actual attack identifier (T-ID). An algorithm for determining similarity of natural language, such as TF-IDF, may be performed through learning of a machine learning algorithm.

The embodiment may reduce unnecessary calculations and shorten inference time by removing such an unnecessary pattern.

In detail, the embodiment may perform a similarity algorithm based on text representation of various types of natural language processing on the converted block-unit code data. Through the similarity algorithm, by removing the code of the pattern unrelated to the attack identifier, execution of the algorithm performed below and execution of the classification process according to machine learning may be greatly shortened.

The embodiment may perform classification modeling to classify a pattern of an attack identifier based on a feature or pattern on block-unit code. The embodiment may learn whether a vectorized block-unit code feature or pattern is a pattern of a known attack identifier, and classify the code feature or pattern by an accurate attack technique or implementation method. The embodiment uses various ensemble machine learning models to categorize an accurate attack implementation method, that is, an attack identifier and an attacker, for code determined to have a code pattern similar to that of malware.

The ensemble machine learning models are techniques that generate several classification nodes from prepared data, and combine node predictions for each classification node, thereby performing accurate prediction. As described above, the ensemble machine learning models that classify the attack implementation method of the word feature or pattern in the block-unit code, that is, the attack identifier or the attacker, are performed.

When applying the ensemble machine learning models, a threshold value for classification of prepared data may be set to prevent excessive detection and erroneous detection. Only data above the set detection threshold value may be classified, and data that does not reach the set detection threshold value may not be classified.

As described, conversion of several data formats may be used to increase the data processing speed and to accurately analyze the data. A specific embodiment in which the above-described data conversion method is applied to ensemble machine learning models will be described in detail below.

As a fourth step, a profiling process for identifying and labeling an attack technique (TTP) will be described as follows.

An example of vectorizing through extraction of a feature of disassembled code including opcode and ASM code of input binary data based on an previously analyzed attack code or malware has been described above.

The vectorized data is classified as a specific attack technique after being learned through machine learning modeling, and the classified data is labeled in a profiling process for classified code.

Labeling may be largely performed in two parts. One is to attach a unique index to an attack identifier defined in a standardized model, and the other is to write information about a user creating attack code.

Labeling is assigned according to an attack identifier (T-ID) reflected in a standardized model, for example, MITRE ATT&CK, so that accurate information may be delivered to the user without additional work.

In addition, labeling is assigned to distinguish not only an attack identifier but also an attacker implementing the attack identifier. Therefore, labeling may be provided so that it is possible to identify not only an attack identifier, but also an attacker and an implementation method accordingly.

In an embodiment, advanced profiling is possible based on data learned from a data set of disassembled code (opcode, ASM code, or a combination thereof) previously classified. In an embodiment, data of the static analysis, dynamic analysis, or correlation analysis disclosed above may be utilized as reference data for performing labeling. Therefore, even when a data set has not been previously analyzed, profiling data may be obtained significantly rapidly and efficiently by considering results of static, dynamic, and correlation analysis together.

The process of learning code having a pattern similar to that of the malware and classifying the learned data in the third step and the profiling process of the classified data in the fourth step may be performed together by an algorithm in machine learning.

A detailed example thereof is disclosed below. In addition, an actual example of the profiled data set is illustrated with reference to the drawings below.

FIG. 8 is a diagram illustrating values obtained by converting opcode and ASM code of disassembled code into normalized code according to a disclosed embodiment.

As described above, when the executable file is disassembled, data, in which opcode and ASM code are combined, is output.

The embodiment may remove annotation data output for each function from the disassembled data and change the arrangement order of the opcode, ASM code, and corresponding parameter to facilitate processing.

The reconstructed opcode and ASM code are changed to normalized code data, and the example of this figure illustrates CRC data as normalized code data.

For example, the opcode may be converted into CRC-16 and the ASM code may be converted into CRC-32.

In a first row of an illustrated table, a push function of the opcode is changed to CRC-16 data of 0x45E9, and 55 of the ASM code is changed to CRC-32 data of 0xC9034AF6.

In a second row, a mov function of the opcode is changed to CRC-16 data of 0x10E3, and 8B EC of the ASM code is changed to CRC-32 data of 0x3012FD2C. In a third row, a lea function of the opcode is changed to CRC-16 data of 0xAACE, and 8D 45 0C of the ASM code is changed to CRC-32 data of 0x9214A6AA.

In a fourth row, a push function of the opcode is changed to CRC-16 data of 0x45E9, and 50 of the ASM code is changed to CRC-32 data of 0xB969BE79.

Unlike this example, it is possible to use normalized code data different from CRC data or code data having a different length.

When the disassembled code is changed to a normalized code in this way, it is possible to easily and rapidly perform subsequent calculation, similarity calculation, and vectorization while ensuring uniqueness of each code.

FIG. 9 is a diagram illustrating vectorized values of opcode and ASM code of disassembled code as an example of data conversion of a disclosed embodiment.

This figure illustrates results of vectorizing code of a normalized opcode (CRC-16 according to the example) and a normalized ASM code (CRC-32 according to the example), respectively.

A vectorized value of the code of the normalized opcode (opcode Vector) and a vectorized value of the code of the normalized ASM code (ASM code Vector) are illustrated in a table format in this figure.

The opcode vector value and the ASM code vector value of each row of this figure correspond to the normalized value of the opcode and the normalized value of the ASM code of each row illustrated above, respectively.

For example, vectorized values of CRC data 0x45E9 and 0xB969BE79 in the fourth row of the table are 17897 and 185 105 121 44 in a fourth row of the table of this figure, respectively.

When vectorization is performed on the normalized data in this way, the disassembled opcode function and ASM code are changed to vectorized values while each including unique features.

FIG. 10 is a diagram illustrating an example of converting a block unit of code into a hash value as an example of data conversion of a disclosed embodiment.

In order to perform similarity analysis, the vectorized data set of each of the opcode and the ASM code is reconverted into a byte data format. The reconverted byte data may be converted into a block-unit hash value. Further, based on the hash values in the block unit, a hash value of the entire reconverted byte data is generated again.

In an embodiment, to calculate the reconverted hash value, hash values such as MD5 (Message-Digest algorithm 5), SHA1 (Secure Hash Algorithm 1), and SHA 256 may be used, and a fuzzy hash function for determining similarity between pieces of data may be used.

The first row of the table in this figure represents human-readable characters that may be included in the data. In the reconverted byte data, a value included in a block unit may include such readable characters.

The characters may each correspond to 97, 98, 99, 100, . . . , 48, 49, which are ASCII values (ascii val) in a second row.

Data including character values in a first row may be segmented and separated into blocks in which ASCII values can be summed.

A third row of the table shows the sum of ASCII values corresponding to respective character values within a block unit having 4 characters.

The first block may have a value of 394, which is the sum (ascii sum) of ASCII values (ascii val) 97, 98, 99, and 100 corresponding to the characters in the block.

In addition, the last row shows the case where the sum of ASCII values in block units is converted into base-64 expression. The letter K is the sum of the first block.

In this way, a signature referred to as Kaq6KaU may be obtained for the corresponding data.

Based on such a signature, it is possible to calculate similarity of two pieces of block-unit data.

In this embodiment, a hash value may be calculated using a fuzzy hash function for determining similarity for block units included in code in reconverted byte data, and similarity may be determined based on the calculated hash value. Even though context triggered piecewise hashing (CTPH) is illustrated as a fuzzy hash function for determining similarity, it is possible to use other fuzzy hash functions that can calculate similarity of data.

FIG. 11 is a diagram illustrating an example of an ensemble machine learning model according to a disclosed embodiment.

An embodiment may accurately classify an attack identifier (T-ID) of a file determined to be malware by using an ensemble machine learning model.

The hash value of the block unit including String Data (Byte Data) may be digitized based on N-gram characteristic information, and then similarity may be calculated using a technique such as TF-IDF to determine whether the value is an attack identifier (T-ID) or a class to be classified.

In order to increase performance of identifying an attack technique by reducing unnecessary operations, the embodiment may remove unnecessary patterns based on similarity among the hash values.

In addition, attack identifiers may be classified by modeling data, from which unnecessary patterns are removed, through ensemble machine learning.

There are methods such as voting, bagging, and boosting as a method of combining learning results of several classification nodes of an ensemble machine learning model. An ensemble machine learning model that properly combines these methods may contribute to increasing classification accuracy of training data.

Here, a method of more accurately classifying an attack identifier will be described by taking the case of applying the random forest method of the bagging method as an example.

The random forest method is a method of generating a large number of decision trees to reduce classification errors due to a single decision tree and obtaining a generalized classification result. An embodiment may apply a random forest learning algorithm using at least one decision tree for prepared data. Here, the prepared data refers to data from which unnecessary patterns are removed from the fuzzy hash value in block units.

A decision tree model having at least one node is performed to determine similarity of a block-unit hash value. It is possible to optimize a comparison condition for a feature value (here, the number of expressions of classification patterns based on block-unit hash values) capable of distinguishing one or more classes (attack identifier; T-ID) according to a degree of information gain of a decision tree.

To this end, a decision tree illustrated in the figure may be generated.

In this figure, upper quadrilaterals **2510**, **2520**, **2530**, and **2540** are terminal nodes indicating conditions for classifying classes, and the lower quadrants **2610**, **2620**, and **2630** indicate classes classified as terminal nodes.

For example, when a random forest model is applied as an ensemble machine learning model, the model is a classification model that uses an ensemble technique using one or more decision trees. Various decision trees are constructed by varying characteristics of input data of a decision tree included in the random forest model. Classification is performed on several generated decision tree models, and a final classification class is determined using a majority vote technique. A test of each node may be performed in parallel, resulting in high computational efficiency.

When classifying a class, threshold values are set to prevent excessive detection and erroneous detection, a value less than a lower threshold value is discarded, and classification may be performed for data of a detection threshold value or more.

FIG. 12 is a diagram illustrating a flow of learning and classifying data by machine learning according to a disclosed embodiment.

Profiling of input data may include a classification step (**S2610**) and a learning step (**S2620**).

In an embodiment, the learning step (**S2620**) may include (a) a hash value extraction process, (b) an N-gram pattern extraction process, (c) a natural language processing analysis (TF-IDF analysis) process, (d) a pattern selection process, (e) a model learning process, etc.

Further, in an embodiment, the classification step (**S2610**) may include (a) a hash value extraction process, (b) an N-gram pattern extraction process, (f) a pattern selection process, (g) a classification process by vectorization, etc.

The classification step (**S2610**) in a profiling step according to the embodiment will be first described as follows.

Input data is received from an executable file set or processed files.

Input data is received from executable file sets stored in the database, or input data including an executable file delivered from the processing process illustrated above is received. The input data may be data obtained by converting disassembled code including opcode and ASM code, and may be vectorized data.

A fuzzy hash value is extracted from the disassembled code, which is the input data, (a), and N-gram pattern data for a specific function is extracted (b). In this case, 2-gram pattern data including patterns determined to be similar to malware among the existing semantic pattern sets may be selected (f).

The N-gram data of the selected pattern may be converted into vectorized data, and the vectorized data may be classified as a function, a semantic pattern of which is determined, (g).

The learning step (**S2620**) in the profiling step according to the embodiment is performed as follows.

When input data is a new file, a fuzzy hash value is extracted from disassembled code that is the input data (a).

The extracted fuzzy hash value is vectorized into N-gram data (2-gram in this example) (b).

Natural language processing analysis such as TF-IDF is performed on an extracted specific pattern (c).

A data set having high similarity is selected among data sets having patterns related to an existing attack identifier (T-ID), and the remaining data sets are filtered (d). In this instance, it is possible to select sample data sets including some or all features of the data sets having patterns related to the attack identifier (T-ID) by comparing with data sets stored in an existing semantic pattern set.

It is possible to learn vectorized N-gram data based on the extracted sample data set (e).

A probability is obtained for each attack identifier (T-ID) by inputting the vectorized N-gram data into the classification model. For example, it is possible to obtain A % as a probability that vectorized data of an N-gram structure is a specific attack identifier (T-ID) T1027, and obtain (100-A) % as a probability that vectorized data of an N-gram structure is an attack identifier T1055.

An ensemble machine learning model such as a random forest including at least one decision tree may be used as the classification model.

Here, it is possible to determine an attack technique or attacker of the vectorized N-gram data based on the classification model.

Labeling is performed by classifying input data according to a classification result of the classification model (e) or a selection (f) result of the existing stored pattern (g).

A result of final labeling is illustrated with reference to the following drawings.

FIG. 13 is a diagram illustrating an example in which an attack identifier and an attacker are labeled by learning and classifying input data according to a disclosed embodiment.

This figure is a diagram illustrating each of an attack identifier, an attacker or an attack group, a fuzzy hash value corresponding to assembly code, and an N-gram corresponding thereto (indicated as 2-gram data here) in tabular form as a result of the profiler.

According to an embodiment, when profiling is completed, it is possible to obtain classified data in relation to implementation of the following tactic.

According to profiling according to the embodiment, it is possible to perform labeling with an attack identifier (T-ID) and an attacker or an attacker group (Attacker or Group).

Here, the attack identifier (T-ID) may follow the standardized model as described. In this example, a result of assigning the attack identifier (T-ID) provided by MITRE ATT&CK® is exemplified.

Labeling may be added to the identified attacker or attacker group (Attacker or Group) as described above. This figure illustrates an example in which the attacker TA504 is identified by labeling of the attacker or attacker group (Attacker or Group).

SHA-256 (size) indicates a fuzzy hash value and data size of malware corresponding to each attack identifier (T-ID) or attacker group (Attacker or Group). As described above, such malware may correspond to the rearrangement and combination of opcode and ASM code.

In addition, a value of a section marked with N-gram is N-gram pattern data corresponding to the attack identifier (T-ID) or the attacker group and a fuzzy hash value of malware, and is displayed as a part of 2-gram data in this example.

As illustrated in this figure, fuzzy hash values of malware (opcode and ASM code) and attack identifiers (T-IDs) or attacker groups corresponding to N-gram pattern data may be labeled and stored.

The illustrated labeled data may be used as reference data for ensemble machine learning, and may be used as reference data for a classification model.

FIG. 14 is a diagram illustrating a result of identifying an attack identifier according to an embodiment.

This figure illustrates a Euclidean distance matrix, which may represent similarity between two data sets.

In this figure, a bright part indicates that the similarity between the two data sets is low, and the dark part indicates that the similarity between the two data sets is high.

In this figure, T10XX denotes an attack identifier (T-ID), and characters T, K, and L in parentheses denote an attacker group creating an attack technique according to the corresponding attack identifier (T-ID).

That is, the row and column indicate attack identifiers (T-IDs) generated by respective attacker groups (T, K, and L), and row and column have the same meaning. For example, T1055 (K) indicates an attack T1055 created by the attacker group L, and T1055 (K) indicates the same tactic T1055 created by the attacker group K.

Since samples of each data set include the samples, when distances from other samples are calculated respectively, a distribution, in which uniformity is high in a diagonal direction from the top left to the bottom right, is obtained.

Referring to this figure, it can be seen that the same attack identifier (T-ID) exhibits similar characteristics even when the attacker groups are different. For example, even when the attack group is T or K, the attack identifier of T1027 may have high similarity when the attack technique is similar.

Therefore, when learning is carried out based on the extracted data set as in the above embodiment, it can be found that the characteristics of the same attack technique (T-ID) implemented by the same attacker are clearly identified (darkest part), and similarity of the same attack technique (T-ID) implemented by other attackers is high (middle dark part).

Therefore, when the attack technique is classified by extracting and applying the sample data based on the com-

bination of the opcode and the ASM code in this way, even if the attacker is different, a specific attack technique or identifier (T-ID) may be reliably classified. Conversely, by the combination of the opcode and the ASM code, it is possible to clearly identify specific code implemented inside malware, as well as identify an attack implementation method including an attacker and an attack identifier.

FIG. 15 illustrates an example of matching an attack technique with code extracted from binary code according to a disclosed embodiment. Here, an example of using a standardized model as an example of matching an attack technique is disclosed.

Here, MITRE ATT&CK® Framework is exemplified as a standardized model.

For example, in terms of cybersecurity, “malicious activity” is interpreted differently depending on the analyst, and is interpreted differently depending on the insight of each person in many cases.

Internationally, many efforts are being made among experts to standardize “malicious activity” that occurs on the system and to ensure that everyone makes the same interpretation. MITRE (<https://attack.mitre.org>), a non-profit R&D organization that performs national security-related tasks with support from the US federal government, studied the definition of “malicious activity” and created and announced the ATT&CK® Framework. This framework was defined so that everyone can define the same “malicious activity” for cyber threats or malware.

MITRE ATT&CK® Framework (hereinafter referred to as MITRE ATT&CK®) is an abbreviation of Adversarial Tactics, Techniques, and Common Knowledge, which summarizes latest attack technology information of attackers. MITRE ATT&CK® is standardized data obtained by analyzing tactics and techniques of adversary behaviors of an attacker after observing actual cyberattack cases to classify and list information on the attack techniques of various attack groups.

MITRE ATT&CK® is a systematization (patterning) of threatening tactics and techniques to improve detection of advanced attacks with a slightly different point of view from the concept of the traditional Cyber Kill Chain. Originally, ATT&CK started by documenting TTP, which are methods (Tactics), techniques, and procedures, for hacking attacks used in corporate environments using Windows operating systems in MITRE. Since then, ATT&CK has developed into a framework that may identify activity of the attacker by mapping TTP information based on analysis of a consistent attack activity pattern generated by the attacker.

The malicious activity mentioned in the disclosed embodiment may be expressed by matching the malware to the attack technique based on a standardized model such as MITRE ATT&CK®, and the malware may be identified and classified for each element and matched to an attack identifier regardless of the standardized model.

The example of this figure conceptually illustrates a scheme of matching the malicious activity of the malware to the attack technique based on the MITRE ATT&CK model.

An executable file EXE may include various functions (Function A, B, C, D, E, . . . , N, . . . , Z) executed when the file is executed. A function group including at least one of the functions may perform one tactic.

In the example of this figure, functions A, B, and C correspond to tactic A, and functions D, B, and F correspond to tactic B. Similarly, functions Z, R and C correspond to tactic C, and functions K and F correspond to tactic D.

The embodiment may match a set of functions corresponding to each tactic and a specific disassembled code

segment. The database stores attack identifiers (T-IDs) of tactics, techniques, and procedures (TTP) that can correspond to disassembled code previously learned by AI.

Attack identifiers (T-IDs) of tactics, techniques, and procedures (TTP) follow a standardized model, and the example in this figure illustrates MITRE ATT&CK® as a standardized model of cyber threat information.

Accordingly, the embodiment may match result data extracted from the disassembled code in the binary file with the standardized attack identifier. A more specific scheme of matching an attack identifier is disclosed below.

FIG. 16 is a diagram illustrating an example of matching an attack technique with a code set including opcode according to a disclosed embodiment.

Most AI engines use a data set learned based on various characteristic information of malware to identify the malware. Then, whether the malware is malicious is determined. However, in this way, it is difficult to describe a reason why the malware is malware. However, as illustrated, when the standardized tactic (TTP) identifier is matched, it is possible to identify a type of threat included in the malware. Accordingly, the embodiment may accurately deliver cyber threat information to a security administrator and enable the security administrator to systematically manage cyber threat information over the long term.

When generating a dataset for AI learning to identify a tactic (TTP) based on the disassembled code, the embodiment not only distinguishes only the identifier or labeling of the tactic (TTP), but also can reflect characteristics of a scheme of implementing the tactic (TTP) as an important factor.

Even malware that implements the same tactic (TTP) is impossible to generate with the same code depending on the developer. That is, even though the tactic (TTP) is described in the form of human oral language, an implementation method and a code writing method are not the same depending on the developer.

Such a difference in coding depends on the ability of the developer or scheme or habit of implementing the program logic, and this difference is expressed as a difference between binary code or opcode and ASM code obtained by disassembling the binary code.

Therefore, when an attack identifier is simply assigned or matched according to the type of the resulting tactic (TTP), it is difficult to accurately identify an attacker or a group of attackers generating the malware.

Conversely, when modeling is performed by reflecting the characteristics of the disassembled opcode and ASM code as important variables, it is possible to identify a developer developing specific malware or a specific attack tool, or even an automatically created tool itself.

The disclosed embodiment may generate threat intelligence, which is significantly important in modern cyber warfare, according to the unique characteristics of the disassembled opcode and ASM code combined code. That is, based on these unique characteristics, the embodiment may identify a scheme of operating the attack code or malware, a person developing the attack code or malware, and the development purpose.

In the future, based on characteristic information about continuous attacks by the attacker, it will be possible to supplement a vulnerable system and to enable an active and preemptive response to cybersecurity threats.

Based on this concept, the embodiment provides a completely different result from that in the method and performance of simply identifying an attack technique according to an attack result based on the opcode.

The embodiment may generate a data set of disassembled code based on the characteristics of the combination of the disassembled opcode and ASM code to accurately identify and classify the coding technique used to implement the tactic (TTP). When modeling is performed to identify unique characteristics from this generated data set, it is possible to identify not only the tactic (TTP) but also characteristic information of the developer, that is, the developer (or automated creation tool).

This figure illustrates an example of matching an opcode data set modeled in the manner described above to an attack identifier.

This example illustrates that a first opcode set (opcode set #1) matches an attack technique identifier T1011, and a second opcode set (opcode set #2) matches an attack technique identifier T2013. Further, a third opcode set (opcode set #3) may match an attack technique identifier T1488, and an Nth opcode set (opcode set #N) matches an arbitrary attack technique identifier T1XXX. While the standardized model, MITRE ATT&CK®, expresses the identifier of the attack technique in a matrix format for each element, the embodiment may additionally identify an attacker or an attack tool in addition to the identifier of the attack technique.

This figure is illustrated as an opcode data set for convenience. However, when an attack technique is identified by a data set of disassembled code including opcode and ASM code, it is possible to identify a more subdivided attack technique comparing to identifying an attack technique only by an opcode data set.

According to an embodiment, by analyzing a combination of disassembled code data sets, it is possible to identify not only the attack technique identifier but also the attacker or the attack group.

Accordingly, the embodiment may provide a more advanced technology in terms of acquiring intelligence information when compared to the conventional technology, and solve problems that have not been solved in the conventional security area.

Fast data processing and algorithms are required to ensure accurate intelligence information in the complex environment as described above. Hereinafter, additional embodiments related thereto and performance thereof will be disclosed.

FIG. 17 is a diagram for describing an example of identifying an attack technique and an attack group in units of functions.

In this example, it is assumed that an executable file (for example, EXE) has been disassembled and functions included in the executable file have been identified. The functions identified here are illustrated as Function 1, Function 2, Function 3, and Function 4.

Among the identified functions, Function 2 may include instructions for performing a function operation. Here, the instructions included in Function 2 are indicated as Instruction 1, Instruction 2, Instruction 3, Instruction 4, Instruction 5, Instruction 6, and Instruction 7.

However, one function in a program may be separated and executed according to several subfunctions during execution. In this example, it is assumed that Function 2 is separated into two subfunctions and executed. Then, the two subfunctions included in Function 2 may be separated into instructions.

Here, for convenience of description, the case where Instruction 1, Instruction 2, and Instruction 3 are included in one subfunction included in Function 2, and Instruction 4,

31

Instruction 5, Instruction 6, and Instruction 7 are included in the other subfunction is illustrated.

However, subfunctions may be included in one function, namely Function 2 in the program.

When characteristic information related to cyber threats is extracted in units of functions, one piece of characteristic information corresponding to Function 2 (cyber threat characteristic information A, simply indicated as characteristic information A) may be identified.

When the characteristic information related to the cyber threat in units of functions disclosed above is analyzed according to the above-described embodiment, an attack technique and an attack group may be identified.

FIG. 18 is a diagram for describing an example of identifying an attack technique and an attack group when a function is separated.

This embodiment is an embodiment showing the same result as that in the example disclosed above. However, here, the case in which one of the functions is clearly separated into subfunctions in the program is illustrated.

That is, the case in which Function 2 among the functions identified from the executable file is separated into Function 2-1 and Function 2-2 in the program is illustrated. Here, even when Function 2 is separated into Function 2-1 and Function 2-2, there is no change in program logic when compared to the case in which one function of Function 2 is executed.

When Function 2 is simply separated into two functions (Function 2-1 and Function 2-2) even though the program logic is the same, characteristic information (characteristic information B and characteristic information C) corresponding to each function is changed, and thus identification results of the attack technique and the attack group based on the characteristic information may be changed.

Therefore, even when the attack technique or the attack group is identified based on several functions executing the same logic in the program as that of execution of one function in this way, the attack technique and the attack group may be identified as the same attack technique and attack group.

The following embodiments disclose examples of identifying an attack technique and an attack group based on characteristic information considering a control flow and order according to instructions executed by several functions in a program.

When characteristic information is used based on a flow and order of instructions in functions of a program, characteristic information may be obtained by implementing substantially the same logic even when the functions in the program are different.

Even when a format of a program causing a cyber threat is slightly modified or even in the case of a variant, an attack technique and an attack group may be clearly identified based on this characteristic information.

Hereinafter, an example of profiling a control flow and identifying orders according to instructions in a function will be disclosed.

FIG. 19 discloses an example of obtaining characteristic information related to a cyber threat according to an embodiment.

Here, ControlBlocks including various functions may be obtained by disassembling an execution function represented by EXE.

After obtaining a control flow in relation to instructions in the obtained ControlBlocks, it is possible to check the order of the ControlBlocks according to the control flow and obtain an instruction sequence based thereon.

32

Further, cyber threat characteristic information may be identified according to the obtained instruction sequence.

Detailed embodiments of obtaining a ControlBlock or a code block corresponding thereto have been disclosed above.

In this example, ControlBlocks obtained by disassembling the execution function EXE are represented by ControlBlock1, ControlBlock2, ControlBlock3, . . . , ControlBlock6.

Here, each of the ControlBlocks, namely ControlBlock1, ControlBlock2, ControlBlock3, . . . , ControlBlock6, may correspond to each instruction set. As described above, even though instruction sets described above are different, execution logic in each instruction set may be the same.

Therefore, the control flow is analyzed for the ControlBlocks to identify whether the ControlBlocks perform the same logic.

For example, here, in order to easily describe the embodiment, a graph analyzing a control flow of code blocks according to program execution is created and described.

For example, in an instruction set included in ControlBlock1, instructions according to an execution order are denoted by C1, C2, C3, . . . , C6. For easier understanding, in the instruction set, the instructions according to the execution order are indicated as a control flow graph (CFG).

An instruction order may be obtained in the CFG of the instructions shown in this example. Here, the obtained order is shown using a depth first search (DFS) method. The DFS method is an iterative method in which an instruction is selected as an addition node for one search tree, an applicable instruction is applied to this node, and an instruction is added as one child node of a next level to the search tree.

Then, it is possible to obtain an instruction order applied according to the instruction control flow in the instruction set corresponding to the ControlBlock.

In this example, an order according to a control flow of instructions included in instruction set 1 corresponding to ControlBlock1 may be (C1, C2, C4, C5, C3, C6).

An order according to a control flow of instructions included in instruction set 2 corresponding to ControlBlock2 may be (C2, C4, C5).

An order according to a control flow of instructions included in instruction set 3 corresponding to ControlBlock3 may be (C3, C6).

An instruction sequence according to the obtained instruction order may be generated, and characteristic information on a cyber threat may be distinguished according to the instruction sequence.

Here, an example is disclosed in which six instruction sequences are obtained by classifying instruction set 1 corresponding to ControlBlock1 according to an order according to a control flow, and one piece of characteristic information is extracted for each of the six instruction sequences.

In this way, even when one function in the program is separated or changed to functions performed with substantially the same logic, cyber threat information according to the same logic may be distinguished.

Hereinafter, various examples of obtaining instruction sequences using various control flows in ControlBlocks including various functions are disclosed.

First, an example of obtaining various control flows within included ControlBlocks is disclosed.

ControlBlocks are obtained by disassembling an executable file.

It is possible to identify an instruction referring to a specific block in the ControlBlocks or a ControlBlock

outside the corresponding ControlBlocks among instructions inside the ControlBlocks. An instruction diverging in the code in this way is referred to herein as a branch instruction type.

Examples of the branch instruction type may include a call function, a jump function, etc. These functions may refer to a specific block in the ControlBlocks or a ControlBlock outside the corresponding ControlBlocks.

Accordingly, when a reference address according to such a branch instruction is identified, a control flow of instructions may be obtained.

FIG. 20 illustrates a process of obtaining a control flow using a branch instruction series according to an embodiment.

A disassembled ControlBlock cblk1 is extracted, and an instruction of a branch instruction type is identified inside the extracted ControlBlock cblk1.

A reference (outgoing reference, indicated as outgoing-ref) indicating an external location of the ControlBlock cblk1 among reference addresses indicating instructions of the branch instruction type diverging in code is checked.

A left side of this figure is an example for describing an example of specific outgoing reference analysis.

In this example, a reference (reference A) indicating an internal location of the ControlBlock cblk1, which is not an outgoing reference, may be ignored. That is, reference A indicates the inside of the ControlBlock cblk1, and thus may not be considered when generating a control flow.

Further, a control flow may be generated separately for the case where an outgoing reference of the ControlBlock cblk1 indicates a start address or a start instruction of another ControlBlock cblk2 (reference B) and the case where the outgoing reference of the ControlBlock cblk1 indicates an internal address or an internal instruction of another ControlBlock cblk3 (reference C).

In this example, since reference B indicates the start address or instruction of the target ControlBlock cblk2, the target ControlBlock cblk2 may be included in control flow generation without change.

Meanwhile, since reference C indicates instruction 2 (instr2) on the inside of the target ControlBlock, a new third ControlBlock cblk3-2 including instruction 2 (instr2) to a last instruction of the corresponding ControlBlock cblk3 may be included in control flow generation during ControlBlock generation.

A right side of this figure is an example of generating a control flow for a specific ControlBlock cblk1 according to the example described above.

As a result of analyzing the control flow of the ControlBlock cblk1 according to the outgoing reference analysis on the left, the control flow for the ControlBlock cblk1 may be generated.

The control flow generated according to this example may include the second ControlBlock cblk2 as a vertex within the control flow when the first ControlBlock cblk1 refers to a start address or instruction of the second ControlBlock cblk2.

Further, when the first ControlBlock cblk1 indicates an internal or intermediate location or instruction of the third ControlBlock cblk3, the generated control flow may separate the third ControlBlock cblk3 from the instruction at the indicated location, and include, as a vertex, a new ControlBlock cblk3-2 having the instruction at the indicated location as a start instruction.

According to an embodiment, when a branch instruction of a specific ControlBlock is an outgoing reference, a control

flow may be generated according to a location or instruction indicated by the outgoing reference.

A control flow generated for a specific ControlBlock includes the second ControlBlock as a vertex when an outgoing reference thereof indicates a start point of the second ControlBlock. Further, when the outgoing reference indicates an intermediate location of the third ControlBlock, the generated control flow includes, as a vertex, a new ControlBlock with an instruction of the indicated location as a start instruction.

In the example of this figure, reference A of the first ControlBlock cblk1 is a reference indicating the inside of the first ControlBlock cblk1, and thus is ignored, and reference B of the first ControlBlock cblk1 indicates a start address of the second ControlBlock cblk2, and thus the second ControlBlock cblk2 is included as a vertex. Reference C of the first ControlBlock cblk1 indicates the inside of the second ControlBlock cblk2, and thus a new ControlBlock may be generated from instruction 2 of the second ControlBlock cblk2 and included as a vertex.

The example of this figure is an example in which the generated control flow is displayed as a CFG, and lower vertices are located on the left side of the graph in ascending order based on a start address of a ControlBlock cblk.

Hereinafter, an example will be disclosed below in which cyber threat characteristic information of an executable file is obtained according to an instruction sequence generated by searching for a reference relationship between ControlBlocks in which the executable file is disassembled as described above.

Instruction sequences generated according to the reference relationship may represent characteristics of cyber threat information.

The control flow generation disclosed above may generate instruction sequences by merging instructions of a ControlBlock according to an order based on a specific principle when the DFS method is used.

Hereinafter, a method of combining instruction sequences capable of obtaining characteristics of cyber threat information will be illustrated.

As a first example of combining instruction sequences, when instruction sequences are generated according to a reference relationship between instructions in a ControlBlock, an instruction sequence may be generated by performing DFS on meaningful instructions of a control flow.

Here, the meaningful instructions of the control flow mean that NOP (non-operation) or RET (return)-type functions or branch-type functions such as JUMP functions or CALL functions among instructions called in a ControlBlock are removed.

When a CFG is generated, these types of functions merely generate edges of the graph, and are not included in an actual instruction sequence. Therefore, when instructions are sequentially combined using DFS in the CFG, these types of functions do not contribute to generating an instruction sequence.

In the first example of generating instruction sequences according to a reference relationship of instructions in a ControlBlock, meaningful instructions that may be included in an actual instruction sequence are combined, and branch or simply referenced instructions are not included.

In the CFG, instructions are combined using the DFS method, and thus an instruction sequence is generated without using a branch-type instruction or a simply referenced instruction.

As a second example of generating instruction sequences according to a reference relationship between instructions in

35

a ControlBlock, a stack frame may be adjusted when the ControlBlock is called by a CALL-type function among instructions in the ControlBlock.

The stack frame refers to a space created to distinguish functions in a stack area. For example, the stack frame may include parameters, return addresses, local variables, etc., and is created when a function is called and destroyed when the function is terminated.

In general, the stack frame includes a stack pointer *sp* indicating a stack start point and a base pointer *bp*, which is a pointer indicating specific data on a stack. When the stack frame is changed, the stack pointer *sp* and the base pointer *bp* may be changed.

Such instructions related to pointers on a stack frame serve as logic noise in a control flow, and thus are not used to combine instruction sequences, for example, using DFS. Similar to not using branch-type instructions to combine instruction sequences as illustrated above, instructions related to a stack frame are not used.

FIG. 21 is a diagram illustrating the case of generating an instruction sequence by combining instructions of a ControlBlock according to an instruction combining principle illustrated according to a second example.

When a ControlBlock is called by a CALL-type function, since instructions related to a stack frame are not related to logic by a control flow, an instruction sequence may be generated without using the instructions during combination.

This figure illustrates a ControlBlock of sample code indicated by *app1* and a ControlBlock of sample code indicated by *app2*. Sample code *app1* and sample code *app2* yield the same result. However, in this example, while sample code *app1* repeats the same code, sample code *app2* does not repeat the same code and causes a function *fool1* to call *fool2* so that the same execution is performed.

When the ControlBlock of sample code *app2* is taken as an example for description, a stack frame may be initialized before start of the ControlBlock of sample code *app2* (0x100003eb0 to 0x100003eb4).

Here, in the code, (*pushq %rbp*) indicates storing the base pointer, and (*movq %rsp, %rbp*) indicates storing the stack pointer in the base pointer.

Further, (*subq %16, %rsp*) in the code indicates moving a stack pointer location to a top of a stack, and the stack has a smaller address at the top than at a base.

The stack may be arranged before return of the ControlBlock in sample code *app2* (0x100003ef9 to 0x100003efd).

(*addq \$16, %rsp*) in the code here indicates moving the stack pointer to the base (bottom), resulting in an effect of removing all values of the stack.

Further, (*popq %rbp*) in the code indicates restoring a previous base pointer that has been saved.

Therefore, when *app1* is called thereafter, since instructions related to a previous stack frame are not related to a control flow, the instructions are combined by the call and are not considered during generation of an instruction sequence.

In this way, when a stack frame is adjusted by separation of a function related to the stack frame, that is, when instructions related to the stack frame are not related to logic by a control flow, an instruction sequence is generated without considering the instructions in generating the instruction sequence.

Another example of generating instruction sequences including characteristic information using instructions in a ControlBlock will be disclosed.

36

When instruction sequences including characteristic information are generated using instructions in a ControlBlock, the instruction sequences may be generated by reflecting an edge weight of a graph according to control flow analysis.

A graph reflecting the edge weight of the graph according to control flow analysis will be compared and illustrated in a figure below.

FIG. 22 is a diagram for describing another example of generating instruction sequences including characteristic information using instructions in a ControlBlock.

Here, sample codes *app1* and *app3* yielding the same result are illustrated.

In this example, a ControlBlock indicated by sample code *app1* on the left side has a structure in which code having the same logic and different variables is repeated twice.

Sample code *app3* on the right side is an example in which the same code is changed to a function without being repeated, and then is called twice (NET supplements-6-110).

Results of the two sample codes in this figure are the same. However, when an instruction sequence is generated based on sample code *app3*, an instruction of ControlBlock 0x100003ef0 called twice may be added twice to a graph analyzing a control flow to generate an instruction sequence.

In this way, when instruction sequences are generated using the instructions in the ControlBlock, a repeatedly called instruction may generate an instruction sequence by reflecting an edge weight in the CFG. Therefore, an instruction that is called a plurality of times in a generated instruction sequence may be reflected as a weight.

A graph reflecting the edge weight of the graph according to control flow analysis will be compared and illustrated in a figure below.

FIG. 23 is a diagram for describing still another example of generating instruction sequences including characteristic information using instructions in a ControlBlock.

A fourth embodiment of generating instruction sequences including characteristic information using instructions in a ControlBlock is as follows.

Sample codes *app1*, *app2*, and *app3* illustrated in this figure have been described above.

Sample code *app1* is code in which the same code is repeated, sample code *app2* is code in which the same code is not repeated and a function *fool1* calls *fool2* so that the same execution is performed, and sample code *app3* is code in which the function *fool2* is called twice.

Even when an instruction sequence is generated based on codes performing the same logic, since an offset is different for each file, the instruction sequence may vary according to an operand of a function in the file.

As illustrated in this figure, operands, which are operators of functions, are all different for the same function.

An instruction sequence capable of representing characteristics of cyber threat information may be affected due to operands that are values in boxes of this figure.

Accordingly, when instruction sequences including characteristic information are generated using instructions in the ControlBlock, function operands may be removed, and the instruction sequences may be generated using only opcode.

FIG. 24 is a diagram for describing yet another example of generating instruction sequences including characteristic information using instructions in a ControlBlock.

As a fifth embodiment of generating instruction sequences including characteristic information using instructions in a ControlBlock, when the instruction sequences are generated based on instructions in the ControlBlock, instructions that simply transmit parameters may act as noise in a logic flow.

In the ControlBlock of the sample code illustrated in this figure, a function 0x100003ef0 is called twice, and each performs a process of transferring a parameter.

An instruction simply related to parameter transfer in this way only generates noise when a control flow is generated, does not significantly contribute to actual characteristic information or an instruction sequence corresponding thereto, and thus is excluded.

Examples of generating an instruction sequence corresponding to characteristic information of cyber threat information based on instructions included in a ControlBlock when an executable file is disassembled to generate assembly code have been described above.

The examples illustrated above may be repeatedly applied, and thus an instruction sequence may be generated according to at least one of the five examples described above.

FIG. 25 discloses an example of generating an instruction sequence according to the above-described examples.

An instruction sequence including characteristic information such as cyber threat information may be generated by considering and combining characteristics, order, and reference of instructions in a ControlBlock.

In the case of generating an instruction sequence in this way, as an example, it is possible to remove a branch-type function diverging in code such as a JUMP function or a CALL function according to a reference relationship of instructions in a ControlBlock, and to generate an instruction sequence according to a control flow.

As another example of generating an instruction sequence, when a stack frame is adjusted by separating a function related to the stack frame, an instruction unrelated to logic by a control flow may be removed, and an instruction sequence may be generated.

Still another example of generating an instruction sequence is generating an instruction sequence by reflecting an edge weight in a CFG of an instruction. An instruction sequence may be generated by reflecting a weight on a graph of control flow analysis for an instruction called a plurality of times in the instruction sequence generated using the same.

As yet another example of generating an instruction sequence, since an offset may vary by an operand in disassembled code, an operand of a function may be removed, and an instruction sequence may be generated using only opcode.

As yet another example of generating an instruction sequence, an instruction related only to parameter transfer does not significantly contribute to an instruction sequence, and thus an instruction sequence may be generated by excluding the instruction when the instruction sequence is generated.

When at least one of these examples is applied, an instruction sequence capable of including characteristic information of cyber threat information may be generated based on a control flow in a disassembled ControlBlock.

An instruction sequence may be generated based on main code (0000000100003f60≤main>) included in sample codes app1, app2, and app3 illustrated above.

Code of the generated instruction sequence may be normalized and vectorized as described above. Further, vectorized content may be converted into hash code. The converted hash code may include unique characteristic information of cyber threat information. The cyber threat characteristic information included in the hash code may

identify an attack technique and an attack group using the converted hash code using the AI technique described above.

In this figure, a row corresponding to “CFG” represents graphs according to control flow analysis for sample codes app1, app2, and app3, respectively.

In this example, a graph according to control flow analysis of sample code app1 is expressed as 0:100003f60→1:100003ed0, and a graph according to control flow analysis of sample code app2 is expressed as 0:100003f60→1:100003f00→2:100003ed0.

In addition, a graph according to control flow analysis of sample code app3 is expressed as 0:100003f60→1:100003f40→2:100003ef0. Here, edge weight 2 is reflected in a control flow of 1:100003f40→2:100003ef0.

A graph according to each control flow analysis is generated by applying at least one of the five examples illustrated above.

A row corresponding to “instruction sequence” represents instruction sequences for sample codes app1, app2, and app3, respectively. Therefore, even when sample codes app1, app2, and app3 are not exactly the same, since the codes yield the same result, it can be confirmed that all the instruction sequences according to the methods illustrated above appear the same.

In a row corresponding to “fuzzy hash,” which is a last row, the instruction sequences for sample codes app1, app2, and app3 are converted into hash codes. Hash information of a ControlBlock of each sample code may be characteristic information.

As can be seen from this example, sample codes app1, app2, and app3 have the same meaning in terms of cyber threat information even though the codes are slightly different from each other. That is, it can be seen that the hash codes of sample codes app1, app2, and app3 are the same, and the corresponding codes have the same characteristic information.

FIG. 26 is a diagram illustrating another embodiment of the disclosed cyber threat information processing apparatus.

Another embodiment of the cyber threat information processing apparatus may include a server 2100 including a processor, a database 2200, and an intelligence platform 10000.

The database 2200 may store previously classified malware or pattern codes of malware.

The processor of the server 2100 may execute a first execution module 18501 for obtaining disassembled code by disassembling an executable file received from the API 1100.

In addition, the processor of the server 2100 may execute a second execution module 18503 for generating an instruction sequence based on a control flow according to a relationship between instructions in the disassembled code.

Examples of a process of executing the second execution module 18103 are illustrated in FIGS. 35 to 41.

In addition, the processor of the server 2100 may execute a third execution module 18505 for converting the generated instruction sequence into a feature data set related to cyber threat information. The feature data set may be feature vector data and a hash function.

In addition, the processor of the server 2100 may execute a fourth execution module 18507 for implementing an AI engine 1230, determining the presence or absence of similarity with the stored malware based on the converted data set having a specific format, and classifying the converted data set having the specific format as at least one standardized attack identifier according to the determination.

An example of a process of executing the fourth execution module **18507** has been described with reference to FIGS. **19, 20, 21, 25, 26,** and **27.**

FIG. **27** is a diagram illustrating another embodiment of the disclosed cyber threat information processing method.

Disassembled code is obtained by disassembling an executable file (**S4100**).

An instruction sequence is generated based on a control flow according to a relationship between instructions in the disassembled code (**S4200**).

Examples of obtaining an instruction sequence based on a control flow according to a relationship between instructions in code are illustrated in detail in FIGS. **35** to **41.**

The generated instruction sequence is converted into a feature data set related to cyber threat information (**S4300**).

The generated instruction sequences may be converted into feature vector data and then converted into hash function values. An example of converting a CodeBlock including an instruction sequence into vector data and a hash function value has been described in detail above. For example, the embodiments of FIGS. **21** to **24** may be used for data conversion. The example of converting a CodeBlock including an instruction sequence into vector data and a hash function value is understood with reference to this embodiment.

Cyber threat information is acquired by learning a feature data set related to the cyber threat information using an AI model (**S4400**). An example of classifying an attack technique or an attack group by learning data including characteristic information related to a cyber threat based on an AI model has been disclosed in detail above. For example, the embodiments of FIGS. **25** to **28** may be applied to a learning model and a classification model.

Accordingly, a pattern related to a specific attack identifier may be identified from a CodeBlock generated by extracting only instruction sequences related only to a cyber threat. In addition, an accurate attack identifier may be determined based on a probability based on data according to the selected attack identifier. As illustrated above, an attack group may be identified.

The acquired cyber threat information may be provided to a user again by the server. The user may obtain specific cyber threat information related to an executable file, for example, detailed information on an attack technique, an attack group, etc., by inquiring about information on the executable file or inputting the executable file on the API.

In the above, embodiments of processing cyber threat information by analyzing executable files for the system in the assembly language domain have been disclosed.

Hereinafter, an embodiment of identifying and processing cyber threat information from a non-executable file is disclosed. Recently, especially due to the COVID-19 pandemic, all activities such as economy, society, and education have been changed to non-face-to-face, and tens of thousands of online platforms including online commercial activities, telecommuting, and remote educations are expanding. Therefore, the number of non-executable files shared online has increased, and attackers are increasingly using this advantage to carry out phishing attacks or advanced persistent threat (APT) attacks through various non-executable files.

However, general users are still not aware of non-executable malicious codes, and existing anti-virus products are developed for executable files, so they cannot detect non-executable malicious files well. In addition, even if a non-executable malicious file is detected, the reason for detection is often insufficient. Therefore, it is necessary to detect

non-executable malicious files and to provide the reasonings for the detection. Considering this point, an embodiment of identifying and obtaining cyber threat information from a non-executable file will be described in detail below.

For reference, the non-executable file here means a non-executable file whose external format is a file that requires a separate execution program to execute the file. In order to accurately describe the non-executable file, it will be described with reference to drawings.

FIG. **28** is a diagram conceptually illustrating a structure of a non-executable file and a reader program for the non-executable file.

Non-executable files whose file extensions may be represented by document-type files such as PDF or DOC may embed media files such as text, scripts, and images, and another executable file or non-executable file inside the file as illustrated in the figure.

As in the example of this figure, a script, text or media may be embedded in a non-executable file. An executable file or another non-executable file may be embedded in a non-executable file.

A non-executable file may be loaded and content thereof may be checked while an executable file (non-executable file reader program) capable of reading the corresponding file is executed. A malicious non-executable file may induce a reader program to perform the following task while being loaded by the reader program (while the reader program is executed).

When a malicious non-executable file is executed, for example, a script containing a malicious action may be executed. Alternatively, due to execution of the script, a malware distribution server may be connected to download and then execute malware, or an executable file in which a malicious action is contained and embedded may be extracted and then executed.

In addition, when a malicious non-executable file is executed, a non-executable file in which a malicious action is contained or embedded may be extracted and then opened, or a media file containing a malicious action may be extracted and then opened.

Hereinafter, embodiments capable of detecting non-executable malicious files and identifying attack techniques and attack groups accordingly are disclosed. The disclosed embodiments may classify non-executable files as normal or malicious, identify attack groups of the non-executable files, or identify attack actions of the non-executable files by utilizing an AI model.

FIG. **29** discloses a block diagram of an embodiment capable of obtaining cyber threat information of a non-executable file.

This embodiment includes a file analysis unit **4300**, a feature processing unit (feature fusion) **4400**, a malignancy detector (malicious document detector) **4500**, an attack technique classifier **4610**, and an attack group classifier **4620**.

The file analysis unit **4300** may receive a non-executable file (unknown document) and analyze various cyber threat information of the non-executable file.

The file analysis unit **4300** may include a first analysis unit **4310**, a second analysis unit **4320**, and a third analysis unit **4330**, and analyze feature information of a non-executable file input from each analysis unit.

The feature processing unit **4400** extracts a feature vector from feature information analyzed by the file analysis unit **4300**, and the extracted vector is converted into an appropriate form so that the malignancy detector **4500** may determine whether the vector is malicious.

41

The malignancy detector **4500** may detect whether a malicious action is included in data obtained by converting the feature vector based on an AI technique. When the malignancy detector **4500** determines that cyber threat information is not included in the input data, the data is determined to be a normal file (normal document).

The attack technique classifier **4610** and the attack group classifier **4620** may classify an attack technique (for example, T1204.001) and an attack group (for example, G001), respectively, according to a cyber threat information system based on an AI technique for data detected as malicious by the malignancy detector **4500**.

Here, as an example, according to a cyber threat information system, an attack action included in a non-executable file corresponds to an attack technique T1204.001, and a group generating the attack action is an attack group G001.

The illustrated blocks may be implemented as hardware or may be implemented as software and each executed by a processor of a server, respectively. Detailed examples of each part of the illustrated block diagram are disclosed below.

FIG. 30 is a diagram disclosing an example of performing a first type of analysis of a file by being included in the file analysis unit in an exemplary diagram capable of obtaining cyber threat information of the file.

The first analysis unit **4310**, which is described here as performing a type of static analysis for convenience, analyzes an input file.

The first analysis unit **4310** performs static analysis such as extracting and analyzing a malicious payload, a script, etc. included in a document of a non-executable file, and identifying a hidden attachment or malicious data disguised as another file.

The first analysis unit **4310** performs a static feature extraction step, a static feature processing step, and a static feature conversion step. When the first analysis unit **4310** is implemented as hardware, the first analysis unit **4310** may include a static feature extraction unit **4312**, a static feature processing unit **4315**, and a static feature conversion unit **4317**.

The first analysis unit **4310** may separate a non-executable file, for example, a file inside a document, based on static analysis, and analyze the separated file. The first analysis unit **4310** may extract a hidden malicious payload in a non-executable file, a script capable of executing the malicious payload, etc. based on static analysis, and extract information about a format of a document.

For example, the static feature extraction unit **4312** may extract URI information (URIs), scripts, embedded files, action-related information (actions), textual contents, document metadata, etc. in a non-executable file.

The static feature extraction unit **4312** may extract, for example, image files (images) or various other formats of attachments for embedded files.

The static feature processing unit **4315** may process static feature information (URIs, scripts, embedded files, actions, etc.) extracted by the static feature extraction unit **4312** to perform additional analysis and processing according to the static feature information.

The static feature processing unit **4315** may subdivide and process the extracted information so that intention information of an attacker may be reflected in feature information capable of distinguishing identification of an attack technique and an attack group.

For example, the static feature processing unit **4315** may obtain URI meta information by parsing a URI using a URI parser, and confirm attacker's intention of inducing down-

42

load of a malicious file for secondary infection or inducing access to an external phishing website from a document.

The static feature processing unit **4315** may obtain script metadata through analysis of an extracted script, and obtain information about a language script preferred by an attacker for attacking vulnerabilities or performing a malicious action based thereon.

The static feature processing unit **4315** may check a hidden payload identifier from an embedded file and obtain a payload type of the embedded file. Based thereon, it is possible to obtain information about a technique employed by an attacker to hide a malicious payload.

In addition, the static feature processing unit **4315** may check a true file type by checking a type of attachment from an embedded file, and obtain information about what data is included and what is disguised as the attachment by an attacker in a document.

The static feature processing unit **4315** may classify various actions included in a non-executable file and obtain action metadata. Based thereon, it is possible to obtain information on which action or technique is used to induce a malicious action.

In this way, the static feature processing unit **4315** may obtain attacker intention information from various extracted static analysis information. In addition, the static feature processing unit **4315** may obtain information on which file is included in a non-executable file in an abnormal form and whether the file is in the form of a script.

The static feature conversion unit **4317** converts static feature information extracted by the static feature processing unit **4315**. For example, the static feature conversion unit **4317** performs a normalization or vectorization process as described above so that cyber threat information may be processed based on static feature information extracted by the feature processing unit **4400**.

FIG. 31 is a diagram disclosing an example of performing a second type of analysis of a file by being included in the file analysis unit in an exemplary diagram capable of obtaining cyber threat information of the file.

The second analysis unit **4320** may extract cyber threat information by analyzing a non-executable file based on dynamic analysis. The second analysis unit **4320** may execute the non-executable file in a corresponding program, such as a reader program, and extract action information that actually occurs during actual execution.

Hereinafter, for convenience, the second analysis unit **4320** is expressed as performing a dynamic analysis step.

The second analysis unit **4320** constructs a safely separated virtual environment for dynamic analysis of a non-executable file and executes a corresponding program suitable for the non-executable file in the virtual environment.

The second analysis unit **4320** may analyze which parameter is used to perform an action when a system call is called in a process that occurs when a non-executable file is executed in a corresponding program.

The second analysis unit **4320** performs an execution step, a dynamic feature extraction step, and a feature conversion step. When the second analysis unit **4320** is implemented as hardware, the second analysis unit **4320** may include an execution unit **4322**, a dynamic feature extraction unit **4325**, and a dynamic feature conversion unit **4327**.

A sandbox reader (sandbox document reader) of the execution unit **4322** executes an entered non-executable file as a corresponding program in a virtual environment.

A system call analysis unit (system call hooking) of the execution unit **4322** may monitor whether a specific system call is called in a process derived from the executed corre-

43

sponding program, and analyze which parameter is used for an execution action in this way.

The system call analysis unit (system call hooking) of the execution unit **4322** may obtain a system call to be monitored based on dynamic analysis and correspondingly extractable parameter data.

For example, when Send API is called while a program is executed, the system call analysis unit (system call hooking) of the execution unit **4322** may analyze packet data corresponding thereto, and obtain system call parameter information about transmitted packet data and the amount of transmission through a network.

The system call analysis unit (system call hooking) of the execution unit **4322** may trace back to a stack of the system call executed by the reader program of the non-executable file and analyze trace information. This trace information includes an execution order of functions according to the system call and used variable information of the functions.

A detailed embodiment of the system call analysis unit (system call hooking) will be described in detail again below.

The dynamic feature extraction unit **4325** may extract and collect result of execution by the execution unit **4322** in a virtual environment. For example, the dynamic feature extraction unit **4325** may collect various command information generated while a script is executed, and a communication type, an IP address, port number information, etc. generated through network connection according to execution of a reader program.

The dynamic feature extraction unit **4325** may collect various packet data downloaded while a reader program is executed, or collect information about a path of a target file or packet content from a payload of a packet thereof.

As another example, the dynamic feature extraction unit **4325** may obtain information about a program executed while a file is executed or opened and the target file.

The dynamic feature conversion unit **4327** converts information collected or extracted by the dynamic feature extraction unit **4325**. For example, the dynamic feature conversion unit **4327** performs a normalization or vectorization process so that cyber threat information may be processed based on feature information extracted by the dynamic feature extraction unit **4325**.

FIG. **32** is a diagram illustrating an object extracted by dynamic execution of a non-executable file and extracted information by a second type of analysis for a file according to an embodiment.

When a non-executable file is executed as a reader program, various actions may be performed on the program. This figure illustrates categories such as script execution/opening, server connection, download, file extraction, and file execution/opening as categories of the performed actions. However, there may be numerous other actions.

When a script is executed by executing a reader program of a non-executable file, functions such as WinExec and System may be executed through a system call API. Command line commands may be executed by executing these functions. Here, powershell.exe is executed as an example.

When another server is connected to by executing a reader program of a non-executable file, Socket may be executed through a system call API. Here, AF_INET is illustrated as a parameter of a communication type that occurs accordingly. In addition, when Connect is executed through a system call API, a port number may be obtained as a parameter.

As in the other examples, when a non-executable file is executed as a reader program, functions such as Send,

44

SendTo, Recv, RecvFrom, Fopen, Fwrite, CreateFile, WriteFile, CreateProcess, and ShellExecute may be executed through a system call API depending on the categories of actions performed. Examples of parameters that may be extracted according to the functions of each system call API are illustrated in a right section.

FIG. **33** is a diagram disclosing an example of performing a third type of analysis of a file by being included in the file analysis unit in an exemplary diagram capable of obtaining cyber threat information of the file.

The third analysis unit **4330** obtains characteristics of cyber threat information based on information stored in a memory in an execution preparation step for a non-executable file. Since data in the memory immediately before dynamic execution in a virtual environment is analyzed, hereinafter, for convenience, the third analysis unit **4330** is described as performing a mild-dynamic analysis step.

When the third analysis unit **4330** performs the mild-dynamic analysis step, the third analysis unit **4330** may extract and analyze opcode and operator information included in the memory or malicious payload data which has been de-obfuscated in a malicious action preparation step according to file analysis.

The third analysis unit **4330** does not extract parameters generated while executing the dynamic analysis described above. The third analysis unit **4330** performs so-called API hooking on main functions of the system inevitably involved with a malicious action immediately before dynamic execution in a virtual environment to put the process in a suspended state when the corresponding function is called, and extracts (dumps) information loaded in the memory at this time.

To this end, the third analysis unit **4330** performs an execution preparation step, a memory extraction step, a data extraction step, and a feature conversion step. When the third analysis unit **4330** is hardware-separated, the third analysis unit **4330** may include an execution preparation unit **4331**, a memory extraction unit **4333**, a data extraction unit **4335**, and a feature conversion unit **4337**.

The third analysis unit **4330** may obtain and analyze data of a malicious payload from the memory based on information of a step of preparing a malicious action.

In the execution preparation step, the execution preparation unit **4331** prepares a non-executable file (target file) and a reader program (application) in a user area. The execution preparation unit **4331** may prepare various file systems, network systems, or memories in preparation for an event to be executed when the application, which is the corresponding reader program, is executed in a kernel area.

In addition, the execution preparation unit **4331** prepares for execution with API hooking list information so that the corresponding application performs API hooking on the main functions of the system immediately before execution. Detailed API hooking list information is illustrated in the following figure.

When a function is called on an API hooking list, the memory extraction unit **4333** puts the process in a suspended state and extracts information by dumping data stored in the memory at that time. The memory extraction unit **4333** may obtain analysis information that may be cyber threat information from data immediately before the process execution of the function.

The data extraction unit **4335** may obtain opcode, operator (operand) data, and de-obfuscated data from data obtained by memory dumping by the memory extraction unit **4333**.

For example, the data extraction unit **4335** may disassemble data obtained by memory dumping by the memory extraction unit **4333**, and classify opcode, operator (operand) data, de-obfuscated data, etc. from the disassembled data.

Here, the data extraction unit **4335** may obtain analysis target data as conversion data for opcode, operator (operand) data, de-obfuscated data, etc. corresponding to functions on the API hooking list rather than the entire executable file.

The data extraction unit **4335** performs a normalization or vectorization process so that cyber threat information may be processed based on opcode, operator (operand) data, de-obfuscated data, etc.

FIG. **34** is a diagram illustrating API hooking list information when the third analysis unit performs mild-dynamic analysis according to an embodiment.

In the illustrated API hooking list information, categories of APIs are illustrated in a left column, and APIs included in each API category and thus may be included in an API hooking list are illustrated in a right column.

Window OS Native API, HTML DOM Parser API, and VBS Script Engine API are illustrated as categories of APIs.

APIs that may be used for API hooking are illustrated for the Window OS Native API category, seven APIs are illustrated for the HTML DOM Parser API category, and 11 APIs are illustrated for the VBS Script Engine API category.

FIG. **35** is a diagram for describing the feature processing unit in an embodiment capable of obtaining cyber threat information of a non-executable file.

As described above, the first analysis unit **4310** and the second analysis unit **4320** may acquire and analyze static feature information and dynamic feature information, respectively, for each non-executable file.

Meanwhile, the third analysis unit **4330** may perform API hooking of an application executed in relation to a non-executable file in a virtual environment, thereby acquiring and analyzing cyber threat information by the non-executable file from memory information at that time. In the disclosed embodiment, analysis by the third analysis unit **4330** is referred to as mild-dynamic analysis.

The feature processing unit **4400** may selectively collect and process static feature information, dynamic feature information, and mild-dynamic feature information extracted by the first analysis unit **4310**, the second analysis unit **4320**, and the third analysis unit **4330**, respectively.

The malignancy detector **4500** may determine whether a non-executable file includes cyber threat information based on information processed by the feature processing unit **4400**.

Further, the attack technique classifier **4610** may specifically classify an attack action or an attack technique of the cyber threat information detected by the malignancy detector **4500** according to a specific system.

The attack group classifier **4620** may classify a person who plans or executes an attack action of the cyber threat information detected by the malignancy detector **4500**.

The feature processing unit **4400** may generate feature information by using one of static feature information, dynamic feature information, and mild-dynamic feature information, or combining at least two thereof.

The feature processing unit **4400** generates feature information by selectively combining extracted information according to characteristics of each of the extracted static feature information, dynamic feature information, and mild-dynamic feature information or based on a classification model of an attack technique or an attack group.

For example, in the extracted feature information, feature information different from feature information for classifying an attack technique and feature information for classifying an attack group may be combined, or feature information may be combined by differently evaluating importance of each piece of feature information, which will be described in detail in the following drawings.

Therefore, the feature processing unit **4400** may use at least one of the extracted static feature information, dynamic feature information, and mild-dynamic feature information selectively or in combination.

For example, when only the mild-dynamic feature information has assembly code level information unlike the static feature information and the dynamic feature information, the mild-dynamic feature information may not be used in an attack group classification model.

In this case, the malignancy detector **4500** or the attack technique classifier **4610** detects malignancy or classifies an attack technique using all of the static feature information, the dynamic feature information, and the mild-dynamic feature information, and the attack group classifier **4620** may separately classify an attack group by selectively using the static feature information and the dynamic feature information.

Since all the feature information extracted in this way has different importance and characteristics, each of malignancy detection, attack technique classification, and attack group classification may be performed based on the feature information selected or combined accordingly.

Meanwhile, the malignancy detector **4500** determines whether a non-executable file is malicious based on a machine learning model. For example, when the feature processing unit **4400** processes at least one of the static feature information, the dynamic feature information, and the mild-dynamic feature information, the malignancy detector **4500** may detect whether there is malignancy based on feature vector data corresponding to the feature information.

An example of determining whether there is malignancy based on feature vector data has been described in detail above.

FIG. **36** is an exemplary diagram comparing importance of feature information extracted from a non-executable file according to a disclosed embodiment.

In the example of this graph, a horizontal axis represents an index according to feature information, and a vertical axis represents an importance score. An index of feature information according to an attack group model and an index of feature information according to a TID model have peak values at different feature indexes.

This means that characteristics of feature information representing an attack technique and feature information representing an attack group are different from each other as described above.

Therefore, the feature processing unit **4400** may differently select or selectively combine the static feature information, the dynamic feature information, and the mild-dynamic feature information at the time of each of malignancy detection, attack technique classification, and attack group classification according to the characteristics of the feature information, so that a detection model or a classification model may be performed.

FIG. **37** is an exemplary diagram for describing a classification model of the attack technique classifier according to a disclosed embodiment.

47

This figure illustrates an example in which the attack technique classifier according to an embodiment classifies and outputs an attack technique.

As disclosed, when a non-executable file includes cyber threat information, and thus is determined to be malicious, the attack technique classifier classifies an attack technique of the non-executable file by performing a machine learning model based on feature vector data for a cyber threat output by the feature processing unit.

When the attack technique classifier classifies an attack technique using the machine learning model, a class label of training data may be used as a correct answer and learning may be performed based thereon. Such training data includes an independent variable, which is the feature vector data, and a dependent variable, which is the class label.

In general, a dependent variable may have an integer value (single label) indicating one index number by a class label.

However, since one file may include several attack techniques, the attack technique classifier may use a multi-label technique that defines a dependent variable as T vectors rather than one integer value. That is, the attack technique classifier may receive input of feature vector data and classify the feature vector data as a binary vector corresponding to an attack technique as multi-labeling classification.

The attack technique classifier may learn a binary classification model for each class label as a multi-output classification model and generate T classification models, the number of which is the number of classifiable attack techniques.

When the above description is expressed as a simple equation, a prediction value y , which is a T-dimensional vector, and a prediction value o_i for an input vector x of an i th attack technique classification model f_i may be defined as follows.

$$\hat{y} = (o_0, o_1, o_2, \dots, o_T)$$

$$o_i = \begin{cases} 1, & \text{if } f_i(x) > 0.5 \\ 0, & \text{otherwise} \end{cases}$$

The class label, which is a dependent variable, is an attack technique identified by T1059.005 when classified as a single label, and may be indicated as a multi-dimensional vector such as [1, 1, 0] for attack technique identifiers T1059.005, T1564.007, and T1204.002 when classified as the above-described multi-labeling.

In addition, the attack technique classifier may output probabilities for three attack techniques as displayed at the bottom of the figure.

FIG. 38 is a diagram illustrating an attack technique identified by selectively combining various analytical techniques for a non-executable file according to a disclosed embodiment.

This figure illustrates an identifier (technique ID) of each attack technique, a name of each attack technique, and a description of each attack technique.

For example, a name of an attack technique identifier T1059.001 is Command and Scripting Interpreter: PowerShell, and this attack technique refers to an attack technique of a non-executable file that performs a malicious action using a PowerShell script.

A name of an attack technique identifier T1059.005 illustrated above is Command and Scripting Interpreter: Visual Basic, and this attack technique refers to an attack

48

technique of a non-executable file that performs a malicious action using the Visual Basic programming language.

FIG. 39 is an exemplary diagram for describing a classification model of the attack group classifier according to a disclosed embodiment.

The attack group classifier may classify an attack group based on a classification model, unlike the embodiments illustrated in FIGS. 27 and 28.

The attack group classifier may classify an attack group intending an attack action based on feature vector data output by the feature processing unit.

As an example of such clustering, the attack group classifier may perform clustering analysis based on feature vector data, and group data including similar characteristics into one group.

The attack group classifier may assign clustering identification information to groups clustered according to a structure and content of a document extracted from a non-executable file, an attack action attachment, a type of malicious data, etc.

Further, the attack group classifier may be trained using training data using a decision tree model and classify clustered groups according to the assigned clustering identification information (or grouping identification information).

An example of this figure illustrates a decision tree performing classification to indicate characteristics dividing groups according to clustering identification information (or grouping identification information).

An uppermost box represents a root node. The root node having a degree of clustering identification is sequentially split at a decision node into sub-nodes according to various characteristics included in a non-executable or executable file, so that a tree structure of a trained decision tree model may be obtained.

Here, the decision node and the sub-nodes are each shown in a box form.

When the attack group classifier classifies an attack group, group profiling information according to clustering and group may be obtained. For example, the attack group classifier may provide language of text in a document, a type of content in a document, and group profiling analysis information including various requirements such as whether a specific script is included in a document, or whether an automatically performed action is included when a document is executed.

The example of this figure is an example in which the attack group classifier classifies groups based on a tree structure, and illustrates a classification model in which last leaf nodes may distinguish groups from each other through a sixth branch.

The last leaf nodes of this tree node may be group profiling information for classifying groups. For example, the last leaf nodes may be profiling information for classifying groups, such as whether text of a document is in English, whether metadata is included and a length thereof, or whether content is included.

For example, the group profiling information may include information such as (1) text in a document is in English, (2) there is no media content in a document, (3) JavaScript is included in a document, and (4) there is an action function automatically performed when a document is executed.

Hereinafter, a detailed embodiment of the system call analysis unit (system call hooking) of the dynamic analysis disclosed above will be disclosed. As described above, there

may be cases in which it is determined whether a non-executable file is malicious based on the static analysis characteristics.

However, in many cases, it is difficult to provide a detailed description of whether a file is a non-executable file containing a malicious action or how a malicious action occurs with only static analysis characteristics. Therefore, when a reader program is executed to load a non-executable file, a process in which a malicious action occurs may be accurately identified, and a description thereof may be provided.

When a reader program related to a non-executable file is executed, the reader program performs an operation according to a combination of system calls provided by an operating system.

When the reader program is executed in the Windows operating system, the following system calls, etc. may be used.

FIG. 40 is a diagram illustrating execution of the reader program of the non-executable file described above and system calls.

A non-executable file may include a script, a media file, an executable file, other non-executable files, text, etc. This non-executable file may be executed by a corresponding reader program. When the reader program is executed in the Windows operating system, as described above, various system calls illustrated in this figure may be used depending on the file included in the non-executable file.

For example, when a script is executed in a non-executable file, system calls such as WinExec, CreateProcess, and ShellExecute are used, and when a server is connected to, system calls such as Socket and connect are used. When a download action is performed by executing a non-executable file, system calls such as send, sendto, recv, and recvfrom may be used. System calls such as fopen, fwrite, CreateFile, and WriteFile may be used when a file is extracted by execution of a non-executable file, system calls such as WinExec, CreateProcess, and system may be used when a file is executed, and system calls such as ShellExecute and system may be used when a file open operation is performed.

However, these system calls called by the reader program may be hooked (indicated by point A on the figure) when the system calls are called.

When hooking a system call at point A, data may be obtained by dumping parameter values or memory values transmitted to each system call.

Even though illustrated here only in the Windows operating system, the same embodiment may be applied to another operating system such as a mobile operating system or a Linux operating system.

FIG. 41 is a diagram for describing an example of hooking a system call on program code according to an embodiment.

A command "send" in this figure may include a function signature as illustrated.

Information transmitted according to the above command on this program code may be confirmed by dumping memory data of [buf] and [len].

In this way, by dumping a parameter value and a memory value thereof transmitted according to a system call performed by the reader program of the non-executable file, it is possible to determine what type of operation is caused by a malicious action and what type of information is used.

FIG. 42 discloses an example capable of tracing cyber threat information through dynamic analysis according to an embodiment.

In the embodiment, when a reader program on a specific operating system uses a system call, stack trace information of the reader program may be generated at a hooking time point.

The example of this figure illustrates a process of obtaining malicious action content according to the order of malicious actions and related variables through stack trace information generated after hooking the system call WinExec in the Windows operating system.

An example of a stack trace at the time when the system call WinExec, which is a last step, is hooked is as follows. According to the generated stack trace information, it can be seen that functions main→find_lastest_target→get_script have previously been called in this order with regard to the system call WinExec.

Local variables used by each function are shown on the right side of the boxes each including the function on this figure. For example, the function find_lastest_target uses count and targets as local variables.

Finally, the system call WinExec is called in the function get_script. Accordingly, when a malicious action occurs, a specific mechanism therefor may be described using the stack trace information.

That is, the following description may be provided according to the reverse order of the calling functions related to the system call on the stack trace information.

(1) Attempt to execute a suspicious command lpCmdLine through the system call WinExec.

(2) Execute functions in the order of main→find_lastest_target→get_script through the reader program.

(3) The local variable of each function is set as follows, and description of the local variable is as follows.

(a) main:

target_list-description of local variable

(b) find_lastest_target:

count-description of local variable

targets-description of local variable

(c) get_script:

script_src-description of local variable

cmd-description of local variable

According to the embodiment, when a non-executable file is executed in a reader program, and a malicious action occurs, after the reader program hooks a system call on the operating system, a specific mechanism for the malicious action may be provided using the order of functions related to the system call and variables of the functions.

The processor may execute a reader program that receives and executes a non-executable file. In this case, when the reader program executing the non-executable file executes a system call of the operating system, stack trace information of the reader program may be generated at the time of hooking the system call. In addition, the processor may obtain a calling function for calling the system call and a variable corresponding to the calling function from the generated stack trace information, and provide description information about the obtained calling function and the obtained variable corresponding to the calling function.

The description information may indicate that a command inducing cyber threat information is executed by the system call. The description information may include a calling order of the calling functions prior to the hooking point of the system call. In addition, the description information may include a description corresponding to a variable corresponding to the calling function.

FIG. 43 is a diagram illustrating another embodiment of the disclosed cyber threat information processing apparatus.

51

Another embodiment of the cyber threat information processing apparatus may include a server **2100** including a processor, a database **2200**, and an intelligence platform **10000**.

The database **2200** may store previously classified malware or pattern code of malware.

The processor of the server **2100** may receive a non-executable file received through the API **1100**.

The processor of the server **2100** may execute a first feature analysis module **18601** for analyzing and extracting static feature information related to a cyber threat of the non-executable file received through the API.

A detailed example of analysis of the static feature information performed by the first feature analysis module **18601** has been described in FIG. **30**, etc.

The processor of the server **2100** may execute a second feature analysis module **18603** for analyzing and extracting static feature information related to the cyber threat of the non-executable file received through the API.

Detailed examples of analysis of the dynamic feature information performed by the second feature analysis module **18603** are disclosed in detail in FIGS. **47**, **48**, and **56** to **58**.

When the second feature analysis module **18603** analyzes the dynamic feature information, by hooking a system call requested by the reader program of the non-executable file from the operating system, cyber threat information may be obtained by dumping memory data generated at that time.

The second feature analysis module **18603** may obtain mechanism information on the malicious action from the order of functions called immediately before hooking the system call and parameters corresponding to the functions.

The processor of the server **2100** may execute a third feature analysis module **18605** for analyzing and extracting mild-dynamic feature information related to the cyber threat of the non-executable file received through the API.

Detailed examples of analysis of the mild-dynamic feature information performed by the third feature analysis module **18605** are disclosed in detail in FIGS. **49** and **50**.

The third feature analysis module **18605** performs API hooking for main functions of an application system executing non-executable files, so that when a corresponding function is called, the process may be suspended, and information loaded in the memory at that time may be extracted (dumped).

The third feature analysis module **18605** may disassemble data of the memory to obtain opcode, operator (operand) data, and de-obfuscated data, and obtain feature information related to the cyber threat information based on the obtained data.

The processor of the server **2100** may execute a feature processing module **18607** for selectively combining feature information related to the cyber threat analyzed by the first feature analysis module **18601**, the second feature analysis module **18603**, and the third feature analysis module **18605** to generate feature data related to the cyber threat information.

A detailed embodiment of the feature processing module **18607** is disclosed in detail in FIG. **35**.

The processor of the server **2100** may execute a malignancy detection module **18608** for detecting whether a malicious action is included in the non-executable file received through the API based on the feature information of the cyber threat information processed by the feature processing module **18607**.

The processor of the server **2100** may execute a classification module **18609** for classifying an attack technique and

52

an attack group of a malicious action by performing the AI engine **1230** when the non-executable file includes the malicious action according to a result yielded by the malignancy detection module **18608**.

Detailed examples of generating information on the attack technique and the attack group of non-executable files classified by the classification module **18609** are disclosed in detail in FIGS. **52** to **55**.

FIG. **44** is a diagram illustrating another embodiment of the disclosed cyber threat information processing method.

Input of a non-executable file is received, and at least one feature related to a cyber threat of the input non-executable file is analyzed (**S4500**).

Examples of analyzing static feature information, dynamic feature information, and mild-dynamic feature information, respectively, related to the cyber threat of the non-executable files are disclosed.

A detailed example of analysis of the static feature information is illustrated in FIG. **30**, and detailed examples of analysis of the dynamic feature information are illustrated in FIGS. **47**, **48**, and **56** to **58**. In addition, detailed examples of analysis of the mild-dynamic feature information are illustrated in FIGS. **49** and **50**.

It is possible to detect whether a malicious action is included in the non-executable file based on feature information obtained by selectively combining analysis information according to at least one feature analysis (**S4600**).

When the non-executable file includes a malicious action, it is possible to generate classification information on an attack technique and classification information on an attack group (**S4700**). Detailed examples of generating information on the attack technique and the attack group of the non-executable file are disclosed in detail in FIGS. **52** to **55**.

Cyber threat information of the non-executable file analyzed as above is provided to a user (**S4800**).

Therefore, according to the disclosed embodiments, depending on the logic of a program including functions even in a program yielding the same result, or when functions are differently used such as being separated even if there is no change in the logic of the program, it is possible to accurately provide cyber threat information for an attack technique and an attack group, and respond to a variant of malware.

According to the embodiments, even when a malicious action is included in a non-executable file, it is possible to accurately detect the malicious action, and to provide cyber threat information about an attack technique and an attack group accordingly.

Hereinafter, disclosed are examples capable of monitoring a webpage, identifying a webpage including a malicious action or information, and identifying whether a component included in a webpage includes a malicious action or information according to embodiments of a cyber threat information processing apparatus and a method thereof.

FIG. **45** discloses an example of receiving input of or collecting webpage information and identifying malicious information based thereon in an embodiment.

The cyber threat information processing apparatus or a method thereof according to an embodiment receives input of or collects world wide web page (hereinafter simply described as a webpage). An embodiment may search a collected webpage, analyze whether the webpage generates a specific malicious action, and provide an analysis result as cyber threat information for a user.

An embodiment of the cyber threat information processing apparatus disclosed in this figure includes a data collection unit **5100** and an analysis detection unit **5200**. When

53

described as an embodiment of the cyber threat information processing method, the embodiment includes a data collection step and an analysis and detection step.

The data collection unit **5100** may include a web collection unit (Web Crawler) **5110** and a data bundle unit (Data Bundle) **5120**.

The web collection unit **5110** may collect information associated with a URL of a webpage input through web crawling.

The web collection unit **5110** collects all information related to a URL of a webpage, and generates a copy of the page or indexes the created page to rapidly perform processing.

The web collection unit **5110** of the embodiment may rapidly process a large amount of URL input data through parallel processing. For example, the web collection unit **5110** may rapidly and simultaneously process, in parallel, HTML information related to a URL input through one thread, JavaScript information in a webpage, media file information such as an image, and information about various files to be distributed by a webpage. A detailed example thereof will be disclosed below.

The data bundle unit **5120** may group and output various pieces of information processed in parallel by the web collection unit **5110**.

The analysis detection unit **5200** may analyze and detect data including a malicious action in a data bundle collected and processed by the data bundle unit **5120**. To this end, the analysis detection unit **5200** may include an antivirus unit (AntiVirus) **5210**, a de-obfuscator **5220**, a malware detection unit (YARA) **5230**, a data parser **5240**, an AI engine **5250**, and a data provision unit (Report) **5260**.

For example, the antivirus unit **5210** may analyze collected web data, and perform antivirus-based malware identification, for example, HTML code identification, in the collected data.

When data output by the data bundle unit **5120** is obfuscated, the de-obfuscator **5220** may de-obfuscate the data.

The malware detection unit **5230** may search for malware including a pattern or signature according to a certain rule, that is, a signature pattern of an attack tool or an attacker for malware analyzed and identified by the antivirus unit **5210** or data output by the de-obfuscator **5220**.

For example, the malware detection unit **5230** may detect and classify malware according to a rule such as YARA for input data.

The data parser **5240** may parse data according to de-obfuscation of the de-obfuscator **5220**.

The AI engine **5250** may determine whether data output by the malware detection unit **5230** or the data parser **5240** is malicious or normal based on a machine learning model.

The web collection unit **5110** of the disclosed embodiment may collect and process data related to webpages in parallel. Further, the analysis detection unit **5200** may identify whether or not data included in or related to a webpage is malicious by using a detection engine according to three detection steps (antivirus detection, signature-based malware detection, and AI-based detection) as described above.

Therefore, the embodiment may rapidly monitor webpage data and accurately identify whether or not the webpage data is malicious.

FIG. **46** is a diagram illustrating an operation of the web collection unit according to an embodiment.

As shown in an example of this figure, the web collection unit may collect webpage data while processing several threads in parallel in one processor.

54

The example of this figure represents an example in which the web collection unit collects data related to different webpages in parallel while performing four processes.

Process #1, process #2, process #3, and process #4 may each receive address information of a different webpage, for example, URL information.

In the example of this figure, when process #1 receives input of address information of a specific webpage (in this example, www.kisa.or.kr), a first collection/analysis thread of process #1 may distribute the addressed information of the input webpage and webpage address information according to lower depths of the webpage to other collection/analysis threads.

The example of this figure illustrates the case where **100** collection/analysis threads simultaneously collect information on a webpage and lower webpages thereof. A plurality of collection/analysis threads operating in parallel may perform in-memory processing of collecting and analyzing each piece of webpage data within a corresponding thread.

For example, each thread may sequentially receive and process data according to a webpage and depth using a dequeue (DeQ) and enqueue (EnQ) method, which is a circular queue method.

Therefore, among a plurality of collection/analysis threads that operates in parallel, a master or first thread may assign webpage analysis tasks to other threads according to depth information of the input webpage.

A collector of a collection/analysis thread may immediately access a webpage according to a queue request, load webpage data in an in-memory collector, and make an HTTP request for the webpage data. In addition, when the collection/analysis thread receives an HTTP response of the corresponding webpage data, the HTTP response may be analyzed by an analyzer in the in-memory processing.

In this case, when the HTTP response received by the analyzer of the collection/analysis thread includes information on a lower webpage, similar webpage data analysis may be performed by immediately distributing the information on the lower webpage to another thread.

The URL of the webpage input in this way may include another URL therein, and analysis may be performed by visiting an additional page according to included depth information.

Process #2, process #3, and process #4 are illustrated in this example. However, similarly, other processes may perform operations in a similar manner.

FIG. **47** discloses an example of storing and managing webpage data according to depth information of a disclosed embodiment.

In this figure, a relationship between a webpage according to an input URL and a webpage linked according to depth is illustrated.

Depth levels for a main webpage and lower webpages thereof are indicated as 0, 1, and 2, respectively. In this example, the main webpage of depth level 0 may include various links, references, script files, etc. therein.

A webpage of depth level 1 may be an HTML file connected to the link of the main webpage or files linked by the script files, respectively.

In this example, the HTML file of depth level 1 is connected to the link of the main webpage and includes link information of a first Java (JS) script file and link information of an image file (for example, logo.png). In addition, in this example, the Java script file of depth level 1 is linked to the script file of the main webpage.

55

Again, a webpage of depth level 2 includes a first Java (JS) script file and an image file linked to the HTML file of depth level 1.

In this way, when the URL information of the main webpage is input, URL information of depth information according to the number of links connected thereto may be stored and managed. In this case, the embodiment may normalize the URL information.

The embodiment may normalize, store, and manage a webpage and a linked webpage according to a link using a scheme of encoding only allowed characters in a host name of a Unicode string using a Punycode technique according to RFC 3492, etc.

FIG. 48 discloses an example of determining whether webpage data is malicious according to analysis of a plurality of steps or layers according to an embodiment.

According to an embodiment, data of a webpage collected by the web collection unit is temporarily stored in the data bundle unit 5120, and then it is determined whether or not the data is malicious according to analysis of several steps or layers of the analysis detection unit 5200.

In the example of this figure, the web collection unit 5110 may analyze and collect various types of data within a webpage. This example illustrates the case of collecting an HTML file, a JavaScript (JS) file, a VB script (VBS) file, an EXE executable file, etc. among various file types.

Various types of data in the webpage collected by the web collection unit 5110 may be stored in the data bundle unit 5120, and the memory buffer 5120 is illustrated as a type of data bundle unit 5120 in the example disclosed above.

Whether the various types of data stored in the memory buffer 5120 are malicious may be determined in several layers.

For example, the antivirus unit 5210 may detect previously known cyber threat information based on a data pattern. The antivirus unit 5210 may identify known web data, for example, HTML malware, based on a previously known antivirus engine.

The de-obfuscator 5220 de-obfuscates obfuscated data among data stored in the memory buffer 5120. For example, when obfuscated JavaScript is present in webpage data, the obfuscated JavaScript may be de-obfuscated.

The malware detection unit 5230 performs pattern-based malicious action detection on data that is stored in the memory buffer 5120 and de-obfuscated or transmitted from the antivirus unit 5210. The malware detection unit 5230 may detect data in a webpage based on a pattern according to, for example, a YARA rule, identify malicious and attacking tools in the data, and identify a signature pattern of an attacker.

The AI engine 5250 may determine whether data transmitted by the malware detection unit 5230 is malicious or normal based on an AI algorithm.

As in the disclosed example, by analyzing collected webpage data through several steps and layers, it is possible to more accurately detect and analyze cybersecurity threats for the webpage data.

Meanwhile, in the case of an executable file such as an EXE file included in a webpage, it is possible to identify whether the file is malicious, an attack technique, and an attack group in the manner described in FIGS. 16 to 32 or FIGS. 33 to 43.

In the case of a non-executable file included in a webpage, it is possible to identify whether the file is malicious, an attack technique, and an attack group in the manner described in FIGS. 44 to 60.

56

In an embodiment, when a malicious action is detected in a collected webpage, record data of the corresponding webpage may be provided to a user or administrator and stored in order to secure data.

For example, in an embodiment, when malicious data is detected in a specific webpage, an HAR format file of the webpage may be stored. Then, the administrator or security officer may perform additional analysis by including log data from the HAR format file of the stored webpage and ensure evidence for malicious detection.

An example of providing a monitoring result of a webpage to a user based on an HAR format file is illustrated below.

FIG. 49 illustrates a concept of analyzing webpage data and providing detected information according to an embodiment.

As disclosed above, webpage crawling of the web collection unit, and data analysis and malignancy detection of a webpage of the analysis detection unit may be sequentially performed.

When data of the webpage is determined to be normal as a result of malignancy detection, webpage data is collected by continuously crawling other webpages. In addition, when the data is determined to be malicious as a result of detection, relevant webpage data may be stored in a HAR format file by revisiting the corresponding webpage.

The HAR format file is a file that records, as log data, an interaction between a web browser and a site. Therefore, a data list recorded in the HAR format file includes all types of resource files of the webpage, records of HTTP requests and responses, and records of script files related to the webpage.

In an embodiment, a user or a cybersecurity officer may obtain record information such as a transaction related to a webpage as a result of webpage monitoring.

The user may reproduce record information of a webpage such as an HAR format file to check the record information of the webpage, and additionally analyze a malicious action or obtain basis data.

FIG. 50 discloses an example in which the above-disclosed embodiment operates on a computer.

As described above, the cyber security threat information processing apparatus including the data collection unit 5100 and the analysis detection unit 5200 may be driven in parallel in several computer nodes.

The figure illustrates the cyber threat information processing apparatus including a master node and a plurality of slave nodes.

A docker container may operate on an operating system of a cloud system of one master node 5710. Even though the data collection unit and the analysis detection unit illustrated above may be implemented as separate pieces of hardware, the data collection unit and the analysis detection unit may operate on the docker container in the example of this figure.

In such a case, applications operating on each docker container may perform the above-disclosed embodiment using resources of the cloud system.

The master node 5710 may include one or more docker containers and databases capable of performing the above-disclosed embodiment.

When operating in one docker container of the master node 5710, the data collection unit operating in a specific docker container may transfer webpage link information related to a collected webpage to other docker containers operating in the master node 5710 or slave nodes 5720. In addition, the master node 5710 may allocate tasks related to

57

monitoring of malignancy detection of a webpage to slave nodes in consideration of load balancing.

Based on the illustrated docker swarm, webpage monitoring systems operating in several hosts may be grouped and managed as one master-slave cluster system.

In this case, the master node 5710 of the cluster system may periodically transmit a heartbeat packet to the slave nodes 5720 to determine whether a server has failed.

The master node 5710 of the cluster system may check the status of the slave nodes 5720 to determine whether there is a failure of the server. Conversely, when the master node 5710 of the cluster system desires to expand the processing capacity of webpage monitoring, a docker image may be distributed to a new node and included in the cluster system.

As such, the master node 5710 of the cluster system may perform scale-out for webpage monitoring by performing registration and release of nodes in the cluster as in the disclosed example.

FIG. 51 discloses an embodiment of a method of processing cyber threat information included in a webpage.

A webpage is collected, and data included in the webpage or data linked according to link depth is classified (S5910). When webpages are collected and classified, the webpages may be processed in a parallel process according to several computer nodes, and may be performed in a docker container of each node according to scale-out of the computer nodes. Detailed examples thereof are disclosed in FIGS. 62, 63, and 66.

Whether the data included in the webpage or the linked data is malicious is detected on a plurality of layers (S5920).

The data included in the webpage refers to various data or files distributed by the webpage, such as HTML data, JavaScript data, and media files such as images and audio. The data linked to the webpage includes various types of data or files linked to the webpage. A detailed example thereof is disclosed in FIG. 48.

For example, in a first layer, cyber threat information may be detected according to the antivirus-based HTML data pattern for the data included in the webpage or the linked data.

For example, in a second layer, malware including a pattern or signature according to a certain rule, that is, cyber threat information according to a signature pattern of an attack tool or an attacker may be detected for the data included in the webpage or the linked data. When the data included in the webpage or the linked data is obfuscated, the data may be de-obfuscated. For example, in the case of obfuscated JavaScript, a de-obfuscation tool may be applied, and a signature pattern may be found according to a YARA rule, etc.

For example, in a third layer, whether cyber threat information such as malicious action data is included may be detected based on an AI algorithm for the data included in the webpage or the linked data.

Three detection steps for the data included in the webpage or the linked data may be performed in parallel or sequentially.

In the detection steps, in the case of the data included in the webpage or the linked data detected to be malicious, record data of the corresponding webpage is provided or stored (S5930).

Record data of a webpage may include record information of the webpage by reproducing webpage record information such as a HAR format file. Based on the recorded data, the user may additionally analyze the malicious action or obtain basis data.

58

Therefore, according to the disclosed embodiments, it is possible to detect and address malware not exactly matching data learned by machine learning and address a variant of malware.

According to the embodiments, it is possible to identify malware, an attack technique, and an attacker in a significantly short time even for a variant of malware, and furthermore to predict an attack technique of a specific attacker in the future.

According to the embodiments, it is possible to accurately identify a cyberattack implementation method based on whether such malware exists, an attack technique, an attack identifier, and an attacker, and provide the cyberattack implementation method as a standardized model. According to the embodiments, it is possible to provide information about malware, for which malware detection names, etc. are not unified or a cyberattack technique cannot be accurately described, using a normalized and standardized scheme.

In addition, it is possible to provide a means capable of predicting a possibility of generating previously unknown malware and attackers who can develop the malware, and predicting a cyber threat attack occurring in the future.

According to the embodiments, it is possible to more clearly detect and recognize different attack techniques or different attack groups generated according to differences in an execution process even when execution results of executed files are the same.

According to the embodiments, it is possible to identify cyber threat information, attack techniques, and attack groups for various file types included in a file even when the file is a non-executable file, not an executable file.

According to the embodiments, it is possible to monitor a webpage, identify a webpage including a malicious action or information, and furthermore, identify cyber threat information, an attack technique, and an attack group included in the webpage.

Therefore, according to the disclosed embodiments, it is possible to detect and address malware not exactly matching data learned by machine learning and address a variant of malware.

According to the embodiments, it is possible to identify malware, an attack technique, and an attacker in a significantly short time even for a variant of malware, and furthermore to predict an attack technique of a specific attacker in the future.

According to the embodiments, it is possible to accurately identify a cyberattack implementation method based on whether such malware exists, an attack technique, an attack identifier, and an attacker, and provide the cyberattack implementation method as a standardized model. According to the embodiments, it is possible to provide information about malware, for which malware detection names, etc. are not unified or a cyberattack technique cannot be accurately described, in a normalized and standardized scheme.

In addition, it is possible to provide a means capable of predicting a possibility of generating previously unknown malware and attackers who can develop the malware, and predicting a cyber threat attack occurring in the future.

According to the embodiments, it is possible to more clearly detect and recognize different attack techniques or different attack groups generated according to differences in an execution process even when execution results of executed files are the same.

According to the embodiments, it is possible to identify cyber threat information, attack techniques, and attack groups for various file types included in a file even when the file is a non-executable file, not an executable file.

Hereinafter, a more specific embodiment of determining whether collected webpage data is malicious will be disclosed.

When reference information providing a website, for example, URL information is acquired, HTML data may be obtained from webpage data of the URL.

In previous malignancy detection or analysis of HTML, the entire HTML data has been simply learned based on machine learning, and malignancy has been determined according to a frequency of a specific tag or a frequency of a specific character in HTML. Therefore, it has been difficult to verify a cause in HTML and a person inducing a specific malicious action.

The disclosed embodiment is capable of identifying a specific attack action and even identifying an attack group in HTML data in order to overcome this problem.

Webpage data includes HTML data describing the webpage, and the HTML data may describe content of the webpage using tags, which are various command sets.

For example, HTML data includes a bundle of tags including opening and closing of each tag in the data, and in this way, a tag bundle may constitute part of the HTML data.

Even though HTML supports slightly different tags for each web browser, HTML generally supports similar tags. Accordingly, the embodiment may detect and identify an attack action of an attacker with respect to described content included in a tag set.

For example, an attacker may perform an attack action by exploiting a function of an HTML tag of a webpage. When an attacker uses the same attack technique in the webpage, data described in the HTML tag of the webpage may appear similarly when analyzing cyber threat information.

An embodiment may identify whether a tag is a malicious tag or a malicious tag similar to the malicious tag based on similarity of a partial region in tag units of HTML data.

Hereinafter, a detailed embodiment thereof is disclosed.

FIG. 52 discloses an embodiment of a method of processing cyber threat information.

Webpage data is acquired based on link information, and tag structure information of the webpage data is analyzed (S6110). As an example of the tag structure information of the webpage data, a document object model (DOM) tree structure is illustrated below.

Data included in a tag area of the webpage data is converted into tag feature data according to the tag structure information (S6120). Depending on the tag structure information, in HTML data, data in tag units that may be modified by an attacker may be converted into tag feature data. A detailed embodiment of the tag feature data will be disclosed below.

Cyber threat information of data included in the tag area is acquired by learning the converted tag feature data (S6130). The tag feature data may be classified by a classification model of an AI model to identify an attack technique and an attack group for a malicious action in each tag part.

FIG. 53 illustrates structure information based on tags of HTML data as a method of processing cyber threat information according to an embodiment.

HTML data may be analyzed in tag units, and this figure is an example of a DOM Tree in tag units of HTML data. The Dom Tree is associated with depth according to the sequential order of tags, and may be an object or a node in units of tags. Therefore, when the DOM tree of the HTML data is obtained, an HTML structure thereof may be easily understood.

The example of this figure is an example of analyzing HTML data, and illustrates a DOM tree structure according to positions and depths of tags.

In the example of this figure, a tag `</html>` 5910 representing an end of a tag part surrounding the entire HTML document, an end `</title>` 5920 of a tag representing a name of the HTML document, an end `</body>` 5930 of a tag area representing a body of the HTML document, and an end `</script>` 5940 of a tag bundle representing a script in the HTML document are illustrated together with respective identification numbers.

In addition, each of an end `</h1>` 5950 of a tag bundle representing a heading of content, an end `</iframe>` 5960 of a bundle of tags for inserting content of nested browsing, that is, another HTML page into the document, and an end `</a>` 5970 of a tag area for generating hyperlink is illustrated in the body of the HTML document.

In this way, HTML data may be analyzed as information of a hierarchical structure, and separated into tag units allowing characteristics of the HTML data to be identified.

Here, an example of classifying HTML data according to a DOM tree is disclosed as an example of separating HTML data into tag areas allowing characteristics of the HTML data to be identified.

FIG. 54 discloses an example of obtaining feature information related to a cyber security threat from structure information based on a tag of HTML data as a method of processing cyber threat information according to an embodiment.

First, in order to facilitate the embodiment, an example of using a webpage having tag structure information in the figure disclosed above to obtain feature information related to a cybersecurity threat of the webpage is disclosed.

As in the example disclosed above, tag structure information of HTML data may be obtained according to DOM tree analysis.

Tag data obtained here is illustrated in a left section, and webpage data corresponding to each piece of tag data is illustrated in a right section.

According to the example disclosed above, `<body>`, `<image>`, `<iframe>`, `<a>`, `<script>`, or `</script>` is illustrated as a tag area or tag data included in the tag structure information of the HTML data.

In the example of this figure, text 5980 included in the tag `<body>` in the tag structure information is as follows as described in the figure.

```
onload="teclear()"
Background="ground.gif"
Link="#ff2ff"
Text="#ff0001e"
Link="#ff2ff"
```

Further, in this example, in the tag structure information, content included in a tag image `<image>` area may be a URL address where an image source is provided (`http://analytics.hosting24.com/do.php` in the example of this figure).

A user or an attacker may arbitrarily modify or add cyber threat information to the HTML data corresponding to each tag area included in the tag structure information.

Accordingly, in this case, data modified or arbitrarily modified by an attacker may be replaced with data for detection or analysis of cyber threat information.

Here, the data arbitrarily modifiable by a user or an attacker refers to values arbitrarily modifiable by the user except for HTML grammar, among HTML data, and refers to a URL address, a string value, etc. within a tag area.

61

According to the above example, the arbitrarily modifiable values among the HTML data may correspond to data in which a function (teclear() in the example of this figure), a URL address (http://analytics.hosting24.com/do.php in the example of this figure), a string (web hosting, etc. in the example of this figure), a variable name (weight in the example of this figure), etc. are modifiable by an attacker.

In the above example, the function teclear() may be replaced with data (for example, <func>) indicating the data is a function among the HTML data, and the URL address may be replaced with data (for example, <http><url><ext:php>) indicating that the data is a URL address among the HTML data.

Further, in the above example, the string (for example, web hosting) may be converted into or replaced with data (for example, <string>, etc.) indicating that the data is a specific character string among the HTML data, and the variable name (for example, height or width) may be converted into or replaced with data (for example, <name>, etc.) indicating that the data is a variable name among the HTML data.

In this way, when a replaceable part of HTML data is replaced according to a certain rule as described above, converted tag information may be converted into vectorized data as information representing cyber threat information.

FIG. 55 illustrates a process of processing and converting a part that may include cyber threat information, except for HTML grammar, in the HTML document illustrated above according to an embodiment.

According to an embodiment, HTML data according to URL information may be analyzed according to a tag area or tag data according to tag structure information.

In this example, when HTML data of a specific webpage is analyzed by a tag area or tag data according to tag structure information, each piece of tag data is located in a left column.

In this example, each piece of tag data 6110 may be sorted as <body>, <image>, <iframe>, <a>, <script>, or </script>.

Data corresponding to each piece of tag data in the HTML document is processed according to a certain rule as described above, which is shown in each preprocessing section 6120.

For example, data of a body part is processed as follows according to a conversion rule.

```
onload="<func>( );"
Background="<name>.gif"
Link="<hex>"
Text="<hex>"
Link="<hex>"
```

As in the example disclosed above, among the HTML data, a function included in the tag area may be converted into <func>(), a name of an image may be converted into <name>, and hexadecimal code included in a link or text may be converted into <hex>.

Further, a string is converted into <string>, a URL address is converted into <http><url>, and a variable name is converted into <name>. In this way, parts other than a part necessarily used in HTML grammar may be changed according to a certain format or principle, and rules converted here may be sufficiently changed by those skilled in the art.

Data of the preprocessing section 6120 is converted into normalized data of a certain length, and the normalized data may be converted into a fuzzy hash value.

62

In the example of this figure, a fuzzy hash section 6130 represents the result of converting the data of the preprocessing section 6120 processed according to a certain rule into a fuzzy hash value.

That is, a first row of the fuzzy hash section 6130 illustrates a fuzzy hash value obtained by converting data in the preprocessing section 6120 processed from the data in the <body> tag area among the HTML data.

A second row of the fuzzy hash section 6130 illustrates a fuzzy hash value obtained by converting data in the preprocessing section 6120 processed from the data in the <image> tag area among the HTML data.

In addition, a third row of the fuzzy hash section 6130 illustrates a fuzzy hash value obtained by converting data in the preprocessing section 6120 processed from the data in the <iframe> tag area among the HTML data.

In this way, each piece of data processed from data in a tag area among the HTML data may be normalized and then converted into a hash value applied to a fuzzy-based hash function.

As illustrated above, an extracted hash value may be converted into N-gram data and converted into tag feature data 61400 using a frequency count according to an M-byte pattern. Here, an example is disclosed in which a 2-gram technique is applied to an extracted hash value to perform conversion into tag feature data using a frequency count according to a 2-byte pattern.

Hereinafter, data capable of representing cyber threat information by converting each tag area according to tag structure information is referred to as tag feature data. That is, the tag feature data may be cyber threat feature information corresponding to a tag unit classified according to tag structure information.

Therefore, when a classification model is trained based on tag vector data, it is possible to determine malignancy thereof.

FIG. 56 is a diagram conceptually illustrating an example of a cyber threat information processing method according to an embodiment.

The embodiment may acquire webpage data, separate the webpage data according to tag structure information 6210 of the webpage data, and process the webpage data. The webpage data may be received as URL information or may be collected through web crawling.

In this example, a result of analysis based on tag structure information 6210 of input webpage data when the webpage data is input is conceptually displayed. For convenience of description, the same example as the above example is used as the tag structure information 6210.

According to the tag structure information 6210, HTML data may be converted according to a certain rule for each tag area or each piece of tag data, and the converted data may be normalized and converted into a hash value. HTML data converted into the hash value may be converted into tag feature data, which is N-gram data.

In this example, a result of converting HTML data corresponding to the tag area <a> of the tag structure information 6210 into tag feature data 6220 is illustrated.

The tag feature data 6220 may include data related to an attack action or pattern data thereof, except for grammar essential for HTML. Accordingly, the tag feature data 6220 may include data capable of identifying an attack action identifier or an attack group in cyber threat information.

The embodiment may train a tree-based classification model 6230 based on the tag feature data 6220. For example, whether the tag feature data 6220 is malicious may be classified by applying a random forest learning algorithm

using at least one decision tree **6245** to the input tag feature data **6220** based on a prepared tag feature database (DB) **6240**.

The tag feature DB **6240** stores data of a tag area included in HTML data as malicious or normal tag feature data according to malicious label information of a webpage. That is, data of a tag area in HTML including a malicious action is stored as malicious tag data in the DB, and data of a tag area in normal HTML is stored as normal tag data in the DB.

That is, according to a classification result of the tree-based classification model **6230** according to the embodiment, whether the tag feature data **6220** is malicious may be probabilistically determined (**6250**). Here, an example in which a probability that the data of the tag area <a> in the HTML document is malicious is determined to be 98% is disclosed.

Further, when the corresponding tag feature data **6220** is malicious, an attack technique identifier and an attacker group included in the tag feature data **6220** may be identified. In this example, an example of identifying an attack technique identifier referred to as Blackhole and an attacker group Lazarus for the tag feature data **6220** is disclosed.

Therefore, according to the embodiment, it is possible to identify which tag area in HTML is malicious as well as whether the HTML document included in the webpage data is malicious. In addition, rather than simply detecting or classifying HTML data as malicious based on machine learning or determining malignancy based on a frequency of a specific tag or a frequency of a specific character in HTML, it is possible to identify an attack technique and an attack group of specific tag data of HTML data. Therefore, accurate malicious detection and analysis are possible.

FIG. **57** is a diagram illustrating an example of an apparatus for processing cyber threat information included in a tag of a webpage according to an embodiment.

Another embodiment of the cyber threat information processing apparatus may include a server **2100** including a processor, a database **2200**, and an intelligence platform **10000**.

The database **2200** may store previously classified malware or a pattern code of malware.

The processor of the server **2100** may receive location information such as link information of a webpage through an application programming interface (API) **1100**.

A receiving module **18801** of a framework **18000** may receive the webpage data using the link information of the webpage received through the API according to an instruction of the processor of the server **2100**.

An analysis module **18803** may analyze the received webpage data based on link information of the webpage to obtain tag structure information for the webpage data. As an example of the tag structure information, a DOM tree structure is illustrated.

A conversion module **18805** may convert data included in the tag area of the webpage data into tag feature data according to the tag structure information of the webpage data. The conversion module **18805** may convert data of a part modifiable by the user, in addition to a part for an essential structure included in the webpage, into tag feature data in tag units according to the tag structure information.

A learning module **18807** acquires cyber threat information of data included in the tag area according to the tag structure information by applying a classification model to the tag feature data using the AI engine **1230**.

The learning module **18807** may classify the tag feature data by the classification model according to an algorithm of

the AI engine **1230** to identify an attack technique and an attack group for a malicious action in each tag part.

When the learning module **18807** classifies feature data such as the tag feature data, examples of the classification model are disclosed in detail in FIGS. **25** to **28** and **52** to **55**.

Therefore, according to the disclosed embodiments, it is possible to detect and address malware not exactly matching data learned by machine learning and address a variant of malware.

According to the embodiments, it is possible to identify malware, an attack technique, and an attacker in a significantly short time even for a variant of malware, and furthermore to predict an attack technique of a specific attacker in the future.

According to the embodiments, it is possible to accurately identify a cyberattack implementation method based on whether such malware exists, an attack technique, an attack identifier, and an attacker, and provide the cyberattack implementation method as a standardized model. According to the embodiments, it is possible to provide information about malware, for which malware detection names, etc. are not unified or a cyberattack technique cannot be accurately described, using a normalized and standardized scheme.

In addition, it is possible to provide a means capable of predicting a possibility of generating previously unknown malware and attackers who can develop the malware, and predicting a cyber threat attack occurring in the future.

According to the embodiments, it is possible to more clearly detect and recognize different attack techniques or different attack groups generated according to differences in an execution process even when execution results of executed files are the same.

Hereinafter, embodiments will be disclosed in which the intelligence platform according to the embodiments provides collected, discriminated or detected, and predicted cyber threat information to the user.

FIG. **58** illustrates an example of processing and providing cyber threat information according to the disclosed embodiments.

An embodiment of providing the cyber threat information may receive input of a file related to the cyber threat information or information related to the file through an intelligence platform including the API illustrated above. To this end, the intelligence platform may output an icon for receiving input of a file on a webpage. Alternatively, the cyber threat information may be collected by crawling webpage data on the Internet, as illustrated above.

This figure illustrates an intelligence platform capable of providing the cyber threat information to a user, and particularly illustrates a page of a web service provided by the intelligence platform.

The intelligence platform according to the disclosed embodiment may receive input of a file that may include cyber threat information or data of first cyber threat information for the file from the user.

The intelligence platform according to the embodiment may process the input file according to the examples disclosed above, and provide cyber threat information related to the input file to the user.

For example, the cyber threat information for the file may include (1) a possibility that the cyber threat information related to the input file is malicious, (2) a hash value of the input file, (3) a tag value related to the input file, (4) a type of input file, (5) a size of the input file, (6) an attack group intending to perform cyber threat using the input file, (7) an identifier of an attack technique related to the input file, (8) a type of risk related to the input file, (9) a country targeted

65

by the input file, (10) an industry targeted by the input file, (11) security vulnerability information related to the input file, etc.

Alternatively, the intelligence platform according to the embodiment may provide the user with second cyber threat information related to the input first cyber threat information.

As another example, the intelligence platform according to the embodiment may provide the second cyber threat information related to the provided first cyber threat information when the user provides the first cyber threat information as illustrated in (1) to (11).

In the example of this figure, the intelligence platform may receive input of a hash value in the form of MD5, SHA-1, or SHA-256 related to a specific file on a webpage.

For example, when the intelligence platform receives input of a hash value of a certain file from a user, the intelligence platform may provide the following information related to the file from the input hash value.

- (1) A possibility that cyber threat information related to the file for the input hash value is malicious,
- (3) a tag value related to the file for the input hash value,
- (4) a type of file for the input hash value,
- (5) a size of the file for the input hash value,
- (6) an attack group intending cyber threat using the file for the input hash value,
- (7) an identifier of an attack technique related to the file for the input hash value,
- (8) a type of risk related to the file for the input hash value,
- (9) a base country of an attack group related to the file for the input hash value or an attack target country,
- (10) an attack target industry of the file for the input hash value,
- (11) security vulnerability information related to the file for the input hash value, etc. may be provided.

As another example, when the platform receives an attack group intending cyber threat as in the item (6) or an identifier of the attack group from the user, the following information related to the input attack group may be provided.

- (1) A possibility that cyber threat information related to the input attack group is malicious,
- (2) a hash value of the file related to the input attack group,
- (3) a tag value related to the input attack group,
- (4) a type of file including cyber threat information related to the input attack group,
- (5) a size of the file including the cyber threat information related to the input attack group,
- (7) an identifier of an attack technique related to the input attack group,
- (8) a type of cyber risk related to the input attack group,
- (9) an attacking country initiating from the input attack group or an attack target country thereof,
- (10) an attack target industry related to the input attack group,
- (11) security vulnerability information of a file including cyber threat information related to the input attack group, etc. may be provided.

The intelligence platform illustrated above may provide cyber threat information related to the input information to the user upon receiving input of a file or information related to the file.

The intelligence platform illustrated above may provide the second cyber threat information related to the first cyber threat information upon receiving input of a file from the user or receiving input of the first cyber threat information related to the file.

66

The user may obtain cyber threat information by setting selections and ranges of conditions related to the cyber threat information in detail using a search icon **8010** such as File Search or a selection icon **8020** such as a detailed search button illustrated in the figure. In particular, even though only File Search (for example, MD5, SHA-1, and SHA-256 hash values) is illustrated in this figure, it is obvious that search may be performed based on an IP address and a domain. In this case, when search is performed based on the IP address, the intelligence platform may output a campaign list associated with the IP address. Here, the campaign list may include information on an attack group, an attack target country, and an attack target industry associated with the corresponding campaign, and Indicators of Compromise (IoC) associated with the corresponding campaign, which will be described later.

An interface of the illustrated intelligence platform may provide cyber threat information frequently queried by users or cyber threat information frequently generated recently or in a specific period by connecting the cyber threat information with a hash tag (#) so that the user can easily select cyber threat information.

The interface of the illustrated intelligence platform may combine and provide hashtags and main keywords related to the cyber threat information (**8040**).

This example discloses that the interface of the intelligence API platform provides the user with various main keywords related to the cyber threat information, such as #banker, #ransomware #trojan, #malware, #downloader, and #apt along with hash tags.

The example of this figure provides a means by which the user may obtain information on a main attack group G at present or in a certain period at the bottom of a search input tool.

The interface of the illustrated intelligence API platform may provide selection options **8050** that provide detailed information on the main attack group.

For example, the disclosed example displays selection options that provide information on an attack group G Lazarus, an attack group G Wizard spider, an attack group G Sandworm Team, an attack group G Gamaredon Group, an attack group G Kimsuky, etc. to the user.

The user may select an option of the corresponding attack group G to obtain detailed information on the selected attack group.

The disclosed embodiment may provide a search means allowing search for cyber threat information, receive input of a file or cyber threat information on the file from a user, and provide cyber threat information related to the input information.

The disclosed embodiment may provide a detailed search icon allowing the user to search for related cyber threat information according to a detailed condition for various cyber threat information, so that the user may obtain more accurate and detailed cyber threat information through the corresponding search tool or means.

The disclosed embodiment may provide cyber threat information selectable by the user in the form of a hash tag or as a selection option, and when the user selects information or an option of the corresponding hash tag, the disclosed embodiment may provide detailed cyber threat information related thereto.

With regard to information on an origin nation of an attack, the intelligence API platform or a server providing the platform may obtain location information of a C&C (Command & Control) server from data included in an executable file or a non-executable file. At this time, the

67

origin nation may correspond to information of a base country where an attack group related to the file is active. Here, it is possible to refer to a base country defined by MITRE ATT&CK as the base country where the attack group is active. For example, a North Korean attack group referred to as "Lazarus" may attack using servers located in other countries rather than having a C&C server in North Korea.

The intelligence API platform or the server providing the platform may obtain information on the origin nation from an IP address or domain information accessed when malware is first executed.

For example, the intelligence API platform or the server providing the platform may obtain country information from a country corresponding to a specific IP address or domain and/or language code in an executable or a binary file. In this case, since the country code includes country codes such as KR, CN, and US in Unicode format, information on the origin nation of the attack related to the cyber threat information may be obtained.

With regard to information on an attack target country (target nation), for example, the intelligence API platform or the server providing the platform may obtain nation information from data analyzed from target software related to an executable file or a non-executable file. The target software related to the executable or the non-executable file may include language-related code or location-related information.

With regard to information on an attack target industry, the intelligence API platform or the server providing the platform may obtain information on a related attack target industry from a database in which various industries related to target software are stored.

In the embodiment, the information on the attack target industry may be obtained from binary data of an executable file or a non-executable file related to cyber threat activity. For example, the information on the attack target industry may be obtained according to country code included in target software or industry code (go.kr, or.kr, etc.) included in an attack target domain address.

In this way, the intelligence platform provides a means for users to receive input of or search for data related to cyber threat information through a webpage.

FIG. 59 illustrates another example of processing and providing cyber threat information according to the disclosed embodiments.

The disclosed embodiments may provide an advanced search icon **8020** to the user so that the user may search for and receive accurate and detailed cyber threat information.

In an embodiment, the intelligence platform may provide a search function using various conditions to be described later in the form of a query through the advanced search icon **8020**. For example, when the user selects the advanced search icon **8020**, the intelligence platform may list various conditions to be described later as a search function and receive a search word therefor. At this time, a description of various conditions to be described later may be provided to the user on a webpage.

This figure illustrates various conditions for obtaining cyber threat information from the advanced search icon **8020**. Here, these conditions are illustrated as Advanced Search Guide.

As an example of the various conditions set in the advanced search icon **8020**, a malignancy condition (referring to a malignancy condition of a file determined by an illustrated AI algorithm, represented by AI here) **8021** expresses a probability that the cyber threat information

68

related to the input file is malicious in a score form. In this example, the AI algorithm disclosed above may be used to indicate whether the related cyber threat information is malicious or not as a score.

In this example, the user may find cyber threat information satisfying a condition that a number indicating malignancy determined by the AI algorithm, etc. is greater than or less than a specific range in the advanced search icon **8020**.

As another example of the various conditions set in the advanced search icon **8020**, tag information **8022** represents characteristic information of a file in a tag form. The user may input or select related tag information in the advanced search icon **8020** to find detailed cyber threat information related to the tag information input or selected by the user.

As another example of the various conditions set in the advanced search icon **8020**, file type information **8022** allows the user to select or input a type or format of an input file so that detailed cyber threat information related to a file type set by the user may be found. For example, the user may select exe_32 bit, pdf, doc, xls, etc. to obtain detailed cyber threat information related to the file type.

As another example of the various conditions set in the advanced search icon **8020**, file size information **8023** allows the user to select or input a size of an input file so that detailed cyber threat information for a file size range set by the user may be found. For example, the user may input a number indicating a file size or range to search for only cyber threat information that satisfies a related condition.

Among the set conditions of the advanced search icon **8020**, attack group information **8025** may set an attack group desired to be found by the user. Then, detailed cyber threat information related to the attack group input by the user is searched and provided to the user through the interface of the platform.

Among the set conditions of the advanced search icon **8020**, attack technique identifier information **8026** allows detailed cyber threat information related to an identifier or code of an attack technique set by the user to be searched. The interface of the illustrated platform may provide the user with the detailed cyber threat information related to the identifier or code of the input attack technique.

Among the set conditions of the advanced search icon **8020**, threat type information **8027** allows a threat type related to a malicious action to be received as input. The illustrated platform may provide detailed cyber threat information related to the threat type input by the user, for example, trojan, riskware, downloader, phishing, etc. through the interface.

Among the set conditions of the advanced search icon **8020**, attack target country information **8028** may enable the user to search for a cyberattack in progress in a country input by the user. The illustrated platform may provide detailed cyber threat information related to a user-designated country through the interface.

Among the set conditions of the advanced search icon **8020**, attack target industry information **8029** allows detailed cyber threat information related to an industry set by the user to be provided. For example, when the user desires to know information on a cyberattack related to an energy industry, the user may input information such as energy to the attack target industry information **8029**. Then, the illustrated platform may provide detailed cyber threat information related to an industry set by the user through the interface.

Among the set conditions of the advanced search icon **8020**, vulnerability information **8029** is information set to search for cyber threat information related to vulnerability

code set by the user. When the user inputs specific vulnerability code, the illustrated platform may provide detailed cyber threat information related to the input vulnerability code.

As described above, the embodiment may provide various cyber threat information desired by the user when the user performs advanced search or from other viewpoints.

Regardless of a file or information related to the file to be set, the user may obtain cyber threat information related to the file according to a range and condition of the set information.

Hereinafter, various examples will be disclosed in which such cyber threat information is provided through the interface of the intelligence platform.

FIG. 60 illustrates another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

This embodiment discloses an example of visually providing cyber threat information occurring locally or globally at a specific time or a current time **8060** to the user.

At this time, the intelligence platform may visually provide cyber threat information to the user based on a known attack group and all attack groups. Here, the known attack group corresponds to an attack group known by MITRE ATT&CK, and all attack groups includes known and unknown attack groups. Although not illustrated in this figure, the intelligence platform may provide a known attack group tab and an all-attack-groups tab on a webpage. Accordingly, the user may select the known attack group tab or the all-attack-groups tab provided by the intelligence platform and be provided with cyber threat information included in the known attack group and all attack groups, which will be described later.

This embodiment visually provides a current status of cyber threat information occurring in a specific region or worldwide based on a map at a specific time or the current time **8060** in the cyber threat information.

In this example, the specific time or the current time **8060** may be a UTC-based time (2023-03-10 05:10:06), or may indicate a local time (2023-03-10 14:10:06) of an attack target country or a country where an attack originates from an attack group.

The illustrated example of the cyber threat information may provide information indicating that a certain attack group in an attacking country is attacking a specific facility or industry in a specific attack target country by using a cyberattack based on a map.

In the illustrated cyber threat information, an attacking country, an attacking group, and an attack target country are expressed as icons or images in a separate information area **8070** and provided to the user.

As illustrated, the separate information area **8070** may display cyber threat information to be easily understood by the user by visually providing an attack group and an attack target country based on a map.

For example, the information area **8070** of this figure indicates that an attack group (Lazarus) in North Korea is attacking the US using an EXE-type executable file.

As another example, the information area **8070** of this figure may indicate that an attack group (Lorec53) in Russia is attacking Brazil using a PDF file.

The information area **8070** may enable the user to visually and easily understand cyberattack or cyber threat information displayed as an attack target country and an arrow indicating a direction of an attack by an attacking group on a map.

FIG. 61 illustrates another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

The disclosed cyber threat information may provide the user with a cyber threat action presented in chronological order.

The cyber threat information illustrated in this figure may indicate a time point **8080** when an attack action appears on a time axis. In addition, the illustrated cyber threat information may provide information on an attack group performing an attack at the time point **8080** when the attack action appears on the time axis, an attack target country, or an attack target industry **8090**.

The example of this figure illustrates that attack target countries attacked by a first attack group Pusikurac on Mar. 9, 2023 and an attack target industry and attack target countries attacked by a second attack group Lorec53 are provided as visual icons. The attack target industry may refer to an industry such as telecommunications or energy, and each of the attack target countries may be provided to the user as a country name or national flag.

Even when the user selects a different time point on the time axis, cyber threat information including an attack group which is active at the time point may be displayed, and the user may easily obtain information on recent cyberattack activities or cyberattack activities in the past by selecting a time point other than a time point on the time axis on a screen.

FIG. 62 illustrates another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

The disclosed embodiments may provide information for each attack type of a cyber threat action to the user. The embodiment of this figure discloses an example of providing the user with a type of cyber threat action at a specific time point.

The intelligence platform according to the embodiment may provide distribution information **8090** for each type or detailed information **8100** for each type of the cyber threat action at a specific time point.

In the cyber threat information, as the distribution information **8090** for each type at the specific time point, distribution information according to a plurality of cyberattack types such as downloader, Trojan, Worm, Backdoor, Stealer, etc. may be provided as information such as graphs, percentages, etc.

Accordingly, the user may check the progress of cyber threat types at the specific time point.

The intelligence platform according to the embodiment may provide the detailed information **8100** for each type of cyberattack to the user at the specific time point.

The user may select a specific attack type from among a plurality of cyberattack types on the interface and obtain detailed information for each attack type close to the specific time point among the corresponding attack types.

The example of this figure illustrates information provided when the user selects downloader as a first attack type at the specific time point.

The detailed information **8100** for each type of cyberattack may include a date of initial collection (illustrated based on UTC), AI discrimination malignancy (AI), an attack group, an attack target country, a hash value of an attack type, a file type according to an attack type, an attack target industry, etc. related to the first attack type (downloader) at the specific time point.

This example illustrates the date of initial collection (2023-02-26 16:03:02), the malignancy (100%), the attack

group (Wizard spider in Russia), the attack target country (Turkey), the hash value (F56055C ~), the file type (EXE file), etc. related to the first attack type (downloader) according to an attack collection schedule.

When the user selects Trojan as the first attack type at the specific time point, it is possible to provide information such as a date of initial collection (illustrated based on UTC), AI discrimination malignancy (AI), an attack group, an attack target country, a hash value of an attack type, a file type according to an attack type, an attack target industry, etc. for the attack type Trojan close to the specific time point.

FIG. 63 illustrates another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

Referring to (A) part of FIG. 63, the disclosed embodiments may provide information on an attack technique (Technique ID, T-ID) for each industry according to a cyber threat action. More specifically, it is possible to provide an industry list (for example, financial services, scientific technology, communication, and manufacturing) related to the cyber threat information of the above-described embodiment, and to further output an attack technique (T-ID) frequently used in the industry list. At this time, statistics may be expressed as a percentage (%) for each attack technique frequently used in each industry. In addition, when the user selects an attack technique (T-ID), the intelligence platform may additionally provide a malware list corresponding to the industry and attack technique.

Referring to (B) part of FIG. 63, the disclosed embodiments may provide information on an industry which is an attack target according to a cyber threat action.

Examples of attack target industry information may include distribution information 8110 of an attack target industry at a specific time point and detailed information 8120 of the attack target industry.

In the attack target industry information provided by the intelligence platform, the distribution information 8110 of the attack target industry indicates a distribution of attack target industries according to cyberattack actions occurring at the specific time point. This example illustrates that attack actions are underway at a distribution of telecommunications industry (50%), non-profit organization (33%), government agency (17%), and scientific technology (17%) at the corresponding time point.

In the attack target industry information, the detailed information 8120 of the attack target industry may include a date of initial collection (illustrated based on UTC), AI discrimination malignancy (AI), an attack group, an attack target country, a hash value of an attack type, a file type according to an attack type, etc. related to each attack target industry at the specific time point.

For example, when the user desires to know an attack action occurring in the telecommunications industry, the telecommunications sector may be selected from the detailed information 8120 of each target industry.

This example illustrates the date of initial collection of the attack action (2023-02-16 15:52:39, illustrated based on UTC), the malignancy (0%), the attack group (Mummy Spider), the attack target country (France), the hash value (4E6A6F ~), the file type (unknown), etc. in the telecommunications industry.

The user may obtain various information related to a cyber threat action based on an industry targeted for attack from the illustrated embodiment.

FIG. 64 illustrates another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

The disclosed embodiments may provide upper APT attack information 8130 according to a highest frequency among Advanced Persistent Threat (APT) attacks at a specific time point in the cyber threat information.

This example discloses an example in which the upper APT attack information 8130 is provided in a list format. However, the upper APT attack information 8130 may be displayed using other schemes.

An APT attack may be performed using various schemes such as a file (F), an IP address (IP), a URL address, an Internet domain (Domain), etc., and the illustrated upper APT attack information 8130 may provide detailed information including an attack group for each attack action.

In this example, the upper APT attack information 8130 may include a last activity time for each APT attack, an APT attack group, hash value information of a file related to the APT attack, IP address information and country information related to the APT attack, URL address information related to the APT attack, Internet domain information related to the APT attack, industry information targeted by the APT attack, and detailed information on the APT attack.

For example, in the case of the upper APT attack information 8130, which has a second highest frequency, information on a last activity time of the wizard spider attack group, which is the APT attack group, file hash values of the attack group, attack target industry information, and detailed information on the attack are provided. The detailed information may be displayed as a detailed information window through a separate selection means or icon, or the detailed information window may be overlapped and displayed.

The user may obtain attack information including an APT attack group to be cautious of depending on the time point and detailed information thereof from the upper APT attack information 8130 provided by the intelligence platform according to the embodiment.

FIG. 65 illustrates another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

The disclosed embodiments may provide IoC information 8140 related to a specific attack group in the cyber threat information.

In this example, latest upper IoC information 8140 related to the specific attack group is illustrated in the form of a list. However, the latest upper IoC information 8140 may be provided using other schemes.

For example, in the case of the IoC information 8140 related to the attack group Raspite, it is possible to provide the malignancy (AI) of the attack action determined by the AI algorithm according to the date of initial collection, the hash value related to the attack action, the file type related to the attack action, the attack target country, the attack target industry, and summary information related to the attack action.

This example illustrates that there are 76 files, 10 IP addresses, 0 URL, and 7 domains related to the attack group Raspite.

Accordingly, the user may obtain attack group-oriented IoC information over time from IoC information related to a specific attack group provided by the intelligence platform according to the embodiment.

FIG. 66 is a diagram disclosing an example of a cyber threat information processing method according to an embodiment.

A file or information on the file is received through at least one interface or received as input by the user (S8200).

Examples of the file or the information from the user are illustrated in FIGS. 58 and 59.

Cyber threat information related to the input file or information is processed (**S8210**).

An embodiment of processing the cyber threat information related to the input file or information has been disclosed in the above embodiments.

For example, examples of processing cyber threat information for an executable file are illustrated in FIGS. **1** to **16**, and examples of processing cyber threat information according to a logical structure of instructions in an executable file are illustrated in FIGS. **17** to **27**.

Examples of when the input file is a non-executable file or cyber threat information related to a non-executable file are illustrated in FIGS. **28** to **44**. In addition, examples of processing cyber threat information related to a webpage when the user inputs data related to the webpage are disclosed in FIGS. **45** to **57**.

The real-time-processed or pre-processed cyber threat information may be stored in a storage device of the intelligence platform.

The processed cyber threat information is provided to the user through a user interface (**S8230**).

Examples of the cyber threat information provided to the user are illustrated in FIGS. **60** to **65**.

The user may obtain various cyber threat information from the interface of the intelligence platform. The interface of the intelligence platform may visually provide the user with cyber threat information for each time or region, for example, cyber threat information occurring at a specific time or current time, regionally or globally.

The interface of the intelligence platform may provide the user with cyber threat actions appearing in chronological order or information of the cyber threat actions for each attack type.

The interface of the intelligence platform may provide the user with the upper APT attack information **8130** according to a frequency (for example, high frequency of occurrence) among APT attacks at a specific time point, the IoC information **8140** related to the specific attack group, etc. in the cyber threat information.

FIG. **67** is a diagram disclosing an example of an apparatus for processing cyber threat information according to an embodiment.

An embodiment of the apparatus for processing cyber threat information may include a server **2100** including a processor, a database **2200**, and an intelligence platform **10000**.

The processor of the server **2100** may analyze and provide cyber threat information by receiving various files or information related to the files through an API **1100** or collecting data through online web crawling, etc.

The processor of the server **2100** may process data that may be provided to the intelligence platform **10000**. An example in which the processor of the server **2100** or the intelligence platform **10000** processes data is as follows.

The intelligence platform **10000** may receive a file or cyber threat information related to the file from a client **1010** of a specific user through the API **1100**. For example, the user may input cyber threat information, such as an executable file, a non-executable file, or a hash value of the file, to the intelligence platform **10000**.

The server **2100** operating the intelligence platform **10000** may autonomously and directly collect various executable files or non-executable files of external websites through Internet connection.

The intelligence platform **10000** or the processor of the server **2100** operating the intelligence platform **10000** may analyze cyber threat information from files received from

the user or directly collected, and provide various information so that various users may efficiently recognize cyber-attacks.

An input file or cyber threat information related to the input file is processed by the processor of the server **2100** according to the embodiment disclosed above, and the processed cyber threat information is stored in the database **2200**.

Various processing modules **1211**, **1213**, **1215**, . . . , **1219** in a framework **1200** and an AI engine **1230** may process input files and information according to various embodiments.

For example, examples of processing cyber threat information for an executable file are illustrated in FIGS. **1** to **16**, and examples of processing cyber threat information according to a logical structure of instructions in an executable file are illustrated in FIGS. **17** to **27**.

When the input file is a non-executable file or cyber threat information related to a non-executable file, examples are illustrated in FIGS. **28** to **44**. In addition, when the user inputs data related to a webpage, examples of processing cyber threat information related to the webpage are disclosed in FIGS. **45** to **57**.

The database **2200** may store analyzed cyber threat information such as previously classified malware or malware pattern code.

The user interface of the intelligence platform **10000** provides the processed or stored cyber threat information to the user.

Examples of the cyber threat information provided by the intelligence platform **10000** through the user interface are illustrated in FIGS. **60** to **65**.

The user may obtain various cyber threat information from the user interface of the intelligence platform **10000**. For example, the interface of the intelligence platform may visually provide the user with cyber threat information for each time and region, for example, cyber threat information occurring at a specific time or current time, regionally or globally.

The user interface of the intelligence platform **10000** may provide the user with cyber threat actions appearing in chronological order or information of the cyber threat actions for each attack type.

The interface of the intelligence platform **10000** may provide the user with upper APT attack information according to a highest frequency among APT attacks at a specific time point, IoC information related to a specific attack group, etc. in the cyber threat information.

Accordingly, the user may obtain various cyber threat information in real time or in non-real time through the intelligence platform **10000**. Through the intelligence platform **10000**, the user may obtain detailed information related to an attack action, and detailed information on an attack group, an attacking country, and an attack target industry.

Hereinafter, an intelligence platform capable of providing cyber threat information to a user will be illustrated. In particular, an embodiment included in a web service provided by the intelligence platform will be described.

FIG. **68** discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

The disclosed embodiments may provide an APT attack information list **8300** for APT attacks at a specific time point in the cyber threat information.

In particular, the APT attack information list **8300** of the embodiment may include malware information that is

75

directly or indirectly utilized for an APT attack in a preset period (for example, the most recent 1 day, the most recent 3 days, the most recent 7 days, the most recent 14 days, etc. from a search date according to the above-described embodiment). At this time, the APT attack information list **8300** in the preset period may be displayed as a recent APT attack list.

As described above, the APT attack may be performed through various methods such as a file (F), an IP address (IP), a URL address, an Internet domain, etc., and attack information included in the APT attack information list **8300** may be classified based on an attack group (attacker).

More specifically, in an embodiment, the APT attack information list **8300** may include at least one of an APT attack information list included in a known attack group or an APT attack information list included in an unknown attack group. In the current drawing, as an example, the APT attack information list **8300** may include an APT attack information list **8310** included in the known attack group and an APT attack information list **8320** included in the unknown attack group.

As described above, the intelligence platform may collect detailed information on various APT attacks. The intelligence platform may classify the collected APT attacks into attack groups based on a predetermined rule. At this time, an attack group name is determined in a string format.

Accordingly, the known attack group is characterized in that the attack group of the corresponding APT attack information may be identified through the intelligence platform at the time when the APT attack information is collected. In particular, the name of the attack group may include a name determined through MITRE ATT&CK.

In an embodiment, the interface of the intelligence platform may selectively output the APT attack information list **8310** included in the known attack group and the APT attack information list **8320** included in the unknown attack group. Hereinafter, in this figure, a description will be given of the case where the interface of the intelligence platform is outputting the APT attack information list **8310** included in the known attack group.

In an embodiment, the APT attack information list **8300** may include at least one of initial collection date information, attack group information, AI information, hash value information, file type information, attack target country information, attack target industry information, or a summary icon. In particular, content included in the APT attack information list **8300** is only an example, and it is obvious that other items for indicating the APT attack information may be added, names of the corresponding items may be changed, and the items are not limited to the names.

In this figure, the APT attack information list **8300** may include initial collection date information **8331**, attack group information **8332**, AI information **8333**, hash value information **8334**, file type information **8335**, attack target country information **8336**, attack target industry information **8337**, and a summary icon **8338**.

In particular, in an embodiment, the attack group information **8332**, the attack target country information **8336**, and the attack target industry information **8337** may be grouped into one campaign, and thus may be located in the same column within the APT attack information list **8300**. For example, even though this figure illustrates that the attack group information **8332**, the attack target country information **8336**, and the attack target industry information **8337** are disposed in a second column, a fifth column, and a sixth column, respectively, the attack group information **8332**, the attack target country information **8336**, and the attack target

76

industry information **8337** may be disposed together in the second column, which will be described later.

Here, the first collection date information **8331** may include a date and time when malware directly or indirectly utilized in the corresponding APT attack is initially collected. As described above, since the malware directly or indirectly utilized in the corresponding APT attack has previously been identified through the intelligence platform, the APT attack information list **8300** may indicate the attack group information **8332** including the malware directly or indirectly utilized in the corresponding APT attack.

In addition, the AI information **8333** may indicate the above-described malignancy condition. More specifically, in a process of identifying, collecting, and analyzing a file directly or indirectly utilized in the APT attack, the intelligence platform may determine a degree of malignancy of the file directly or indirectly utilized in the APT attack as a score. At this time, the intelligence platform may determine the degree of malignancy of the file directly or indirectly utilized in the APT attack as a score using the above-described AI module. To this end, the AI module may learn a malware pattern, calculate a score indicating a degree of similarity between the file directly or indirectly utilized in the APT attack and malware, and express the score as a percentage. In general, the intelligence platform may determine the file as malware when the malignancy score is greater than or equal to 50. For example, in the case of a first file **8341**, the intelligence platform may determine the malignancy score of the first file **8341** as "0." On the other hand, in the case of a second file **8342**, the intelligence platform may determine the malignancy score of the second file **8342** as "100."

Hash value information **8350**, file type information **8360**, attack target country information **8370**, and attack target industry information **8380** have been described above.

Accordingly, the user has an advantage of being able to view specific content of APT attack information provided by the intelligence platform according to the embodiment in the form of a table. Specifically, since the APT attack information list **8300** includes all of first collection date information **8330**, attack group information **8340**, the hash value information **8350**, the file type information **8360**, the attack target country information **8370**, and the attack target industry information **8380** for specific malware directly or indirectly utilized in the APT attack within a preset period, there is an advantage in that the user may easily identify APT attack information on one page.

FIG. 69 discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

Unlike the above-described embodiment, in this figure, the APT attack information list **8320** included in the unknown attack group is described. In addition, a description overlapping with the above-described figures will be omitted.

In an embodiment, when information on malware directly or indirectly utilized in the corresponding APT attack is not determined as a known attack group in any one of steps of collection, identification, and analysis, the intelligence platform may determine that the information is included in the unknown attack group.

At this time, in the case of the unknown attack group, since the name of the attack group has not yet been determined, the intelligence platform may set a temporary name for the unknown attack group.

The intelligence platform needs to arbitrarily name the unknown attack group, and may combine three words by

collecting natural language dictionaries. At this time, the intelligence platform may generate words based on clustered characteristics through malware collection, classification, and clustering through the above-described embodiment, rather than combining any words for the unknown attack group.

In an embodiment, the temporary name of the unknown attack group may be determined as a combination of at least one of a first word, a second word, or a third word. For example, the first word may be determined in a first category, the second word may be determined in a second category, and the third word may be determined as a 3-digit number. Here, the first category may be a word representing an emotion, mood, or an action, and the second category may be a word representing an animal.

Accordingly, a temporary name of an unknown attack group of a third file **8343** may be determined as "Gratifying-Newt-757." Similarly, a temporary name of an unknown attack group of a fourth file **8344** may be determined as "Gratifying-Newt-757." This means that, even though the attack groups of the third file **8343** and the fourth file **8344** have not yet been determined through the intelligence platform, it can be seen that the third file **8343** and the fourth file **8344** are included in the same attack group.

Accordingly, the user may check detailed information similar to APT attack information included in the known attack group through the APT attack information list **8320** included in the unknown attack group.

Hereinafter, a description will be given of a screen provided to the user by the intelligence platform in the case of selecting the summary icon **8338** of any file among at least one file included in the APT attack information list **8310** included in the known attack group or the APT attack information list **8320** included in the unknown attack group, which is the above-described embodiment.

FIG. 70 discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

A summary page **8350** of APT attack information may be output as a pop-up window, and the pop-up window may include at least one of a file name, AI information, representative hash value information, hashtag (#) information, overview information, hash value information, threat type information, a pattern detection name, attack group information, or attack target country information for malware directly or indirectly utilized in the APT attack. At this time, it is obvious that items included in the summary page **8350** of the APT attack information may be changed or added to indicate the APT attack information.

In this figure, the pop-up window may include AI information, representative hash value information **8351**, hashtag (#) information **8352**, overview information **8353**, hash value information **8354**, threat type information **8355**, a pattern detection name **8356**, attack group information **8357**, and attack target country information **8358** for the file directly or indirectly utilized in the APT attack.

More specifically, the AI information of the file directly or indirectly utilized in the APT attack has been described above, and the representative hash value information **8351** may indicate one of an MD5 hash value, an SHA-1 hash value, or an SHA-256 hash value. At this time, the representative hash value information **8351** output at the top of the summary page **8350** may correspond to a hash value input by the user to search for APT attack information on the interface of the intelligence platform.

In addition, the hashtag (#) information **8352** may include at least one hashtag (#) collected through the above-de-

scribed embodiment in relation to the malware directly or indirectly utilized in the corresponding APT attack.

The overview information **8353** may indicate a first collection date and time of malware directly or indirectly utilized in the APT attack, a last active date and time, a file type, and a file size.

The hash value information **8354** may indicate MD5 hash value, SHA-1 hash value, and SHA-256 hash value information of malware directly or indirectly utilized in the corresponding APT attack.

The threat type information **8355** may indicate threat type information related to a malicious action. For example, it is possible to provide detailed cyber threat information related to trojan, riskware, downloader, phishing, etc.

The pattern detection name **8356** may indicate a pattern name of malware directly or indirectly utilized in the corresponding APT attack. The pattern detection name **8356** is configured in a form that can be parsed including various types of information for defining a pattern by a person creating the pattern. For example, the pattern detection name **8356** such as Gen: Variant Application.Babar. 17593 is a variant pattern referred to as Variant, and may be interpreted as having a unique number of 17593 as a pattern similar to Babar. That is, a system of the pattern detection name **8356** is significantly diverse and may be differently determined based on at least one of individual personality or a level.

As described above, the attack group information **8357** may indicate an attack group included in any one of the steps of collection, identification, and analysis of the corresponding file. At this time, the attack group information **8357** may indicate at least one attack group including the corresponding file. Further, in an embodiment, the corresponding file may be included in at least one attack group instead of one attack group. In this case, the attack group information **8357** may include content of at least one attack group.

The attack target country information **8358** may indicate an attack target country of malware directly or indirectly utilized in the corresponding APT attack. In particular, at least one country targeted by the malware directly or indirectly utilized in the APT attack may be included. In an embodiment, a percentage may be indicated based on an attack share of at least one country for the malware directly or indirectly utilized in the APT attack. For example, when the malware directly or indirectly utilized in the corresponding APT attack targets only the US as the attack target country, "US 100%" may be shown. In addition, in an embodiment, the information may be indicated through an icon on a world map instead of a string.

FIG. 71 discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

In an embodiment, the intelligence platform may provide an association graph **8360** for APT attack information.

The association graph **8360** for the APT attack information may indicate at least one of a campaign, an attack group, an attack technique, an attack target country, an attack target industry, CVE information (here, the CVE information corresponds to the vulnerability information described above), or IoC information for one file collected, identified, and analyzed through the above-described embodiment in a graph form. At this time, it is obvious that an item included in the association graph **8360** may be changed or added to better represent the APT attack information.

However, for convenience of description, in this figure, a description has been given of an example in which the association graph **8360** for the APT attack information indicates a campaign, an attack group, an attack technique,

an attack target country, an attack target industry, CVE information (here, the CVE information corresponds to the vulnerability information described above), and IoC information for one collected, identified, and analyzed file in a graph form.

Here, the campaign corresponds to a sub-concept of the attack group. The intelligence platform may first identify the campaign. The attack group is present, and campaigns conducted by the attack group are present in the attack group. For example, when the attack group is "kimsuky," C-1 (campaign attacking US), C-2 (campaign attacking South Korea), and C-3 (campaign attacking Brazil) are included in the attack group. At this time, campaigns such as campaign C-1, campaign C-2, and campaign C-3 may be individual campaigns or may be grouped into one attack group. For example, when past histories of campaign C-2 and campaign C-3 are compared, and there is association with an existing known attack group, the campaigns may be included as a sub-concept of the corresponding attack group. In addition, when campaign C-1 is individual, the campaign and the attack group may have a 1:1 concept. That is, logically, the campaign is present as a sub-concept of the attack group, and identification of the intelligence platform starts from the campaign.

In an embodiment, the campaign, the attack group, the attack technique, the attack target country, the attack target industry, the CVE information, and the IoC information for the APT attack information included in the association graph **8360** may be represented as icons.

In addition, the icons corresponding to the campaign, the attack group, the attack technique, the attack target country, the attack target industry, the CVE information, and the IoC information for the APT attack information included in the association graph **8360** may be mutually connected using arrow indicators. At this time, the arrow indicators may indicate relationships between the respective icons. To this end, description indicating the relationships between the respective icons may be added between the arrow indicators. For example, when a first campaign for the APT attack information indicates a first file directly or indirectly utilized in the APT attack, the association graph **8360** may output an arrow including text "indicator" from an icon **8361** corresponding to the first campaign to an icon **8362** corresponding to the first file.

In addition, when the user selects any icon included in the association graph **8360** (for example, when the user selects an icon using a mouse while checking the association graph **8360** using a PC), the intelligence platform may output a description **8363** corresponding to any icon. For example, when the user selects an icon **8364** corresponding to the second file, the association graph **8360** may output the description of the second file **8363** (for example, a hash value of the second file).

In addition, the intelligence platform may provide user experience (UX) interaction to the association graph **8360** for APT attack information. More specifically, as described above, the association graph **8360** may implement an icon corresponding to a campaign for APT attack information, an icon corresponding to an attack group, an icon corresponding to an attack technique, an icon corresponding to an attack target country, an icon corresponding to an attack target industry, an icon corresponding to CVE information, and an icon corresponding to IoC information as respective nodes. Thereafter, based on user input for selecting one node included in the association graph **8360** (for example, clicking and dragging a node using a mouse when checking the association graph **8360** using a PC), the intelligence plat-

form may change a shape of the association graph **8360** differently around the selected node.

Accordingly, the user may recognize a relationship between the campaign, attack group, attack technique, attack target country, attack target industry, CVE information, and IoC information for the APT attack information at once through the association graph **8360**.

FIG. **72** discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

In an embodiment, the intelligence platform may provide APT attack information tables **8371** and **8372** for each campaign included in the above-described association graph. The APT attack information tables **8371** and **8372** may each include at least one of attack group information, attack target country information, attack target industry information, attack technique information, CVE information, or IoC information. As described above, it is obvious that items included in the APT attack information tables **8371** and **8372** may be changed or added so that the APT attack information tables **8371** and **8372** better represent the APT attack information.

In an embodiment, the APT attack information tables **8371** and **8372** in this figure may each include attack group information, attack target country information, attack target industry information, attack technique information, CVE information, and IoC information.

Further, the APT attack information tables **8371** and **8372** may be classified based on different campaigns in the same attack group, and different attack groups may be classified based on the same campaign. Here, the campaign has been described above.

For example, the first APT attack information table **8371** may include an attack group "PKPLUG MUSTANG PANDA," attack target countries "KUWAIT," "TURKEY," "UNITED KINGDOM," "JORDAN," "ITALY," and "GERMANY," attack target industries "FINANCIAL-SERVICES," "GOVERNMENT," and "TELECOMMUNICATIONS SERVICES," and IoC information. Here, the IoC information may include file information, IP information, URL information, and domain information related to the corresponding attack group.

As another example, the second APT attack information table **8372** may include an attack group "PKPLUG MUSTANG PANDA," attack target countries "KUWAIT," "TURKEY," "UNITED KINGDOM," and "JORDAN," attack target industries "FINANCIAL-SERVICES," "GOVERNMENT," and "TELECOMMUNICATIONS SERVICES," and IoC information.

In the above-described embodiment, the first APT attack information table **8371** and the second APT attack information table **8372** illustrate the same attack groups. However, it is obvious that the first APT attack information table **8371** and the second APT attack information table **8372** may include information on different attack groups.

Since several files may be used in one APT attack, the intelligence platform may provide information associated with an APT attack (campaign) related to analysis information on one of files directly or indirectly utilized in the APT attack. Accordingly, the user may view all attack groups, attack target countries, attack target industries, attack techniques, CVE information, and IoC information related to one piece of malware directly or indirectly utilized in the APT attack at once. In particular, even when the user requests information on one file directly or indirectly utilized in the APT attack, the intelligence platform may

provide information on other malware associated with the requested file directly or indirectly utilized in the APT attack.

FIG. 73 discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

In an embodiment, the intelligence platform may provide a code information table **8380** for the file directly or indirectly utilized in the APT attack. At this time, the code information table **8380** may include at least one of information from a start offset to an end offset or information corresponding to attack techniques. At this time, it is obvious that items included in the code information table **8380** may be changed or added to better represent the file directly or indirectly utilized in the APT attack.

In an embodiment, the code information table **8380** for the file directly or indirectly utilized in the APT attack in this figure may include information **8380** from a start offset to an end offset and information **8382** corresponding to attack techniques.

In an embodiment, when the user selects a first row **8383** included in the code information table **8380** (for example, when the user clicks a point in the first row **8383**), the intelligence platform may further provide a detailed information table **8384** for the first row **8383**. Here, the detailed information table **8384** for the first row **8383** may include capacity (byte) information, opcode information, and operand information for offsets included in the start offset to the end offset of the first row **8383**. In addition, since the detailed information table **8384** for the first row **8383** includes individual information for all offsets included in the start offset to the end offset, scroll up and down is possible based on the number of offsets.

Since offsets are all different for each file, and operands and opcodes, which are function operators in a file, are all different, the user may check capacity information, opcode information, and operand information for offset information through the detailed information table **8384** for each attack technique.

In addition, in an embodiment, when the user selects a first attack technique **8385** (for example, when the user places a mouse cursor on the first attack technique **8385**), the intelligence platform may further output information **8386** on the first attack technique **8385**. Here, the information **8386** on the first attack technique **8385** may include a name of the first attack technique **8385**. For example, when the first attack technique **8385** is "T1027," the information **8386** on the first attack technique **8385** may include "Defense Evasion Obfuscated Files or Information." Here, the information **8386** on the first attack technique **8385** may refer to 14 tactics and 218 attack techniques classified by MITRE ATT&CK.

FIG. 74 discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

In an embodiment, the intelligence platform may further provide a similar information table **8390** for the above-described file directly or indirectly utilized in the APT attack. More specifically, the intelligence platform may provide information on n most similar (for example, 5) files for a preset period (for example, the last 3 months) for one file directly or indirectly utilized in the APT attack. At this time, the intelligence platform may determine the n most similar files for the one file directly or indirectly utilized in the APT attack using the above-described malware analysis methods.

In an embodiment, the similar information table **8390** may provide information on n most similar files in relation to the file directly or indirectly utilized in the APT attack. Here, the similar information table **8390** may include initial collection date information **8391**, attack group information **8392**, AI information **8393**, hash value information **8394**, file type information, attack target country information, and attack target industry information for n files directly or indirectly utilized in the APT attack. At this time, the hash value information **8394** may include information on at least one hashtag (#) and at least one attack technique in addition to the hash value.

Further, in an embodiment, the similar information table **8390** may further output a summary icon **8395** for outputting a summary page for each piece of file information included in the similar information table **8390**. When the user selects the summary icon **8395** corresponding to the first file in order to check the details of the first file included in the similar information table **8390**, the intelligence platform may provide a pop-up window for outputting a summary page for the first file, which has been described above.

FIG. 75 discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

In an embodiment, the intelligence platform may further provide a file list **8396** using the same attack technique as the aforementioned APT attack. More specifically, the intelligence platform may provide the list **8396** including at least n (for example, 5) files using the same attack technique for a predetermined period (for example, the last 3 months) for the file directly or indirectly utilized in the APT attack.

The file list **8396** using the same attack technique as that of the file directly or indirectly utilized in the APT attack may include at least one of initial collection date information, attack group information, AI information, hash value information, file type information, attack target country information, attack target industry information, or a summary icon for n files. The items included in the file list **8396** are used to identify a file using the same attack technique as that of the file directly or indirectly utilized in the APT attack, and it is obvious that the items included in the file list **8396** may be changed or added.

In an embodiment, the file list **8396** using the same attack technique as that of the file directly or indirectly utilized in the APT attack in the figure may include initial collection date information, attack group information, AI information, hash value information, file type information, attack target country information, attack target industry information, and a summary icon for n files.

At this time, the initial collection date information, the attack group information, the AI information, the hash value information, the file type information, the attack target country information, the attack target industry information, and the summary icon included in the file list **8396** using the same attack technique as that of the file directly or indirectly utilized in the APT attack have been described above in detail.

That is, when the user selects the summary icon corresponding to the first file in order to check the details of the first file included in the file list **8396** using the same attack technique as that of the file directly or indirectly utilized in the APT attack, the intelligence platform may provide a pop-up window for outputting a summary page for the first file, which has been described above.

In this way, the user has an advantage of being able to check other files related to the file directly or indirectly utilized in the APT attack in advance by checking the similar

information table **8390** for one file directly or indirectly utilized in the APT attack and the file list **8396** using the same attack technique.

FIG. **76** is a diagram disclosing an example of a method of processing cyber threat information according to the disclosed embodiments.

A file or information on the file is received from the user through the user interface (**S83000**).

Examples of the file or information received from the user are illustrated in FIGS. **58** and **59**.

Cyber threat information related to the received file or information is processed (**S83100**). An embodiment of processing the cyber threat information related to the received file or information has been disclosed in the above embodiments. For example, examples of processing cyber threat information for an executable file are illustrated in FIGS. **1** to **16**, and examples of processing cyber threat information according to a logical structure of instructions in an executable file are illustrated in FIGS. **17** to **27**. Examples of when the received file is a non-executable file or cyber threat information related to a non-executable file are illustrated in FIGS. **28** to **44**. In addition, examples of processing cyber threat information related to a webpage, when the user inputs data related to the webpage, are disclosed in FIGS. **45** to **57**. In this way, real-time-processed or pre-processed cyber threat information may be stored in the storage device of the intelligence platform.

The processed cyber threat information is provided to the user through the user interface (**S83200**).

Examples of cyber threat information provided to the user are illustrated in FIGS. **58** to **75**. Accordingly, the user may obtain various cyber threat information from the interface of the intelligence platform. In particular, the interface of the intelligence platform may provide an APT attack information list for APT attacks at a specific time point. At this time, when information on the file used by the corresponding APT is not determined as one of known attack groups in any one of steps of collection, identification, and analysis, the interface of the intelligence platform may determine that the file is included in an unknown attack group. Accordingly, the intelligence platform may individually provide the APT attack list included in the known attack group and the APT attack list included in the unknown attack group to the user. In addition, the intelligence platform may provide AI information, representative hash value information, hashtag (#) information, overview information, hash value information, threat type information, a pattern detection name, attack group information, attack target country information, etc. for the file directly or indirectly utilized in the APT attack through the interface.

FIG. **77** is a diagram disclosing an example of an apparatus for processing cyber threat information according to the disclosed embodiments.

An embodiment of the apparatus for processing the cyber threat information may include a server **2100** including a processor, a database **2200**, and an intelligence platform **10000**.

The processor of the server **2100** may analyze and provide cyber threat information by receiving various files or related information through an API **1100** or collecting data through online web crawling, etc.

The intelligence platform **10000** may receive a file or cyber threat information related to the file from a client **1010** of a specific user through the API **1100**. For example, the user may input cyber threat information, such as an executable file, a non-executable file, or a hash value of the file, to the intelligence platform **10000**.

The server **2100** operating the intelligence platform **10000** may autonomously and directly collect various executable files or non-executable files of external websites, etc. through Internet connection.

The intelligence platform **10000** or the server **2100** operating the intelligence platform **10000** may analyze cyber threat information from files received from the user or directly collected, and provide various information so that various users may efficiently recognize cyberattacks.

An input file or cyber threat information related to the input file is processed by the processor of the server **2100** according to the embodiment disclosed above, and the processed cyber threat information is stored in the database **2200**.

Various processing modules **1211**, **1213**, **1215**, . . . , **1219** in a framework **1200** and an AI engine **1230** may process input files and information according to various embodiments.

For example, examples of processing cyber threat information for an executable file are illustrated in FIGS. **1** to **16**, and examples of processing cyber threat information according to a logical structure of instructions in an executable file are illustrated in FIGS. **17** to **27**.

Examples of when the input file is a non-executable file or cyber threat information related to a non-executable file are illustrated in FIGS. **28** to **44**. In addition, examples of processing cyber threat information related to a webpage, when the user inputs data related to the webpage, are disclosed in FIGS. **45** to **57**.

The database **2200** may store analyzed cyber threat information such as previously classified malware or malware pattern code.

The user interface of the intelligence platform **10000** provides the processed or stored cyber threat information to the user.

Examples of the cyber threat information provided by the intelligence platform **10000** through the user interface are illustrated in FIGS. **58** to **75**.

More specifically, the user may obtain various cyber threat information from the user interface **20000** of the intelligence platform **10000**. For example, the user interface **20000** of the intelligence platform may visually provide the user with an APT attack information list for APT attacks at a specific time point.

In addition, the intelligence platform **10000** may classify the file directly or indirectly utilized in the APT attack based on the attack group in order to provide the APT attack information list for the file directly or indirectly utilized in the APT attack to the user. At this time, when the processed file such as the collected, identified, and analyzed file is the known attack group, the intelligence platform may output the file to a known attack group APT attack list, and when the processed file is the unknown attack group, the intelligence platform may output the file to an unknown attack group APT attack list.

In addition, the intelligence platform **10000** may provide AI information, representative hash value information, hashtag (#) information, overview information, hash value information, a threat type information pattern detection name, attack group information, and attack target country information for the file directly or indirectly utilized in the APT attack to the user through a summary page of APT attack information.

In addition, the intelligence platform **10000** may provide an association graph for APT attack information. The association graph for APT attack information may indicate a campaign, an attack group, an attack technique, an attack

85

target country, an attack target industry, CVE information (here, the CVE information corresponds to the vulnerability information described above), and IoC information for the file directly or indirectly utilized in the APT attack collected, identified, and analyzed through the above-described embodiment in a graph form. The intelligence platform may provide an APT attack information table for each campaign included in the aforementioned association graph.

In addition, the intelligence platform **10000** may provide a code information table for the file directly or indirectly utilized in the APT attack. The code information table may include information from a start offset to an end offset and information corresponding to attack techniques.

In an embodiment, the intelligence platform **10000** may further provide a similar information table for the above-described file directly or indirectly utilized in the APT attack. Here, the similar information table may include initial collection date information, attack group information, AI information, hash value information, file type information, attack target country information, and attack target industry information for n files.

In an embodiment, the intelligence platform **10000** may further provide a file list using the same attack technique as that of the file directly or indirectly utilized in the APT attack described above. The file list using the same attack technique as that of the file directly or indirectly utilized in the APT attack may include initial collection date information, attack group information, AI information, hash value information, file type information, attack target country information, attack target industry information, and a summary icon for n files.

Accordingly, the user may obtain various cyber threat information in real time or in non-real time through the intelligence platform **10000**. Through the intelligence platform **10000**, the user may obtain detailed information related to an attack action, or detailed information on an attack group, an attacking country, and an attack target industry.

FIG. **78** discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

In an embodiment, the intelligence platform may provide at least one piece of attack group information **8400** for the cyber threat information processed through the above-described embodiment. Here, the attack group information **8400** may include at least one of an attack group name, inflow route information, attack target country information, attack target industry information, or manufacturing tool or program information. At this time, the items included in the attack group information **8400** are items for describing the attack group, and it is obvious that items may be added or changed.

The attack group information **8400** in this figure may include an attack group name, inflow route information, attack target country information, attack target industry information, a manufacturing tool, and program information.

A description will be given by taking a first attack group **8401** as an example. According to the attack group information **8400** provided by the intelligence platform, the user may learn that a name of the first attack group **8410** is "APT30," and a related group name is "Override Panda," learn about a description of the first attack group **8410**, and learn that attack target countries targeted by the first attack group **8401** are Malaysia, Vietnam, Thailand, South Korea, India, the US, etc., and attack target industries correspond to "government," "national defense," "media," and "marine."

86

In addition, the originating country (for example, China) of the first attack group **8401** is displayed in the form of an icon so that the user may immediately learn a country from which the corresponding attack group originates.

Similarly, a description will be given by taking a second attack group **8402** as an example. According to the attack group information **8400** provided by the intelligence platform, the user may learn that a name of the second attack group **8402** is "APT29," and related group names are "IRON RITUAL," "IRON HEMLOCK," "NobleBaron," and "Dark Halo," and learn about a description of the second attack group **8402**, and attack target countries and attack target industries targeted by the second attack group **8402**.

Further, more detailed information provided by the intelligence platform on at least one piece of attack group information **8400** will be described later. To check more detailed information, the user may select the first attack group **8401**.

FIG. **79** discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

Hereinafter, a description will be given of an embodiment in which the user selects the first attack group from among at least one piece of attack group information described above.

The intelligence platform may provide information **8410** on the first attack group. The information **8410** on the first attack group may include at least one of a name of the first attack group, origin country information, a description, a related group name, attack target industry information, inflow route information, a manufacturing tool, program information, a related event, target organization information, or main attack target country information. At this time, the items included in the first attack group information **8410** are items for describing the attack group, and it is obvious that items may be added or changed.

The information **8410** on the first attack group described in this figure may include a name of the first attack group, origin country information, a description, a related group name, attack target industry information, inflow route information, a manufacturing tool, program information, a related event, target organization information, and main attack target country information. Content included in the information **8410** on the first attack group has been described above.

In an embodiment, the intelligence platform may display a location of an origin country and a location of an attack target country of the first attack group as separate icons on a world map to indicate the main attack target country information of the first attack group, and connect the origin country and the attack target country using an arrow indicator to show a relationship therebetween.

Further, in an embodiment, the intelligence platform may output a timeline icon **8411** corresponding to the first attack group together with the information **8410** on the first attack group. When the user selects the timeline icon **8411** corresponding to the first attack group, a timeline for the first attack group may be output, which will be described later.

Further, in an embodiment, the information **8410** on the first attack group provided by the intelligence platform may further include a visualization graph, which will be described later.

FIG. **80** discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

In an embodiment, the intelligence platform may provide a timeline **8420** for the attack group. At this time, the

intelligence platform may output at least one of a date (at least one of year, month, or day) or a summary of attack content on the timeline **8420**. As described above, the timeline **8420** for the attack group represents a history of attack attempts by the attack group in chronological order, and it is obvious that other items for describing the history may be added or changed.

The timeline **8420** of this figure may output a date (at least one of year, month, or day) and a summary of attack content.

Further, in an embodiment, the intelligence platform may arrange the timeline **8420** for the attack group in ascending or descending order based on at least one of dates.

The timeline **8420** for the attack group shows cyber threat actions for that attack group in chronological order. In order to examine the timeline **8420** for the attack group, the second attack group **8402** will be described as an example.

The intelligence platform may indicate content of attack attempts of the second attack group **8402** according to the time order on the timeline **8420**. In more detail, when the timeline **8420** is examined, first, the second attack group **8402** attempted a spear phishing attack that enticed users to click on a flash video containing a malicious executable file at a private research institution based in Washington DC in March 2014, attempted a spear phishing attack on a Pentagon email system in August 2015, and attempted spear phishing attacks targeting US-based think tanks and non-governmental organizations (NGOs) in 2016. In addition, the second attack group **8402** attacked the democratic national committee (DNC) in June 2016, attempted a spear phishing attack targeting personal email accounts of people belonging to the ministry of defense, the ministry of foreign affairs, and the labor party in January 2017, and attempted an attack targeting the Dutch government department to steal a secret government document in February 2017. In addition, the second attack group **8402** attempted to steal various information related to COVID-19 vaccine development in Canada, the US, and the UK in 2020, impersonated the United States Agency for International Development (USAID) to attack several organizations, including government agencies and human rights organizations in 2021, attacked a Microsoft 365 account to access foreign policy information of NATO countries in 2022, and attempted an attack on a diplomat and government using HTML smuggling and spear phishing techniques in January 2022.

The user may immediately learn attacks attempted by the attack group in chronological order through the timeline **8420** provided by the intelligence platform.

FIG. **81** discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

In an embodiment, the intelligence platform may provide a visualization graph **8430** for APT attack information. Hereinafter, the visualization graph **8430** provided by the intelligence platform will be described in detail. In particular, when one attack group is selected, the visualization graph **8430** may indicate an attack technique generally used by the corresponding attack group. More specifically, the intelligence platform may collect statistics of attack techniques obtained by analyzing malware associated with the corresponding attack group to extract attack technique information mainly used by the corresponding attack group, and display the attack technique information on the visualization graph **8430**.

In addition, the visualization graph **8430** may be output at the bottom of a summary page **8350** of the above-described APT attack information, or may be displayed as a part of a

detailed screen when the user selects APT attack information, or at the bottom of information on the attack group.

The intelligence platform may analyze an attack technique according to the above-described embodiment. At this time, the attack technique may be output based on an analysis result, and in particular, the attack technique may be output to correspond to a MITRE ATT&CK matrix.

More specifically, tactics and attack techniques used in one APT attack that has been analyzed may be classified into at least one of 14 tactics and 218 attack techniques by MITRE ATT&CK.

At this time, since it is not visually favorable to show all tactics and all attack techniques for one APT attack that has been analyzed in an embodiment, the present invention is characterized by providing a method of differently representing only tactics and attack techniques used in the analyzed APT attack.

In more detail, the visualization graph **8430** may output a block corresponding to an attack technique used in an APT attack by shading the block in a first color, and output a block corresponding to a tactic by shading the block in a second color.

For example, blocks corresponding to 12 attack techniques used in an APT attack may be shaded in the first color and output, and blocks corresponding to 5 tactics may be shaded in the second color and output.

In an embodiment, the first color is characterized by having a higher chroma than that of the second color. For example, the intelligence platform may output light red as the second color and dark red as the first color on the visualization graph **8430**. In this way, the intelligence platform may output a block corresponding to a tactic used for an APT attack by shading the block in light red, and output a block corresponding to a used attack technique by shading the block in dark red. That is, vertical blocks corresponding to tactics used for the analyzed APT attack may be shaded in light red and output, and among the blocks, blocks corresponding to attack techniques may be shaded in dark red and output.

Accordingly, a user using the visualization graph **8430** of the present invention has the advantage of being able to immediately recognize tactics and attack techniques used in the APT attack.

In an embodiment, the intelligence platform may output the total number of attack techniques used in the APT attack. For example, the intelligence platform may output **12** at the top of the screen as the total number of attack techniques used in the selected APT attack.

Further, in addition to expressing the tactics and attack techniques used in the APT attack as a matrix, the intelligence platform may output specific names for tactics, T-IDs (Technique IDs), attack techniques, and sub-attack techniques (Sub-Techniques) corresponding to the visualization graph **8430** through a table **8431** in one area outside the visualization graph **8430**.

FIG. **82** is a diagram disclosing an example of a method of processing cyber threat information according to the disclosed embodiments.

A file or information on the file is received from the user through the user interface (**S84000**).

Examples of the file or information received from the user are illustrated in FIGS. **58** and **59**.

Cyber threat information related to the received file or information is processed (**S84100**). An embodiment of processing the cyber threat information related to the received file or information has been disclosed in the above embodiments. For example, examples of processing cyber threat

information for an executable file are illustrated in FIGS. 1 to 16, and examples of processing cyber threat information according to a logical structure of instructions in an executable file are illustrated in FIGS. 17 to 27. Examples when the received file is a non-executable file or cyber threat information related to a non-executable file are illustrated in FIGS. 28 to 44. In addition, examples of processing cyber threat information related to a webpage when a user inputs data related to the webpage are disclosed in FIGS. 45 to 57. In this way, real-time-processed or pre-processed cyber threat information may be stored in the storage device of the intelligence platform.

The processed cyber threat information is provided to the user through the user interface (S84200).

Examples of the cyber threat information provided to the user are illustrated in FIGS. 58 to 81. Accordingly, the user may obtain various cyber threat information from the interface provided by the intelligence platform.

In particular, the interface of the intelligence platform may provide at least one piece of attack group information on cyber threat information processed through the above-described embodiment. Here, the attack group information may include an attack group name, inflow route information, attack target country information, attack target industry information, a manufacturing tool, and program information.

Further, the intelligence platform may provide information on the first attack group. The information on the first attack group may include a name of the first attack group, origin country information, a description, a related group name, attack target industry information, inflow route information, a manufacturing tool, program information, a related event, target organization information, and main attack target country information.

In addition, the intelligence platform may provide a timeline for an attack group. At this time, the intelligence platform may output a date (at least one of year, month, or day) and a summary of attack content on the timeline. Further, in an embodiment, the intelligence platform may arrange the timeline for the attack group in ascending or descending order based on at least one of dates.

In addition, the intelligence platform may provide a visualization graph for APT attack information. Here, the visualization graph may output a block corresponding to an attack technique used in an APT attack by shading the block in a first color, and output a block corresponding to a tactic by shading the block in a second color.

FIG. 83 is a diagram disclosing an example of an apparatus for processing cyber threat information according to the disclosed embodiments.

An embodiment of the apparatus for processing the cyber threat information may include a server 2100 including a processor, a database 2200, and an intelligence platform 10000.

The processor of the server 2100 may analyze and provide cyber threat information by receiving various files or related information through an API 1100 or collecting data through online web crawling, etc.

The intelligence platform 10000 may receive a file or cyber threat information related to the file from a client 1010 of a specific user through the API 1100. For example, the user may input cyber threat information, such as an executable file, a non-executable file, or a hash value of the file, to the intelligence platform 10000.

The server 2100 operating the intelligence platform 10000 may autonomously and directly collect various executable files or non-executable files of external websites through Internet connection.

The intelligence platform 10000 or the server 2100 operating the intelligence platform 10000 may analyze cyber threat information from files received from the user or directly collected, and provide various information so that various users may efficiently recognize cyberattacks.

An input file or cyber threat information related to the input file is processed by the processor of the server 2100 according to the embodiment disclosed above, and the processed cyber threat information is stored in the database 2200.

Various processing modules 1211, 1213, 1215, . . . , 1219 in a framework 1200 and an AI engine 1230 may process input files and information according to various embodiments.

For example, examples of processing cyber threat information for an executable file are illustrated in FIGS. 1 to 16, and examples of processing cyber threat information according to a logical structure of instructions in an executable file are illustrated in FIGS. 17 to 27.

Examples of when the input file is a non-executable file or cyber threat information related to a non-executable file are illustrated in FIGS. 28 to 44. In addition, when the user inputs data related to a webpage, examples of processing cyber threat information related to the webpage are disclosed in FIGS. 45 to 57.

The database 2200 may store analyzed cyber threat information such as previously classified malware or malware pattern code.

The user interface of the intelligence platform 10000 provides the processed or stored cyber threat information to the user.

Examples of the cyber threat information provided by the intelligence platform 10000 through the user interface are illustrated in FIGS. 58 to 81.

In particular, a user interface 20000 of the intelligence platform 10000 may provide at least one piece of attack group information on cyber threat information processed through the above-described embodiment. Here, the attack group information may include at least one of an attack group name, inflow route information, attack target country information, attack target industry information, a manufacturing tool, or program information.

Further, the intelligence platform 10000 may provide information on the first attack group. The information on the first attack group may include at least one of a name of the first attack group, origin country information, description, a related group name, attack target industry information, inflow route information, a manufacturing tool, program information, a related event, target organization information, or main attack target country information.

In addition, the intelligence platform 10000 may provide a timeline for an attack group. At this time, the intelligence platform may output at least one of a date (at least one of year, month, or day) or a summary of attack content on the timeline. Further, in an embodiment, the intelligence platform may arrange the timeline for the attack group in ascending or descending order based on at least one of dates.

In addition, the intelligence platform 10000 may provide a visualization graph for APT attack information. Here, the visualization graph may output a block corresponding to an attack technique used in an APT attack by shading the block in a first color, and output a block corresponding to a tactic by shading the block in a second color.

Accordingly, the user may obtain various cyber threat information in real time or in non-real time through the intelligence platform **10000**. Through the intelligence platform **10000**, the user may obtain detailed information related to an attack action, or detailed information on an attack group, an attacking country, and an attack target industry.

Cyber threats such as malware are becoming more advanced and high-tech day by day. In order to respond to such malware, an API interface provided by the intelligence platform of the present invention may form and provide a malware sample as a dataset for research.

Accordingly, the user has an advantage of being able to prevent malware to which new attack methods and bypass techniques are applied. In addition, the user may utilize the latest dataset to rapidly respond to the latest malware attack trends and global trend changes, and use the latest dataset to strengthen security.

Hereinafter, a description will be given of an interface for selling a malware dataset included in cyber threat information processed according to the above-described embodiment. The dataset package may include a sample file for malware, and the file included in the dataset package is based on the latest collection and analysis date. To this end, the intelligence platform may display a collection period and threat type (for example, Trojan, Ransomware, Backdoor, etc.) of the file included in the dataset package.

FIG. **84** discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

The intelligence platform may provide an interface that generates and sells the cyber threat information processed through the above-described embodiment as a dataset. Here, the cyber threat information may include executable malware and non-executable malware.

In an embodiment, the intelligence platform may sell an integrated malware dataset package **8500** through an interface. Here, the integrated malware dataset package **8500** may include an executable malware dataset package and a non-executable dataset package. At this time, the intelligence platform may set a ratio of the executable dataset to the non-executable dataset included in the integrated malware dataset package **8500**.

The intelligence platform may sell the dataset package at a different price based on the number of pieces of malware included in the integrated malware dataset package **8500** through an interface. At this time, the number of pieces of malware included in the integrated malware dataset package **8500** may be changed according to need of the user. For example, a first integrated malware dataset package **8501** includes 1,000 raw malware files. Among the 1000 files, the first integrated malware dataset package **8501** includes an EXE file, a DLL file, an ELF file, and an APK file as executable files and includes a PDF file, a DOC(X) file, an XLS(X) file, a PPT(X) file, and an HWP file as non-executable files. In addition, it is obvious that types of files included in the executable files and types of files included in the non-executable files may be changed according to the need of the user. Similarly, a second integrated malware dataset package **8502** may include 3,000 raw malware files, a third integrated malware dataset package **8503** may include 5,000 raw malware files, and a fourth integrated malware dataset package **8504** may include 10,000 raw malware files.

In this way, the user may purchase processed cyber threat information in the form of a dataset package through the interface provided by the intelligence platform. In particular,

by purchasing a dataset package including both executable malware and non-executable malware, the user may prevent malware to which new attack methods and bypass techniques are applied.

FIG. **85** discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

Unlike the above-described embodiment, the intelligence platform may provide an interface for selling an executable malware dataset package **8510**. That is, as described above, the intelligence platform may provide the package **8510** including an executable file malware dataset rather than an integrated malware dataset including both executable malware and non-executable malware.

In particular, an executable malware dataset provided by the intelligence platform is characterized by being cyber threat information processed according to FIGS. **1** to **16**. At this time, the executable malware dataset may include 32-bit and 64-bit based EXE files and DLL files running on the Windows operating system, and 32-bit and 64-bit based ELF files and SO files running on the Linux operating system. Here, the types of files included in the executable malware dataset are not limited to the example, and it is obvious that other executable malware files may be included.

In an embodiment, the intelligence platform may sell the dataset package at a different price based on the number of pieces of malware included in the executable malware dataset package **8510** through an interface, which may be understood with reference to the above description.

In this way, the user may purchase a dataset package including an executable file in processed cyber threat information through an interface provided by the intelligence platform.

FIG. **86** discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

Unlike the above-described embodiment, the intelligence platform may provide an interface for selling a non-executable malware dataset package **8520**. That is, the intelligence platform may provide the package **8520** including a non-executable malware dataset rather than the integrated malware dataset including both executable malware and non-executable malware or the executable malware dataset, as described above. Here, the non-executable malware dataset includes Hangul (HWP), Word (DOC(X)), Excel (XLS(X)), and PDF document files. In addition, the types of files included in the non-executable malware dataset are only examples, and are not limited thereto. Further, it is obvious that other non-executable malware files may be included in the non-executable malware dataset according to the need of the user.

The present invention discloses an embodiment in which the intelligence platform identifies and processes cyber threat information from the non-executable file as illustrated in FIGS. **28** to **44** to collect the non-executable malware dataset package **8520**. That is, the non-executable file requires a different analysis and processing process from that of the executable file since the non-executable file may be loaded and content thereof may be checked while an execution file (non-executable file reader program) capable of reading the file is executed unlike the executable file. In order to analyze such a non-executable file, in the above-described embodiment, processes of static analysis and dynamic analysis are performed. Thereafter, the intelligence platform may extract a hidden malicious payload in the non-executable file, a script that can execute the hidden malicious payload, etc. based on static analysis, and extract

information on a document form. In this way, the intelligence platform may determine whether the non-executable file is infected with malware. A detailed description thereof is disclosed in FIGS. 28 to 44.

In an embodiment, the intelligence platform may accurately detect malware included in the executable file as well as malware included in the non-executable file, and process cyber threat information on attack techniques and attack groups accordingly. Accordingly, the intelligence platform may sell the non-executable malware dataset package **8520** including processed cyber threat information to the user through the interface.

FIG. 87 is a diagram disclosing an example of a method of processing cyber threat information according to the disclosed embodiments.

A file or information about the file is received from the user through a user interface (**S85000**).

Examples of the file or information received from the user are illustrated in FIGS. 58 and 59.

Cyber threat information related to the received file or information is processed (**S85100**). An embodiment of processing the cyber threat information related to the received file or information has been disclosed in the above embodiments. For example, examples of processing cyber threat information for an executable file are illustrated in FIGS. 1 to 16, and examples of processing cyber threat information according to a logical structure of instructions in an executable file are illustrated in FIGS. 17 to 27. Examples when the received file is a non-executable file or cyber threat information related to a non-executable file are illustrated in FIGS. 28 to 44. In addition, examples of processing cyber threat information related to a webpage when a user inputs data related to the webpage are disclosed in FIGS. 45 to 57. In this way, real-time-processed or pre-processed cyber threat information may be stored in the storage device of the intelligence platform.

The processed cyber threat information is provided to the user through the user interface (**S85300**).

Examples of cyber threat information provided to the user are illustrated in FIGS. 58 to 83. Accordingly, the user may obtain various cyber threat information from the interface provided by the intelligence platform.

In an embodiment, the intelligence platform may sell the integrated malware dataset package to the user through the user interface. Here, the integrated malware dataset package may include an executable malware dataset package and a non-executable dataset package.

In addition, the intelligence platform may separately sell an executable malware dataset package and a non-executable dataset package, through the user interface. At this time, the executable malware dataset includes 32-bit and 64-bit based EXE files and DLL files running on the Windows operating system, and 32-bit and 64-bit based ELF files and SO files running on the Linux operating system, and the non-executable malware dataset includes Hangul (HWP), Word (DOC(X)), Excel (XLS(X)), and PDF document files. In particular, the files included in the non-executable malware dataset package are characterized by being cyber threat information processed through FIGS. 28 to 44.

FIG. 88 is a diagram disclosing an example of an apparatus for processing cyber threat information according to the disclosed embodiments.

An embodiment of the apparatus for processing the cyber threat information may include a server **2100** including a processor, a database **2200**, and an intelligence platform **10000**.

The processor of the server **2100** may analyze and provide cyber threat information by receiving various files or related information through an API **1100** or collecting data through online web crawling, etc.

The intelligence platform **10000** may receive a file or cyber threat information related to the file from a client **1010** of a specific user through the API **1100**. For example, the user may input cyber threat information, such as an executable file, a non-executable file, or a hash value of the file, to the intelligence platform **10000**.

The server **2100** operating the intelligence platform **10000** may autonomously and directly collect various executable files or non-executable files of external websites, etc. through Internet connection.

The intelligence platform **10000** or the server **2100** operating the intelligence platform **10000** may analyze cyber threat information from files received from the user or directly collected, and provide various information so that various users may efficiently recognize cyberattacks.

An input file or cyber threat information related to the input file is processed by the processor of the server **2100** according to the embodiment disclosed above, and the processed cyber threat information is stored in the database **2200**.

Various processing modules **1211**, **1213**, **1215**, . . . , **1219** in a framework **1200** and an AI engine **1230** may process input files and information according to various embodiments.

For example, examples of processing cyber threat information for an executable file are illustrated in FIGS. 1 to 16, and examples of processing cyber threat information according to a logical structure of instructions in an executable file are illustrated in FIGS. 17 to 27.

Examples of when the input file is a non-executable file or cyber threat information related to a non-executable file are illustrated in FIGS. 28 to 44. In addition, examples of processing cyber threat information related to a webpage, when the user inputs data related to the webpage, are disclosed in FIGS. 45 to 57.

The database **2200** may store analyzed cyber threat information such as previously classified malware or malware pattern code.

The user interface **20000** of the intelligence platform **10000** provides the processed or stored cyber threat information to the user through an online website (for example, malwares.com).

Examples of the cyber threat information provided by the intelligence platform **10000** through the user interface **20000** are illustrated in FIGS. 58 to 76. Accordingly, the user may obtain various cyber threat information from the user interface **20000** provided by the intelligence platform **10000**.

In an embodiment, the intelligence platform **10000** may provide the user with an interface for selling an integrated malware dataset package through the user interface **20000**. For example, the intelligence platform **10000** may sell a malware dataset package on a website such as malwares.com. Here, the integrated malware dataset package may include an executable malware dataset package and a non-executable dataset package.

In addition, the intelligence platform **10000** may separately sell an executable malware dataset package and a non-executable dataset package, not an integrated malware dataset package, through the user interface. At this time, the executable malware dataset includes 32-bit and 64-bit based EXE files and DLL files running on the Windows operating system, and 32-bit and 64-bit based ELF files and SO files running on the Linux operating system, and the non-execut-

able malware dataset includes Hangul (HWP), Word (DOC (X)), Excel (XLS(X)), and PDF document files. In particular, the files included in the non-executable malware dataset package are characterized by being cyber threat information processed through FIGS. 28 to 44.

In this way, the user may purchase processed cyber threat information in the form of a dataset package through the user interface 20000 provided by the intelligence platform 10000, preventing malware to which new attack methods and bypass techniques are applied.

The intelligence platform that provides information on an APT attack in the above-described manner may further provide a real-time intelligence line feed service.

More specifically, the intelligence platform may perform natural language processing on cyber threat information in real time, and provide the processed cyber threat information to the user. In this way, the user may download cyber threat information automatically collected and analyzed for each time and processed through the intelligence platform.

A specific embodiment will be described below.

FIG. 89 discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

The intelligence platform may provide an APT attack information list 8600 for APT attacks at a specific time point among pieces of cyber threat information processed through the embodiments described above with reference to FIGS. 68 to 77.

In an embodiment, the intelligence platform may provide information included in the APT attack information list 8600 in the form of a feed so that the information is delivered and easily understood by the user. At this time, the feed may be provided in the form of a single line.

To this end, the intelligence platform may convert the information included in the APT attack information list 8600 into metadata 8601 and store the metadata 8601. At this time, it is obvious that the metadata 8601 included in this figure is only an example and may be stored in other forms.

Thereafter, the intelligence platform may perform natural language processing on the information included in the APT attack information list 8600 based on the metadata 8601. To this end, the intelligence platform may utilize an AI engine.

More specifically, the intelligence platform may summarize the information included in the APT attack information list 8600 into one of words, sentences, and paragraphs based on language selected by the user in order to provide the information to the user. At this time, the intelligence platform may sequentially generate content based on information included in the metadata 8601 in natural language. That is, in the example of this figure, based on the metadata 8601 for information included in a first row of the APT attack information list 8600, the intelligence platform may generate natural language such as "At 4:03 pm on Feb. 26, 2023, an EXE file attack targeting China was detected from Wizard Spider, a Russian hacking group."

In the embodiment of this figure, the intelligence platform generates natural language by sequentially connecting an initial date of collection, file type information, AI information, hash value information, attack target country information, attack group information, and attack target industry information. However, it is obvious that the order may be changed. In an embodiment, the intelligence platform may generate natural language in a different order based on the importance of initial date of collection, attack group information, AI information, hash value information, file type information, attack target country information, and attack target industry information. For example, even on the basis

of the metadata 8601 for the information included in the first row of this figure, when the intelligence platform determines "attack group information" as the highest priority, "From Wizard Spider, at 4:03 pm on Feb. 26, 2023, an EXE file attack targeting China was detected." may be generated as natural language.

Thereafter, the intelligence platform may provide the generated natural language to the user in the form of a first feed 8602. The first feed 8602 may be output on a user terminal through a user interface provided by the intelligence platform. At this time, the intelligence platform may perform natural language processing on cyber threat information collected and analyzed in real time, and output a result to the user terminal and output an alarm at the same time.

In addition, the intelligence platform may generate natural language based not only on the APT attack information list 8600 described in this figure, but also on the information provided by the intelligence platform, which is the embodiment described with reference to FIGS. 58 to 67, or the attack group information described above with reference to FIGS. 78 to 83.

In this way, the user may check and respond to cyber threat information collected, analyzed, and processed through the intelligence platform in real time.

FIG. 90 discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

The intelligence platform may provide a user terminal 8603 with at least one feed generated through the above-described embodiment. More specifically, the intelligence platform may process cyber threat information through the above-described embodiment, and perform natural language processing on the processed cyber threat information so that the processed cyber threat information can be easily understood by a person. The intelligence platform may provide the cyber threat information subjected to natural language processing to the user terminal 8603 in the form of a feed.

This figure illustrates at least one of feeds 8604, 8605, 8606, and 8607 output from the user terminal 8603. Here, the at least one feed corresponds to the cyber threat information processed through the above-described embodiment and subjected to natural language processing by the intelligence platform.

Accordingly, the user terminal 8603 may sequentially output the at least one of the feeds 8604, 8605, 8606, and 8607. At this time, the first feed 8604, the second feed 8605, the third feed 8606, and the fourth feed 8607 output on the user terminal 8603 may correspond to natural language generated using the same or different methods. For example, the intelligence platform may generate the first feed 8604 based on information included in the above-described APT attack information list, and output the second feed 8605 based on information included in the above-described attack group information.

In particular, the at least one of the feeds 8604, 8605, 8606, and 8607 processed in this way does not simply include only time, and may include content of at least one of a probability that cyber threat information related to a file for an input hash value is malicious, a tag value related to the file for the input hash value, a type of file for the input hash value, a size of the file for the input hash value, an attack group intending to perform cyber threat using the file for the input hash value, an identifier of attack technique related to the file for the input hash value, a type of risk associated with the file for the input hash value, an attacking country or an attack target country starting from the file for the input hash

value, an attack target industry of the file for the input hash value, or information on security vulnerability related to the file for the input hash value. In particular, the at least one of the feeds **8604**, **8605**, **8606**, and **8607** is characterized in that the content is subjected to natural language processing to be easily understood by a person.

In addition, the intelligence platform may not perform natural language processing on all the content, and may select only a part based on the need of the user and generate the part in natural language. At this time, the intelligence platform may receive, from the user, input of an element to be generated in natural language in the content included in the APT attack information list.

In addition, the intelligence platform may provide the at least one of the feeds **8604**, **8605**, **8606**, and **8607** to the user terminal **8603** only for a preset time. For example, the intelligence platform may provide only cyber threat information initially collected and processed 10 minutes before a current time as the at least one of the feeds **8604**, **8605**, **8606**, and **8607**.

In this way, the user may easily understand and respond to cyber threat information collected, analyzed, and processed through the intelligence platform.

FIG. **91** is a diagram disclosing an example of a method of processing cyber threat information according to the disclosed embodiments.

A file or information on the file is received from the user through the user interface (**S86000**).

Examples of the file or information received from the user are illustrated in FIGS. **58** and **59**.

Cyber threat information related to the received file or information is processed (**S86100**). An embodiment of processing the cyber threat information related to the received file or information has been disclosed in the above embodiments. For example, examples of processing cyber threat information for an executable file are illustrated in FIGS. **1** to **16**, and examples of processing cyber threat information according to a logical structure of instructions in an executable file are illustrated in FIGS. **17** to **27**. Examples when the received file is a non-executable file or cyber threat information related to a non-executable file are illustrated in FIGS. **28** to **44**. In addition, examples of processing cyber threat information related to a webpage when a user inputs data related to the webpage are disclosed in FIGS. **45** to **57**. In this way, real-time-processed or pre-processed cyber threat information may be stored in the storage device of the intelligence platform.

The processed cyber threat information is provided to the user through the user interface (**S86200**).

Examples of the cyber threat information provided to the user are illustrated in FIGS. **58** to **86**. Accordingly, the user may obtain various cyber threat information from the interface provided by the intelligence platform.

Natural language processing is performed on the provided cyber threat information (**S86300**).

In an embodiment, the intelligence platform may provide the processed cyber threat information in the form of a feed. More specifically, the intelligence platform may perform natural language processing based on metadata of the processed cyber threat information so that the information can be easily understood by a person.

Thereafter, the intelligence platform provides the generated natural language to the user in the form of a feed (**S86400**).

In an embodiment, at least one feed may be output on a user terminal through a user interface provided by the intelligence platform.

FIG. **92** is a diagram disclosing an example of an apparatus for processing cyber threat information according to the disclosed embodiments.

An embodiment of the apparatus for processing the cyber threat information may include a server **2100** including a processor, a database **2200**, and an intelligence platform **10000**.

The processor of the server **2100** may analyze and provide cyber threat information by receiving various files or related information through an API **1100** or collecting data through online web crawling, etc.

The intelligence platform **10000** may receive a file or cyber threat information related to the file from a client **1010** of a specific user through the API **1100**. For example, the user may input cyber threat information, such as an executable file, a non-executable file, or a hash value of the file, to the intelligence platform **10000**.

The server **2100** operating the intelligence platform **10000** may autonomously and directly collect various executable files or non-executable files of external websites through Internet connection.

The intelligence platform **10000** or the server **2100** operating the intelligence platform **10000** may analyze cyber threat information from files received from the user or directly collected, and provide various information so that various users may efficiently recognize cyberattacks.

An input file or cyber threat information related to the input file is processed by the processor of the server **2100** according to the embodiment disclosed above, and the processed cyber threat information is stored in the database **2200**.

Various processing modules **1211**, **1213**, **1215**, . . . , **1219** in a framework **1200** and an AI engine **1230** may process input files and information according to various embodiments.

For example, examples of processing cyber threat information for an executable file are illustrated in FIGS. **1** to **16**, and examples of processing cyber threat information according to a logical structure of instructions in an executable file are illustrated in FIGS. **17** to **27**.

Examples of when the input file is a non-executable file or cyber threat information related to a non-executable file are illustrated in FIGS. **28** to **44**. In addition, when the user inputs data related to a webpage, examples of processing cyber threat information related to the webpage are disclosed in FIGS. **45** to **57**.

The database **2200** may store analyzed cyber threat information such as previously classified malware or malware pattern code.

The user interface **20000** of the intelligence platform **10000** provides the processed or stored cyber threat information to the user through an online website (for example, malwares.com).

Examples of the cyber threat information provided by the intelligence platform **10000** through the user interface **20000** are illustrated in FIGS. **58** to **86**. Accordingly, the user may obtain various cyber threat information from the user interface **20000** provided by the intelligence platform **10000**.

In an embodiment, the intelligence platform **10000** may provide the processed cyber threat information in the form of a feed. More specifically, the intelligence platform **10000** may perform natural language processing based on metadata of the processed cyber threat information so that the information can be easily understood by a person. Thereafter, the intelligence platform **10000** may provide the generated natural language to the user in the form of a feed. In an

embodiment, at least one feed may be output on a user terminal through the user interface 20000 provided by the intelligence platform.

FIG. 93 discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

In an embodiment, the intelligence platform may provide a related campaign list based on an IP address (for example, 178.63.254.36 in this figure). As described above, the intelligence platform may provide a search function for an IP address on a webpage. When the user inputs an IP address, the intelligence platform may provide a campaign list associated with the IP address based on the input IP address.

The associated campaign list may include information on at least one campaign associated with the IP address. At this time, an arbitrary identification number is attached as a campaign name (in this figure, Threat-30791e0f-2339-5b66-A308-Fdf781f36ba7, etc.) to identify the campaign.

That is, the associated campaign list may include at least one of an attack group, an attack target country, an attack target industry, a date when the corresponding IP address is identified in the campaign, a protocol, a tag, or a detection basis associated with each campaign.

In addition, IoC information for each campaign may be included, and the IoC information may include File information, IP information, URL information, and domain information, which will be described later.

FIG. 94 discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

In the above-described embodiment, the intelligence platform may provide details of the File information in the IoC information. More specifically, the details of the File information may include a file list associated with the aforementioned one campaign. At this time, the intelligence platform may provide a campaign name for the one campaign, an attack group, an attack target country, and an attack target industry associated with each campaign, and a file list associated with the corresponding campaign. The file list may provide information on n files associated with the corresponding campaign. Here, the information on the files may include AI scores, hash values, and file type information of the files.

FIG. 95 discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

In the above-described embodiment, the intelligence platform may provide details of the URL information in the IoC information. More specifically, the details of the URL information may include a URL list associated with the aforementioned one campaign. At this time, the intelligence platform may provide a campaign name for the one campaign, an attack group, an attack target country, and an attack target industry associated with each campaign, and a file list associated with the corresponding campaign. The file list may provide information on n URLs associated with the corresponding campaign. Here, the information on the URLs may include a date of last confirmation and a URL address.

FIG. 96 discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

In the above-described embodiment, the intelligence platform may provide details of the domain information in the IoC information. More specifically, the details of the domain information may include a domain list associated with the aforementioned one campaign. At this time, the intelligence platform may provide a campaign name for the one cam-

paign, an attack group, an attack target country, and an attack target industry associated with each campaign, and a file list associated with the corresponding campaign. The file list may provide information on n domains associated with the corresponding campaign. The information on the domains may include a date of first confirmation, a date of last confirmation, a domain address, a detection name, a threat type, a tag, and grounds of the domain.

FIG. 97 discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

In an embodiment, the intelligence platform may provide a related campaign list based on a domain (for example, asassass.autos in this figure). As described above, the intelligence platform may provide a search function for a domain on a webpage. When a user inputs a domain, the intelligence platform may provide a campaign list associated with the domain based on the input domain.

Based on the domain, the campaign list associated with the domain may include IoC information, and an embodiment provided by selecting File information, IP information, URL information, and domain information included in the IoC information has been described above. A description overlapping with the above description will be omitted.

FIG. 98 discloses another example of processing cyber threat information and providing the cyber threat information to the user according to the disclosed embodiments.

As described above, in the intelligence platform, the user may directly upload a file and request analysis of the uploaded file. At this time, the intelligence platform may provide file upload details and file analysis results as illustrated in this figure.

In an embodiment, the intelligence platform may set whether to open or close the file. The user may set whether to open or close the file when uploading the file.

At this time, when the uploaded file is set to be open, the intelligence platform may include an analysis result of the open file in a file search target of a webpage (for example, malwares.com).

On the other hand, when the uploaded file is set to be closed, the intelligence platform may not include an analysis result of the closed file in a file search target of a webpage or API. That is, in the case of a closed uploaded file, only a user uploading the file may receive an analysis result.

However, even in the case of a closed uploaded file, when the same file is found on an open web through web crawling according to the above-described embodiment, the file may be changed to an open file. Similarly, even in the case of a closed uploaded file, when the file is associated with another open campaign or attack group, the intelligence platform may publicly provide relevant information.

In this way, when requesting analysis of an internal document of a company or a document containing personal information, a file may be analyzed after being uploaded as a closed file, so that the user may maintain security of the internal document or personal information document.

What is claimed is:

1. A method of processing cyber threat information, the method comprising:
 - receiving a file or information on the file from a user through at least one interface,
 - wherein the file includes either an executable file or non-executable file;
 - processing cyber threat information on advanced persistent threat (APT) attacks related to the received file or the information on the file,

101

wherein when the received file is the executable file, a first cyber threat feature being extracted from one or more functions in the file is classified into a first attack technique identifier and a first attack group identifier, and

when the received file is the non-executable file, a second cyber threat feature is extracted from memory data in a suspended state of an application of the non-executable file at the time of executing the application in a kernel area, and the extracted second cyber threat feature is classified into a second attack technique identifier and a second attack group identifier, and

wherein the cyber threat information is generated based on the first or second attack technique identifier and the first or second attack group identifier;

and

performing natural language processing on the cyber threat information and providing a real-time intelligence line feed service on the APT attacks based on the natural language through a user interface.

2. The method according to claim 1, wherein the real-time intelligence line feed service includes a probability that the file is to be malicious.

3. An apparatus for processing cyber threat information, the apparatus comprising:

a database configured to store data; and

a server comprising a processor, wherein:

the server receives a file or information on the file from a user through at least one interface, wherein the file includes either an executable file or non-executable file, and

the processor:

processes cyber threat information on advanced persistent threat (APT) attacks related to the received file or the information on the file,

wherein when the received file is the executable file, a first cyber threat feature being extracted from one or more functions in the file is classified into a first attack technique identifier and a first attack group identifier, and

when the received file is the non-executable file, a second cyber threat feature is extracted from memory data in a suspended state of an application of the non-executable file at the time of executing the application in a kernel area, and the extracted second cyber threat feature is classified into a second attack technique identifier and a second attack group identifier, and

102

wherein the cyber threat information is generated based on the first or second attack technique identifier and the first or second attack group identifier;

and

performs natural language processing on the cyber threat information and provides a real-time intelligence line feed service on the APT attacks based on the natural language through a user interface.

4. The apparatus according to claim 3, wherein the real-time intelligence line feed service includes a probability that the file is to be malicious.

5. A non-transitory computer-readable storage medium storing a cyber threat information processing program that executes computer instructions for:

receiving input of a file or information on the file from a user through at least one interface,

wherein the file includes either an executable file or non-executable file;

processing cyber threat information on advanced persistent threat (APT) attacks related to the received file or the information on the file,

wherein when the received file is the executable file, a first cyber threat feature being extracted from one or more functions in the file is classified into a first attack technique identifier and a first attack group identifier, and

when the received file is the non-executable file, a second cyber threat feature is extracted from memory data in a suspended state of an application of the non-executable file at the time of executing the application in a kernel area, and the extracted second cyber threat feature is classified into a second attack technique identifier and a second attack group identifier, and

wherein the cyber threat information is generated based on the first or second attack technique identifier and the first or second attack group identifier;

and

performing natural language processing on the cyber threat information and providing a real-time intelligence line feed service on the APT attacks based on the natural language through a user interface.

6. The non-transitory computer-readable storage medium according to claim 5, wherein the real-time intelligence line feed service includes a probability that the file is to be malicious.

* * * * *